

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕНА К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по теме:
**РАЗРАБОТКА ПЛАТФОРМЫ ОНЛАЙН-КУРСОВ
ПРОГРАММИРОВАНИЯ НА СТОРОНЕ КЛИЕНТА С
АВТОМАТИЧЕСКОЙ ПРОВЕРКОЙ КОДА И ПЕРСОНАЛЬНЫМИ
РЕКОМЕНДАЦИЯМИ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Студент: _____ / Зернов Дмитрий Михайлович, 221-323/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Верецагин Владислав Юрьевич, к.т.н./
подпись *ФИО, уч. звание и степень*

Москва 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Тема ВКР	Разработка платформы онлайн-курсов программирования на стороне клиента с автоматической проверкой кода и персональными рекомендациями
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Создание программной системы для автоматизации процесса обучения программированию, включающего выполнение пользовательского кода в клиентской среде, его безопасную серверную проверку по формализованным тестовым спецификациям и персонализацию учебной траектории на основе анализа учебной активности.
Основные функции	1. Регистрация, аутентификация и разграничение прав доступа пользователей. 2. Предоставление теоретических материалов и учебного контента. 3. Выполнение практических заданий во встроенном редакторе кода. 4. Автоматическая проверка пользовательского кода в изолированной серверной среде. 5. Проведение тестирования и оценка результатов обучения. 6. Формирование персональных рекомендаций на основе анализа учебной активности. 7. Предоставление аналитической информации о прогрессе обучающихся. 8. Администрирование учебного контента и параметров системы.
Используемые технологии и платформы	Клиент: React, TypeScript, Vite, React Router, Axios, редактор кода на базе CodeMirror 6. Сервер: NestJS, Prisma ORM, PostgreSQL. Безопасность и исполнение кода: JWT, bcrypt, isolated-vm, linkedom. Контейнеризация и версионирование: Docker, Docker Compose, Git. Интеллектуальное сопровождение обучения: модуль ИИ-агентов.

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Проанализировать предметную область онлайн-обучения программированию, целевую аудиторию и существующие платформы-аналоги. 2. Исследовать теоретические основы построения адаптивных образовательных систем и современные подходы к интеллектуальным программным агентам, а также сформулировать функциональные и нефункциональные требования к разрабатываемой системе. 3. Спроектировать архитектуру платформы и модель данных с учётом требований к изоляции исполнения пользовательского кода и алгоритму формирования рекомендаций. 4. Реализовать клиентскую и серверную части платформы, включая встроенный редактор кода, систему маршрутизации и разграничение прав доступа. 5. Реализовать механизм автоматической проверки решений в изолированной серверной среде, формат тест-спецификаций и алгоритм формирования персональных рекомендаций. 6. Интегрировать в платформу модуль интеллектуальных агентов сопровождения обучения с использованием подходов подключаемых навыков и разработки по спецификации. 7. Провести функциональное тестирование, проверку качества программного кода и тестирование безопасности, выполнить развёртывание платформы в серверной среде и подготовить эксплуатационную документацию.
Состав технической документации	1. Пояснительная записка. 2. Техническое задание.
Состав графической части	1. Презентация к защите: 1 экз. 2. Схема процесса автоматической проверки решения учебного задания: 1 экз. 3. Диаграмма потоков данных между компонентами платформы: 1 экз. 4. Концептуальная ER-модель: 1 экз. 5. Даталогическая модель данных: 1 экз. 6. Обобщённая физическая модель данных: 1 экз. 7. Диаграмма компонентов клиентской и серверной

	частей: 1 экз. 8. Схема развёртывания эксплуатационной версии: 1 экз. 9. Макеты экранов пользовательского интерфейса: 10 экз.
--	--

ПЛАН РАБОТЫ НАД ВКР

Этапы	Недели семестра																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Анализ предметной области онлайн-обучения программированию и существующих образовательных платформ																		
Формирование функциональных и нефункциональных требований к системе																		
Проектирование архитектуры платформы и структуры базы данных																		
Разработка клиентской части платформы с механизмом выполнения кода																		
Разработка серверной части системы и механизма автоматической проверки кода																		
Реализация алгоритма персональных рекомендаций, а также механизмов аутентификации и разграничения доступа																		
Проведение функционального и нагрузочного тестирования системы																		
Развертывание платформы в серверной среде и подготовка технической документации																		

РУКОВОДИТЕЛЬ ОП:

«__» _____ 2026 / Даньшина Марина Владимировна /
подпись
ФИО, уч. звание и степень

РУКОВОДИТЕЛЬ ВКР:

«__»_____2026_____ / Верещагин Владислав Юрьевич, к.т.н. /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«__»_____2026_____ / Зернов Дмитрий Михайлович, 221-323 /
подпись *ФИО, группа*

АННОТАЦИЯ

Наименование работы: разработка платформы онлайн-курсов программирования на стороне клиента с автоматической проверкой кода и персональными рекомендациями.

Цель работы заключается в разработке платформы онлайн-курсов программирования на стороне клиента. Платформа должна поддерживать интерактивное выполнение кода в клиентской среде, автоматическую проверку решений в изолированной серверной среде и персональные рекомендации на основе анализа учебной активности.

Для достижения поставленной цели в работе необходимо решить следующие задачи:

- а) проанализировать предметную область онлайн-обучения программированию, целевую аудиторию и существующие платформы-аналоги;
- б) исследовать теоретические основы построения адаптивных образовательных систем и современные подходы к интеллектуальным программным агентам, а также сформулировать функциональные и нефункциональные требования к разрабатываемой платформе;
- в) спроектировать архитектуру платформы и модель данных с учётом требований к изоляции исполнения пользовательского кода и алгоритму формирования рекомендаций;
- г) реализовать клиентскую и серверную части платформы, включая встроенный редактор кода, систему маршрутизации и разграничение прав доступа;
- д) реализовать механизм автоматической проверки решений в изолированной серверной среде, формат тест-спецификаций и алгоритм формирования персональных рекомендаций;
- е) интегрировать в платформу модуль интеллектуальных агентов сопровождения обучения с использованием подходов подключаемых навыков и разработки по спецификации;

ж) провести функциональное тестирование, проверку качества программного кода и тестирование безопасности, выполнить развёртывание платформы в серверной среде и подготовить эксплуатационную документацию.

Работа состоит из введения, трёх глав, заключения, списка использованных источников и трёх приложений. Общий объём работы составляет 146 страниц, включая 21 рисунок, 13 таблиц и 12 листингов программного кода; объём основной части (введение, три главы и заключение) составляет 98 страниц. Список использованных источников содержит 73 наименования.

Во введении обоснована актуальность разработки, сформулированы цель и задачи, определены объект и предмет исследования, раскрыты теоретическая и практическая значимость работы и её научная новизна.

В первой главе описана предметная область и проанализированы платформы-аналоги. Рассмотрены адаптивные образовательные системы и интеллектуальные программные агенты. Обоснован архитектурный подход и сформулированы функциональные и нефункциональные требования к платформе.

Во второй главе приведены архитектура платформы и модель данных, проектирование и реализация серверной и клиентской частей, механизм безопасного исполнения кода и алгоритм персональных рекомендаций. Отдельно описаны агенты сопровождения обучения и подходы подключаемых навыков и разработки по спецификации.

В третьей главе рассмотрены безопасность агентов и работа с языковыми моделями, средства качества кода, результаты тестирования и обзор информационной безопасности. Описана апробация на учебном занятии и развёртывание в серверной среде.

В заключении представлены выводы по поставленным задачам, оценено соответствие достигнутого результата сформулированной цели и определены направления дальнейшего развития платформы.

В приложениях приведены макеты пользовательского интерфейса, модели данных и схемы процессов, а также листинги ключевых программных модулей.

Практический результат работы представлен функционирующей платформой в серверной среде для образовательного использования. Итоги апробированы на учебном занятии с обучающимися. Корректность сценариев проверена функциональным тестированием. Выполнен обзорный контроль информационной безопасности.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	12
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ	15
1.1 Анализ онлайн-обучения программированию	15
1.2 Анализ существующих платформ-аналогов	18
1.3 Подходы к построению адаптивных образовательных систем . . .	20
1.4 Интеллектуальные программные агенты в образовательных системах	21
1.5 Особенности автоматической проверки кода и требования к безопасности исполнения	23
1.6 Обоснование выбора архитектурного подхода и стека технологий	27
1.7 Формирование требований к разрабатываемой платформе	31
1.8 Выводы	38
2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ПРОГРАММНОЙ ПЛАТФОРМЫ	40
2.1 Проектирование архитектуры платформы	40
2.2 Проектирование модели данных	43
2.3 Проектирование серверной части	48
2.4 Проектирование клиентской части	54
2.5 Проектирование архитектуры агентов сопровождения обучения .	59
2.6 Применение подходов подключаемых навыков и разработки по спецификации	64
2.7 Реализация серверной части	68
2.8 Реализация клиентской части	71
2.9 Реализация модуля безопасного исполнения программного кода .	75
2.10 Реализация алгоритма формирования персональных рекомендаций	80
2.11 Выводы	84

3	ОБЕСПЕЧЕНИЕ КАЧЕСТВА, ТЕСТИРОВАНИЕ, БЕЗОПАСНОСТЬ И РАЗВЁРТЫВАНИЕ ПЛАТФОРМЫ	85
3.1	Безопасность интеллектуальных агентов и работа с языковыми моделями	85
3.2	Качество программного кода и инструменты статического анализа	90
3.3	Функциональное тестирование системы	91
3.4	Тестирование безопасности системы	94
3.5	Апробация платформы с участием обучающихся	96
3.6	Развёртывание платформы в серверной среде	104
3.7	Выводы	106
	ЗАКЛЮЧЕНИЕ	107
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	110
	ПРИЛОЖЕНИЕ А	117
	ПРИЛОЖЕНИЕ Б	126
	ПРИЛОЖЕНИЕ В	134
	ПРИЛОЖЕНИЕ Г	142

ВВЕДЕНИЕ

Современная система высшего и непрерывного образования всё острее реагирует на необходимость массовой и при этом индивидуализированной подготовки специалистов в области информационных технологий [1–3]. Дистанционное обучение программированию опирается на практику написания и проверки кода. При недостатке оперативной обратной связи и объективной верификации решений снижаются мотивация обучающегося и качество усвоения материала [4]. Рост числа образовательных сервисов и конкуренция между ними сопровождаются типовой линейной моделью курса, слабо учитывающей различия в подготовке и характере ошибок, что ограничивает эффективность обучения [5, 6]. Параллельно остаётся нерешённой инженерная задача безопасной автоматической проверки пользовательского программного кода: исполнение недоверенного кода в инфраструктуре платформы требует изоляции, ограничений ресурсов и формализованных тестовых сценариев. Без таких механизмов сервис работает нестабильно, а масштабирование затрудняется [7].

Развитие адаптивных образовательных систем позволяет опираться на данные учебной активности и формировать индивидуальную траекторию [8]. Правило-ориентированные методы сохраняют интерпретируемость рекомендаций для обучающегося. Параллельно распространяются интеллектуальное сопровождение и большие языковые модели для разъяснения теории, разбора ошибок и генерации учебных артефактов. Для управляемости поведения модели в инженерной практике используют архитектуру с подключаемыми навыками и спецификационно-ориентированную разработку [9–13]. Клиент-серверная платформа, объединяющая изолированную проверку кода, прозрачную адаптацию и регламентированное интеллектуальное сопровождение, приближает массовый онлайн к качеству индивидуального сопровождения преподавателем при соблюдении требований информационной безопасности и сопровождаемости продукта.

Целью выпускной квалификационной работы является разработка платформы онлайн-курсов программирования на стороне клиента с автоматической проверкой кода и персональными рекомендациями.

Для достижения поставленной цели в работе необходимо решить следующие задачи:

а) проанализировать предметную область онлайн-обучения программированию, целевую аудиторию и существующие платформы-аналоги;

б) исследовать теоретические основы построения адаптивных образовательных систем и современные подходы к интеллектуальным программным агентам, а также сформулировать функциональные и нефункциональные требования к разрабатываемой платформе;

в) спроектировать архитектуру платформы и модель данных с учётом требований к изоляции исполнения пользовательского кода и алгоритму формирования рекомендаций;

г) реализовать клиентскую и серверную части платформы, включая встроенный редактор кода, систему маршрутизации и разграничение прав доступа;

д) реализовать механизм автоматической проверки решений в изолированной серверной среде, формат тест-спецификаций и алгоритм формирования персональных рекомендаций;

е) интегрировать в платформу модуль интеллектуальных агентов сопровождения обучения с использованием подходов подключаемых навыков и разработки по спецификации;

ж) провести функциональное тестирование, проверку качества программного кода и тестирование безопасности, выполнить развёртывание платформы в серверной среде и подготовить эксплуатационную документацию.

Актуальность проекта обусловлена необходимостью сочетать на одной платформе безопасное исполнение и проверку учебного кода, персонализацию траектории и интеллектуальное сопровождение при ограниченных

ресурсах образовательной организации. Это позволяет сократить долю ручной проверки элементарных решений, повысить предсказуемость нагрузки на серверную инфраструктуру и повысить удовлетворённость обучающихся за счёт своевременной обратной связи.

Целевой аудиторией разрабатываемой платформы являются обучающиеся, осваивающие программирование в дистанционном формате, преподаватели, заинтересованные в автоматизации контроля знаний и анализе прогресса, а также образовательные организации, использующие цифровые инструменты для повышения эффективности учебного процесса.

Практическая значимость в том, что на платформе сочетаются клиентская часть со встроенным редактором кода, серверная часть с изолированной проверкой по тест-спецификациям, персональные рекомендации и интеллектуальное сопровождение с журналированием обращений к языковой модели. Это поддерживает объективность оценивания и снижает нагрузку на преподавателей. Программное обеспечение применимо в учебном процессе как автономный стенд или как основа для интеграции с инфраструктурой организации.

Достижение поставленной цели опирается на последовательное решение сформулированных задач в логике разработки программного комплекса. Сначала на основе анализа предметной области и аналогов формулируются требования к платформе, затем выполняется проектирование и реализация архитектуры с механизмом изолированной проверки кода, персональными рекомендациями и модулем интеллектуального сопровождения, после чего работоспособность и пригодность платформы к учебному применению оцениваются тестированием и апробацией. Указанные возможности отражены в перечне задач и раскрываются в требованиях и проектных решениях последующих глав.

Созданное решение вносит вклад в цифровизацию образовательного процесса в области программирования и отражает переход к автоматизированным учебным системам.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ

1.1 Анализ онлайн-обучения программированию

Современный этап развития информационных технологий характеризуется устойчивым ростом спроса на специалистов в области разработки программного обеспечения и непрерывным расширением спектра применяемых технологий. В этих условиях возрастает роль цифровых образовательных платформ, обеспечивающих дистанционное освоение языков программирования, инструментов разработки и технологий веб-инженерии.

Принципиальным отличием обучения программированию от дисциплин теоретического профиля является его выраженный практико-ориентированный характер. Освоение языков программирования и инструментов разработки невозможно без систематической работы обучающегося с программным кодом: написания, выполнения, отладки и анализа результатов; базовые представления об алгоритмах и структурах данных при этом опираются на общепринятые учебные источники [14]. Сложившаяся методика подготовки специалистов по информационным технологиям предполагает сочетание теоретического материала, практических заданий и контрольных мероприятий, при этом доля практической составляющей последовательно увеличивается по мере перехода к более сложным разделам учебной программы [2, 15].

Эффективность освоения программирования напрямую зависит от возможности оперативной проверки решений и получения своевременной обратной связи. Отсроченная или нерегулярная проверка ослабляет связь между обучением и контролем результата, снижает мотивацию и затрудняет формирование устойчивых навыков алгоритмического мышления. По указанной причине одной из ключевых задач, решаемых при построении современных образовательных платформ, является автоматизация проверки про-

граммных решений, реализуемая посредством формализованных тестовых сценариев [4, 16].

Структуру предметной области онлайн-обучения программированию целесообразно рассматривать как совокупность взаимосвязанных компонентов, каждый из которых решает самостоятельную образовательную задачу:

а) теоретический компонент, обеспечивающий изложение учебного материала и формирование понятийной базы;

б) практический компонент, включающий выполнение программных заданий с использованием встроенного редактора кода и средств автоматической проверки решений;

в) контрольный компонент, реализующий тестирование знаний и оценку степени усвоения учебного материала;

г) аналитический компонент, накапливающий сведения об учебной активности обучающегося и предоставляющий преподавателю инструменты анализа прогресса;

д) адаптивный компонент, формирующий индивидуальную образовательную траекторию на основе анализа собранных данных.

Перечисленные компоненты находятся в отношениях взаимной зависимости. Теоретический материал формирует основу для последующего выполнения практических заданий, результаты выполнения заданий и тестирования являются исходными данными для аналитического и адаптивного компонентов, а сформированные рекомендации, в свою очередь, влияют на последовательность изучения теоретического материала и подбор практических заданий. Подобная организация образовательного процесса предъявляет дополнительные требования к программной системе, обеспечивающей поддержку всех перечисленных компонентов и согласованность данных между ними [3].

Целевая аудитория платформ онлайн-обучения программированию характеризуется существенной неоднородностью. К ней относятся обучающиеся образовательных организаций высшего и среднего профессионального

образования, специалисты, осваивающие новые направления разработки в рамках программ дополнительного профессионального образования, а также лица, самостоятельно изучающие программирование в формате непрерывного образования. Обучающиеся отличаются по уровню начальной подготовки, темпу освоения учебного материала, предпочитаемой глубине проработки тем и характеру допускаемых ошибок. Подобные различия делают неэффективной единую линейную модель обучения, при которой все пользователи проходят один и тот же набор тем и заданий в фиксированной последовательности [5, 6].

Особенностью обучения программированию на стороне клиента является необходимость освоения нескольких взаимосвязанных технологий, исполняемых в среде браузера, включая языки разметки, стилизации и программирования для интерактивного поведения страниц. Обучающийся одновременно формирует представления о структуре документа, визуальном оформлении и программной логике, поэтому особенно важны оперативная обратная связь, согласованность учебных заданий и наглядный результат выполнения программного кода. Эффективная платформа онлайн-курсов должна обеспечивать практику на стороне клиента, автоматическую проверку кода и персональные рекомендации на основе анализа учебной активности.

Дополнительной особенностью предметной области является повышенная значимость требований к безопасности исполнения пользовательского кода. В отличие от других дисциплин, где результатом выполнения практического задания служит текстовое решение или ответ на вопрос, в обучении программированию итогом работы пользователя является фрагмент исполняемого кода. Автоматическая проверка такого решения предполагает его выполнение в вычислительной среде платформы, что создаёт риски нерационального использования вычислительных ресурсов, бесконечного исполнения программ или попыток доступа к системным ресурсам сервера. Указанные риски формируют отдельный класс требований к программной архитек-

туре, связанных с обеспечением изоляции исполнения пользовательских программ и ограничения времени и объёма используемой памяти [7, 17].

В условиях увеличения количества обучающихся и роста требований к качеству дистанционного образования актуальным становится создание программных систем, способных обеспечивать одновременно несколько ключевых возможностей: выполнение программного кода в интерактивной среде, его автоматическую формализованную проверку, безопасное исполнение пользовательских программ, анализ результатов учебной активности и адаптацию образовательной траектории под индивидуальные особенности обучающегося. Совокупность указанных возможностей определяет границы предметной области, в рамках которой выполняется настоящая работа, и формирует исходные предпосылки для дальнейшего анализа существующих решений и формирования требований к разрабатываемой платформе.

1.2 Анализ существующих платформ-аналогов

Для определения направлений разработки и формирования требований к создаваемой системе был выполнен анализ распространённых онлайн-сервисов, ориентированных на обучение программированию с использованием интерактивной среды выполнения программного кода. В качестве репрезентативных примеров рассмотрены международные платформы Codecademy [18] и freeCodeCamp [19], российская образовательная платформа Stepik [20], а также сервисы алгоритмической подготовки LeetCode [21] и Codewars [22]. Указанные сервисы охватывают принципиально разные модели организации образовательного процесса [3].

Codecademy и freeCodeCamp дают интерактивную практику в браузере с автопроверкой, но типично сохраняют линейную траекторию и ограниченную адаптацию; развёрнутые рекомендации по траектории и глубокая персонализация слабо выражены, часть функций Codecademy коммерциализирована. Stepik ближе к универсальной LMS с авторами курсов и проверкой кода,

однако адаптивность и отдельный модуль рекомендаций по комплексным показателям прогресса ограничены. LeetCode и Codewars ориентированы на алгоритмическую практику и соревновательный выбор задач без полноценного теоретического контура и сопровождения траектории.

В целом проведенный анализ показывает, что существующие решения либо обеспечивают интерактивное выполнение программного кода без развитой адаптивной логики, либо ориентированы на решение отдельных классов задач без комплексного управления образовательным процессом. Во многих случаях автоматическая проверка решений реализована без явного описания архитектурных принципов изоляции исполнения пользовательских программ, что затрудняет независимую оценку устойчивости и безопасности подобных систем [7].

Для систематизации полученных сведений рассмотренные платформы сопоставлены по ряду признаков, существенных для построения адаптивной образовательной системы; результаты сопоставления приведены в таблице 1. В качестве признаков выбраны: степень структурированности учебной траектории, наличие алгоритма адаптации, реализация изоляции исполнения программного кода, наличие средств аналитики и сопровождения преподавателем, поддержка обучения на русском языке и модель доступа к функциональным возможностям.

Приведённое выше сопоставление свидетельствует о том, что ни одна из рассмотренных платформ не объединяет в полном объёме совокупность возможностей, требуемых для адаптивного обучения программированию: структурированную образовательную траекторию с теоретическим и практическим компонентами, прозрачный алгоритм адаптации, описанный механизм изолированного исполнения пользовательского кода и развитую аналитику прогресса, доступную как обучающемуся, так и преподавателю. Системы, ориентированные на массовое обучение, недостаточно поддерживают индивидуализацию траектории, а сервисы алгоритмической подготовки не обеспечивают последовательного освоения учебного материала с нуля.

Таблица 1 – Сравнительный анализ существующих платформ онлайн-обучения программированию

Платформа	Структура обучения	Адаптивность траектории	Изоляция исполнения	Аналитика и сопровождение	Русский язык
Codecademy	Теория и практика, фиксированный путь	Ограниченная (выбор курса)	Не описана	Базовые показатели прогресса	Нет
freeCodeCamp	Практика по разделам, линейный путь	Отсутствует	Не описана	Счётчик выполненных заданий	Частично
Stepik	Курсы преподавателей, теория и практика	Ограниченная	На усмотрение автора курса	Развитая аналитика для авторов курсов	Да
LeetCode	Каталог задач, без структурированного курса	Отсутствует	Реализована	Соревновательные показатели	Частично
Codewars	Каталог задач с уровнями	Отсутствует	Реализована	Рейтинговая система	Нет

Отсюда следует потребность в платформе, объединяющей интерактивную клиентскую среду выполнения программного кода, безопасную серверную изолированную проверку решений по формализованным тест-спецификациям, алгоритм формирования индивидуальной учебной траектории и средства интеллектуального сопровождения. Сформулированные требования к такой платформе приведены в подразделе 1.7; архитектурное воплощение этих положений раскрывается при проектировании во второй главе.

1.3 Подходы к построению адаптивных образовательных систем

Адаптивная образовательная платформа на основе анализа учебной активности и результатов обучающегося формирует индивидуальную последовательность изучения материала, корректирует сложность заданий и выдаёт персональные рекомендации [5]. В отличие от линейных курсов с фиксиро-

ванным порядком тем она учитывает различия в подготовке, темпе освоения и характере ошибок.

Подходы к адаптивности различаются по тому, насколько явно задана логика принятия решений и сколько накопленной истории нужно для устойчивой работы. Правила и эвристики, связывающие показатели активности с переходами между темами, прозрачны для преподавателя и не требуют большого числа попыток до запуска. Статистические оценки владения навыками и модели машинного обучения по последовательности попыток потенциально точнее при масштабном наборе данных, но сложнее в сопровождении и нуждаются в достаточной истории обучения.

С учётом поставленных задач, ориентированных на построение адаптивной платформы онлайн-курсов и её эксплуатацию в условиях ограниченного объёма исходных данных, в качестве базового принят подход на основе явных правил. Данный подход обеспечивает прозрачность принятия решений, возможность последовательной отладки, простоту переноса методики обучения преподавателя в правила системы и допускает дальнейшее расширение в направлении вероятностных моделей и моделей машинного обучения по мере накопления данных об учебной активности. Конкретный вид правил и алгоритм формирования индивидуальной образовательной траектории описываются в подразделе 1.7 настоящей главы. Архитектурное решение, реализующее модуль персонализации, приводится во второй главе.

1.4 Интеллектуальные программные агенты в образовательных системах

Существенное влияние на развитие современных образовательных платформ оказывают большие языковые модели и построенные на их основе интеллектуальные программные агенты. Языковые модели, как правило основанные на архитектуре трансформера [23], генерируют связный текст и обрабатывают формулировки учебных задач на естественном языке. Про-

граммный агент использует модель как управляющее ядро: планирует действия, обращается к внешним источникам и вызывает предоставленные инструменты [9].

Различают программную процедуру (workflow), где разработчик задаёт граф вызовов модели и инструментов, и агента (agent), где последовательность шагов формирует сама модель по текущему состоянию задачи. В образовательной платформе процедуры уместны для типовых операций (проверка решения, подсказка по шаблону, отчёт о прогрессе). Агенты применяются при нестандартном сопровождении, когда заранее заданный граф шагов не подходит.

У моделей ограничен объём контекста. Качество ответа зависит от того, какие системные указания, фрагменты учебного материала и сведения об истории обучающегося включены в запрос. Для подстановки релевантных материалов из базы знаний применяют дополнение генерации поиском (RAG) [24]. Для модульного подключения инструкций и инструментов используют подключаемые навыки (Agent Skills) [10, 12].

На практике выделяют разделение между компактной управляющей моделью и специализированными исполнительными модулями («малое ядро», «крупные исполнители»): проверка кода, доступ к учебному материалу, формирование отчётов. Это повышает предсказуемость поведения и снижает число обращений к ресурсоёмкой модели.

Ключевые компоненты платформы описываются формальными спецификациями: контракты интерфейсов, схемы данных, тест-спецификации задач. Они служат и для автоматической проверки, и для модулей интеллектуального сопровождения. В платформе такой подход применяется при описании учебных заданий, детали приводятся во второй главе.

В совокупности перечисленные понятия (интеллектуальный агент, дополнение генерации поиском, подключаемые навыки, поиск инструментов, протокол модельного контекста, шаблон с малым ядром и крупными исполнителями, спецификационно-ориентированная разработка) образуют ме-

тодическую основу для построения адаптивной платформы онлайн-курсов. Конкретные архитектурные решения, реализующие эти принципы в разрабатываемой платформе, рассматриваются при обосновании стека и в последующих разделах второй главы. Вопросы практической реализации и тестирования изложены в третьей главе.

1.5 Особенности автоматической проверки кода и требования к безопасности исполнения

Автоматическая проверка программного кода является ключевым функциональным элементом любой образовательной платформы, ориентированной на обучение программированию. Её назначение состоит в формировании оперативной обратной связи и формализованной оценке корректности решения, представленного пользователем. В отличие от ручной проверки, выполняемой преподавателем, автоматическая проверка должна функционировать в пределах ограниченного времени отклика, обрабатывать произвольный пользовательский код и сохранять устойчивость при некорректных или потенциально опасных входных данных. Указанные особенности определяют отдельный класс архитектурных и инженерных задач, рассматриваемых в настоящем подразделе [7].

В практике построения образовательных платформ выделяются несколько устоявшихся подходов к проверке программного кода обучающегося. Наиболее распространённым является ввод-выводной подход, при котором система передаёт программе фиксированный набор входных данных и сравнивает её выходной поток с эталонным значением. Подход прост в реализации и применим к широкому классу учебных задач, однако предполагает, что программа представляет собой самостоятельное приложение с явно определёнными входным и выходным потоками. Второй подход, проверка по тест-спецификации, опирается на то, что фрагмент пользовательского кода реализует определённую программную функцию

или класс, а проверка осуществляется набором тест-сценариев, использующих результат вычисления функции и побочные эффекты в качестве предмета сравнения. Подобный подход согласуется с практикой модульного тестирования. Третий подход, проверка структуры программы, предполагает анализ синтаксического дерева программного кода и применяется в случаях, когда оценке подлежит не только результат выполнения, но и применение конкретных языковых конструкций, что особенно характерно для учебных заданий, ориентированных на освоение разметки документа и стилизации в среде браузера. В рамках разрабатываемой платформы предусмотрено сочетание перечисленных подходов в зависимости от типа учебного задания.

Система автоматической проверки кода должна обладать рядом свойств: одинаковые входные данные должны приводить к одинаковому результату проверки (детерминированность); проверка должна завершаться в пределах заданного времени и заданного объёма памяти независимо от свойств пользовательского кода (ограниченность ресурсов); результат проверки должен содержать сведения, достаточные для устранения ошибки в решении, без раскрытия эталонного решения (информативность); система должна корректно классифицировать как неудачные те решения, чьё поведение зависит от случайных или системных факторов, не предусмотренных условием задания (устойчивость к нестабильным результатам); проверка не должна приводить к нарушению целостности и доступности окружения, в котором она выполняется (защищённость). Соответствие этим свойствам формирует исходные требования к программной архитектуре проверяющего модуля.

Принципиальной особенностью автоматической проверки в обучении программированию является исполнение программного кода, источником которого является пользователь. С точки зрения модели угроз пользовательский код следует рассматривать как недоверенный: его поведение не может быть гарантировано на основании декларации автора задания, а в ряде случаев пользователь может намеренно или вследствие ошибки сформировать

код, ведущий к нежелательным последствиям. К основным классам угроз относятся:

а) исчерпание вычислительных ресурсов (бесконечные циклы, рекурсия чрезмерной глубины, неограниченное выделение памяти), приводящее к деградации производительности всего сервера проверки;

б) попытки доступа к файловой системе сервера, переменным окружения и иным процессам, направленные на получение конфиденциальных сведений или нарушение целостности данных;

в) установление сетевых соединений с внешними системами, в том числе с целью загрузки дополнительного кода или передачи похищенных сведений;

г) обход механизмов изоляции среды выполнения, в частности эксплуатация уязвимостей реализации виртуальных машин и сред исполнения языков программирования;

д) модификация общих структур данных среды выполнения, способная исказить поведение последующих проверок; типичный случай для динамических языков, поддерживающих изменение прототипов объектов.

Перечисленные угрозы классифицируются в соответствии с положениями стандарта ГОСТ Р 56939–2024 [17] и руководства NIST по тестированию информационной безопасности [25]. Особое внимание уделяется уязвимостям, связанным с обработкой недоверенного содержимого в среде браузера и серверной среде, систематизированным в каталоге OWASP Top 10 [26,27]. Существенной является и оценка риска уязвимостей в составе используемых программных платформ: в качестве иллюстрации можно привести уязвимость CVE-2025-29927, обнаруженную в популярном программном каркасе серверной разработки и связанную с обходом механизмов авторизации [28], что подтверждает необходимость регулярного контроля состояния зависимостей системы.

Устранение перечисленных угроз достигается изоляцией исполнения. На практике применяют обособленные контексты виртуальной машины

(в среде JavaScript это V8-изоляция с отдельной кучей и лимитами ресурсов [29]), контейнеры и микровиртуальные машины с высокой степенью изоляции при большей накладной стоимости, а также исполнение в WebAssembly с явной моделью памяти. Для учебных задач на JavaScript предпочтительны изоляты из-за малой задержки и согласованности со стеком серверной части.

Ключевыми критериями выбора подхода к изоляции в настоящей работе являются: малое время отклика на проверку решения, ограниченные требования к инфраструктуре, согласованность с программной платформой серверной части системы и достаточность уровня изоляции для исполнения пользовательского программного кода ограниченного объёма. С учётом перечисленных критериев в качестве базового принят подход, основанный на использовании обособленных контекстов виртуальной машины V8: он обеспечивает изоляцию пользовательского кода без необходимости развёртывания дополнительных контейнеров, согласуется с серверной средой исполнения NestJS и допускает прозрачное ограничение времени и объёма памяти. Дополнительные меры защиты, накладываемые на проверяющий модуль системы, включают ограничение длины пользовательского решения, фильтрацию небезопасных языковых конструкций до передачи кода в обособленный контекст, ограничение времени выполнения тест-сценария и контроль состояния изолированного контекста после завершения проверки. Конкретные значения ограничений и архитектурное оформление модуля проверки рассматриваются при обосновании стека настоящей главы и при проектировании во второй главе.

Для автоматической проверки учебного задания в платформе используется формализованная тест-спецификация: перечень тест-сценариев, ожидаемые результаты и правила выставления итоговой оценки. Подобная формализация согласуется с инженерной практикой описания программных испытаний. При подготовке документации на проверяющий модуль ориентируются на положения ГОСТ 19.301–79 [30] и стандарта IEEE 829 [31] в части

состава сведений о тестировании программы. В рамках спецификационно-ориентированного подхода, рассмотренного в подразделе 1.4, те же тест-спецификации служат опорным артефактом для интеллектуального сопровождения. И проверяющий модуль, и модули формирования подсказок опираются на одни и те же сведения, что обеспечивает согласованность поведения системы.

1.6 Обоснование выбора архитектурного подхода и стека технологий

Выбор архитектурного подхода и стека программных технологий разрабатываемой платформы определяется рядом инженерных требований: необходимостью независимого развития клиентской и серверной частей, поддержкой одновременной работы значительного числа пользователей, возможностью изолированного исполнения программного кода в рамках проверки решений, согласованностью используемых программных средств между уровнями платформы и доступностью развитой документации и инструментальной поддержки. С учётом перечисленных требований в качестве архитектурного шаблона принят подход, основанный на разделении платформы на клиентскую и серверную части, взаимодействующие посредством интерфейса прикладного программирования, построенного по архитектурному стилю REST.

В основу проектирования положена многоуровневая организация серверной части: уровни представления, прикладной логики, доступа к данным и интеграции с внешними системами. Такое разделение упрощает изменения, модульное тестирование и сопровождение кодовой базы [32, 33].

Для интерфейса приняты React и TypeScript [34, 35]: компонентная модель и экосистема снижают стоимость сопровождения учебных экранов и редактора кода. Для сборки клиентской части используется Vite [36].

Таблица 2 – Состав ключевых программных средств и объявленные версии зависимостей репозитория проекта

Компонент	Версия или диапазон (объявление в репозитории)
Среда исполнения серверной части в контейнерной сборке (backend/Dockerfile)	Node.js 22 (node:22-alpine)
TypeScript (frontend/package.json, backend/package.json)	клиентская часть: ~5.9.3; серверная часть: ^5.7.3
Библиотека интерфейса React	^18.3.1
Сборка клиентской части (Vite)	^7.2.4
Маршрутизация SPA (react-router-dom)	^7.13.0
Редактор кода в браузере (CodeMirror 6, пакеты @codemirror/*, обёртка @uiw/react-codemirror)	@codemirror/* ^6.x; @uiw/react-codemirror ^4.25.4
Программный каркас серверной части (NestJS, @nestjs/core)	^11.x (^11.1.17)
ORM Prisma (prisma, @prisma/client)	^6.19.2
Драйвер PostgreSQL (pg)	^8.18.0
Контейнер СУБД (docker-compose.dev.yml)	образ postgres:16
Песочница исполнения кода (isolated-vm)	^6.0.2
Модульное тестирование серверной части (Jest)	^30.0.0
Статический анализ (ESLint)	клиентская часть: ^9.39.2; серверная часть: ^9.18.0
Форматирование исходного кода (Prettier)	клиентская часть: ^3.8.1; серверная часть: ^3.4.2
Контейнеризация и связка сервисов	Docker; Docker Compose по файлам репозитория (docker-compose*.yml); образ приложения собирается из backend/Dockerfile

Встроенный редактор реализован на CodeMirror 6 [37] за счёт модульной поставки и меньшего объёма клиентского кода по сравнению с тяжёлыми альтернативами, что согласуется с требованиями отзывчивости интерфейса, изложенными в подразделе 1.5; состав пакетов редактора приведён в колонке вышеприведённой таблицы.

Серверная часть строится на NestJS и TypeScript [38]: модульность, внедрение зависимостей и встроенные средства маршрутизации, валидации и документирования прикладного программного интерфейса согласуются с многоуровневой архитектурой и единым языком с клиентом.

Данные размещаются в PostgreSQL 16 [39]; реляционная модель отражает устойчивые связи сущностей учебного процесса [40,41]. Доступ к схеме оформлен через Prisma с декларативными миграциями и генерацией типизированного клиента [42].

В качестве механизма аутентификации пользователей принято использование подписанных идентификаторов сессии в формате JWT (JSON Web Token), стандартизованном в RFC 7519 [43]. Применение подписанных идентификаторов согласуется с архитектурным стилем REST, позволяет реализовать аутентификацию без хранения состояния сессии на стороне сервера и упрощает горизонтальное масштабирование. Идентификаторы сессии формируются с использованием криптографической подписи, а для обновления сессии без повторного ввода учётных данных применяется механизм длительных идентификаторов обновления, хранящихся в защищённых cookie-файлах с признаком HttpOnly. Управление доступом построено по ролевой модели и опирается на положения общего принципа разрешительной модели доступа в соответствии со стандартом информационной безопасности [26]. Применительно к интеграции с внешними системами авторизации в долгосрочной перспективе предполагается переход к механизму выдачи токенов по протоколу OAuth 2.0, описанному в RFC 6749 [44].

В соответствии с обоснованием, приведённым в подразделе 1.5, для исполнения пользовательского кода на языке JavaScript в составе серверной части принят программный модуль `isolated-vm` (объявленная версия приведена в той же таблице) [29]. Модуль обеспечивает изолированное исполнение кода в обособленных контекстах виртуальной машины V8 с явным заданием ограничений по времени и объёму памяти и согласуется с серверной средой исполнения NestJS. Особенности применения модуля и сопутствующих мер защиты рассматриваются при проектировании во второй главе.

Компоненты системы упаковываются средствами Docker; согласованное развёртывание связки контейнеров описывается утилитой Docker Compose [45]. Контейнеризация обеспечивает воспроизводимость окруже-

ния исполнения, упрощает развёртывание системы в различных средах (разработка, опытная эксплуатация, эксплуатация) и согласуется с практикой современной разработки веб-приложений. Использование декларативного описания связки контейнеров в формате Compose позволяет фиксировать состав, порядок запуска и параметры взаимодействия компонентов в едином файле, поддерживаемом средствами системы контроля версий.

Для клиентской и серверной частей системы приняты статический анализатор ESLint [46] и анализатор стилевых правил Stylelint [47] для контроля соблюдения соглашений по оформлению, форматировщик исходного кода Prettier [48] для приведения кода к единому виду, инструмент jscpd [49] для выявления дублирования фрагментов исходного кода и инструмент Knip [50] для выявления неиспользуемых файлов, зависимостей и экспортов. Объявленные версии ESLint и Prettier совпадают с колонкой той же таблицы. Применение перечисленных инструментов в составе процесса непрерывной интеграции согласуется с положениями ГОСТ Р ИСО/МЭК 25010–2015 в части обеспечения сопровождаемости программного обеспечения [51]; организация проверок в репозитории раскрывается в третьей главе.

Обобщая результаты анализа, закрепим выбор базового подхода: клиент-серверная многоуровневая архитектура с разделением серверной части на уровни представления, прикладной логики, доступа к данным и интеграции. На стороне клиента используются React, TypeScript и Vite; на стороне сервера используются NestJS, TypeScript, Prisma и PostgreSQL; изоляция пользовательского кода реализуется модулем isolated-vm; редактор в браузере построен на CodeMirror; управление сессиями опирается на JWT; упаковку и развёртывание выполняют Docker и Docker Compose. Конкретные объявленные версии перечисленных зависимостей сведены в приведённой выше таблице. Эти решения задают исходные предпосылки для проектирования системы во второй главе.

1.7 Формирование требований к разрабатываемой платформе

На основании результатов анализа предметной области, существующих платформ-аналогов, подходов к адаптивному обучению, концепций интеллектуальных программных агентов, особенностей автоматической проверки кода и принятых архитектурных решений сформирован перечень требований к разрабатываемой платформе. Требования сгруппированы по категориям функциональных требований, нефункциональных требований и требований по обеспечению безопасности; систематизация выполнена с учётом положений ГОСТ Р ИСО/МЭК 25010–2015, определяющего модель качества программных систем [51], а также положений работ И. Соммервилла и С. А. Орлова в части методики формирования требований [32, 33]. Для каждого требования в последующих главах приводится проектное обоснование и описание проверки: требования по информационной безопасности сформулированы в подразделе 1.7; функциональные требования FR-01–FR-16 сопоставлены с прецедентами в таблице А.1, а подтверждение реализации раскрывается в подразделах 2.7, 2.8, 2.9, 2.10 второй главы и в подразделах 3.3, 3.5 третьей главы. Нефункциональные требования NFR-01–NFR-10 подтверждаются текстом второй и третьей глав, как указано в абзаце после таблицы А.2.

В разрабатываемой платформе выделены три категории пользователей: обучающийся, преподаватель и администратор. Состав функциональных требований сформирован отдельно для каждой роли с учётом задач, решаемых пользователями соответствующей категории в процессе работы с платформой. Функциональные требования к возможностям обучающегося сводятся к следующему перечню:

- а) регистрация на платформе и аутентификация с использованием подписанного идентификатора сессии;
- б) просмотр каталога учебных курсов, состава тем и учебных заданий, входящих в выбранный курс;

в) изучение теоретического материала, представленного в форме структурированных учебных блоков;

г) выполнение практических заданий с использованием встроенного редактора программного кода;

д) автоматическая проверка предложенного решения по совокупности тестовых сценариев с предоставлением сведений о результате каждого сценария и обобщённой оценки;

е) получение персональных рекомендаций по дальнейшему изучению учебного материала, формируемых на основании анализа учебной активности;

ж) обращение к интеллектуальному сопровождению за разъяснением учебного материала, формированием подсказки по выполнению задания и анализом допущенных ошибок;

з) просмотр сведений о собственном прогрессе по курсу, включая совокупный показатель завершённых заданий и индивидуальные показатели по темам.

Функциональные требования к возможностям преподавателя отражают задачи конструирования учебного содержания и сопровождения группы обучающихся. Задачи конструирования учебного содержания включают следующие положения:

а) создание учебных курсов, формирование структуры тем и учебных заданий;

б) подготовка учебных заданий, включающая описание условия, формирование тестовых сценариев и задание ограничений по времени исполнения и объёму памяти;

в) опубликование подготовленного учебного материала и его последующее редактирование с сохранением версионной истории.

Задачи сопровождения группы обучающихся включают следующие положения:

а) просмотр сведений об учебной активности группы обучающихся, включая агрегированные показатели завершённости тем и распределение допущенных ошибок;

б) формирование адресных рекомендаций отдельным обучающимся на основании сведений об их учебной активности.

Функциональные требования к возможностям администратора связаны с управлением учётными записями, конфигурацией системы и контролем событий информационной безопасности и формулируются следующим образом:

а) управление учётными записями пользователей и назначение ролей в соответствии с ролевой моделью;

б) конфигурирование параметров платформы, в том числе ограничений на исполнение пользовательского кода, параметров аутентификации и параметров взаимодействия с внешними сервисами;

в) просмотр журналов обращений к платформе и сведений о событиях информационной безопасности.

Для формализации функциональной модели в нотации UML построена детализированная диаграмма прецедентов (приложение Б, рисунок В.7). Актёры расположены вне границы системы; внутри границы приведены прецеденты, сгруппированные по зонам ответственности (учётная запись, материалы, практика и контроль, интеллектуальное сопровождение, преподаватель, администратор). Пунктирные рёбра со стереотипами расширения (extend) и включения (include) отражают типовые зависимости между прецедентами; связь с внешним провайдером языковой модели показана для прецедента вызова его программного интерфейса. Агрегированная трассировка функциональных требований к шести укрупнённым прецедентам UC-01–UC-06 задаётся таблицей А.1.

Таблица А.1 задаёт коды функциональных требований FR-01–FR-16 и трассировку к прецедентам UC-01–UC-06; формулировки соответствуют приведённым перечням по ролям (в перечне для преподавателя – два под-

перечня, в каждом подперечне и в остальных перечнях нумерация пунктов начинается с а).

Подтверждение выполнения требований FR-01–FR-16 в реализованной платформе изложено при описании серверной и клиентской частей (подразделы 2.7 и 2.8), модуля изолированной проверки (2.9), алгоритма рекомендаций (2.10) и проверено автоматизированными и ручными сценариями критического пути по ролям (3.3), а также учебной апробацией (3.5).

Расшифровка прецедентов: UC-01 охватывает учётную запись, каталог учебных материалов, теорию и практические задания; UC-02 соответствует автоматической проверке программного кода; UC-03 включает тестирование (в том числе диагностический тест при первичной настройке профиля), учёт прогресса и персональные рекомендации; UC-04 относится к обращениям к языковой модели; UC-05 объединяет подготовку контента и сопровождение группы обучающихся; UC-06 охватывает администрирование платформы, модерацию материалов и работу с журналами.

Ключевой сценарий первичной настройки пользователя после аутентификации (проверка необходимости прохождения диагностического теста, загрузка заданий теста, отправка ответов, сохранение попытки и назначение рекомендуемой стартовой темы) представлен диаграммой деятельности в нотации UML (приложение Б, рисунок В.8). Диаграмма разделена на дорожки клиентской части (интерфейсные шаги и проверки OnboardingGuard) и серверной части (прикладные операции REST API); построение выполнено средствами Mermaid с экспортом в растровый формат.

Нефункциональные требования сформулированы в виде измеримых характеристик с указанием количественного значения там, где это допустимо. Перечень нефункциональных требований представлен в таблице А.2; характеристики качества соответствуют категориям модели ГОСТ Р ИСО/МЭК 25010–2015 [51].

Подтверждение требований NFR-01–NFR-10 изложено во второй главе (подразделы 2.1, 2.4, 2.7, 2.9, 2.6) и в третьей (подразде-

лы 3.2, 3.3, 3.4, 3.5, 3.6). Производительность API и проверки кода (NFR-01, NFR-02) обеспечиваются выбранной архитектурой и модулем изолированного исполнения; NFR-02 дополнительно проверяется модульными тестами. Сопровождаемость и качество кода (NFR-04, NFR-05, NFR-10) подтверждаются Jest, ESLint и Snyk. Удобство использования и локализация (NFR-06, NFR-07) раскрыты при проектировании клиентской части и при апробации. Переносимость и совместимость (NFR-08, NFR-09) обеспечиваются контейнерным развёртыванием, JSON API и каталогом contracts/. Показатели NFR-01 при 50 одновременных пользователях и NFR-03 за учебный семестр в работе заданы как целевые; формальные нагрузочные испытания и годовая оценка доступности не проводились.

Требования по обеспечению информационной безопасности сформированы с учётом положений ГОСТ Р 56939–2024 в части разработки безопасного программного обеспечения [17], рекомендаций руководства NIST [25], каталога OWASP Top 10 [26], а также Федерального закона «О персональных данных» [52] и Федерального закона «Об образовании в Российской Федерации» [53]. К системе предъявляются следующие требования по обеспечению информационной безопасности:

а) аутентификация пользователей с использованием криптографически стойких механизмов хранения учётных данных и подписанных идентификаторов сессии в формате JWT в соответствии с RFC 7519 [43];

б) ролевая модель управления доступом с разграничением прав по категориям «обучающийся», «преподаватель», «администратор»; запрет повышения привилегий иначе как явным действием администратора;

в) изолированное исполнение пользовательского программного кода в обособленных контекстах виртуальной машины с обязательным заданием ограничений по времени выполнения, объёму памяти и числу обращений к внешним системам;

г) защита от типовых атак на веб-приложения, включая внедрение программного кода в среде браузера (XSS), межсайтовую подделку запроса

(CSRF), внедрение языковых конструкций в запросы к системе управления базами данных (SQLi);

д) защита персональных данных пользователей в соответствии с требованиями Федерального закона № 152-ФЗ; ограничение состава собираемых сведений необходимыми для функционирования системы;

е) ведение журналов обращений к платформе, событий аутентификации и обращений к функциям проверки кода с фиксацией времени, идентификатора пользователя и результата операции;

ж) автоматизированный контроль состояния используемых программных зависимостей с целью своевременного обновления компонентов, в составе которых выявлены известные уязвимости.

Перечисленные требования задают единый контур защиты платформы. Безопасная аутентификация и разграничение доступа по ролям обучающегося, преподавателя и администратора реализуются в серверной части с использованием подписанных идентификаторов сессии JWT и защищённых cookie-файлов (подразделы 2.3, 2.7). Изоляция пользовательского кода и ограничения по времени, памяти и числу обращений к внешним системам обеспечиваются модулем на базе *isolated-vm* (подраздел 2.9). Защита от типовых угроз веб-приложений учитывается при проектировании HTTP-границы, параметризованного доступа к данным и санитизации входящих данных (подраздел 2.7). Состав обрабатываемых персональных данных ограничен необходимым для учебного процесса; обращения к языковой модели и журналирование интеллектуального сопровождения регламентированы при проектировании клиентской части и в подразделе 3.1. События аутентификации и административные действия фиксируются в журналах с возможностью просмотра в административном контуре. Состояние программных зависимостей контролируется в процессе разработки (подраздел 1.6). Подтверждение перечисленных мер выполняется модульными и сценарными испытаниями критического пути (подраздел 3.3), обзорной проверкой безопасности и свод-

кой по таблице 12 третьей главы (подраздел 3.4), а также проверками качества кода (подраздел 3.2).

Условиями эксплуатации разрабатываемой платформы являются следующие положения. Развёртывание платформы осуществляется в среде, обеспечивающей возможность контейнеризации компонентов с использованием утилиты Docker Compose [45].

Требования к аппаратным и программно-эксплуатационным ресурсам сервера необходимо согласовать с режимом работы подсистемы интеллектуального сопровождения, обращающейся к большой языковой модели. При использовании внешнего программного интерфейса языковой модели по модели оплаты запросов и объёма токенов выделение графического ускорителя на стороне сервера не является обязательным условием: достаточно устойчивого исходящего сетевого доступа к провайдеру, конфигурирования ключей и регламентов расхода лимитов; время ответа и доступность в первую очередь определяются внешней службой. При локальном размещении модели на сервере организации целесообразно предусмотреть графический ускоритель с объёмом видеопамяти, сопоставимым с требованиями выбранной конфигурации модели, поскольку вывод на GPU обеспечивает приемлемую задержку при работе нескольких пользователей. Локальный вывод только на центральном процессоре допускается как упрощённый вариант при малой интенсивности одновременных обращений к модели, однако сопровождается увеличением задержки ответа и, как правило, повышенными требованиями к объёму оперативной памяти для размещения весов модели по сравнению с конфигурацией без локального вывода.

В условиях опытной эксплуатации базового контура без локальной языковой модели (серверная часть на платформе Node.js, система управления базами данных PostgreSQL в контейнерах) в качестве ориентира для устойчивой работы небольшой группы пользователей рекомендуется совокупно не менее четырёх логических ядер центрального процессора и не менее 8 Гиб оперативной памяти для контейнеров прикладной логики и СУБД. Конфигу-

рация в два логических ядра и 4 ГиБ оперативной памяти может применяться как минимально допустимая для разработческого или демонстрационного стенда при ограниченной одновременной нагрузке; при выборе локального вывода языковой модели минимальные значения процессорных ядер и памяти повышаются в зависимости от способа размещения модели и должны уточняться по паспортным данным выбранной реализации.

Клиентская часть рассчитана на использование в актуальных версиях массовых браузеров без необходимости установки дополнительного программного обеспечения. Поддержка работы в режиме без подключения к сети не предполагается; восстановление работоспособности после кратковременных сбоев соединения обеспечивается на уровне клиентской части за счёт повторных запросов с экспоненциальной задержкой.

Сформированный перечень требований используется в качестве исходных данных при проектировании архитектуры и компонентов платформы и при формировании планов тестирования и оценке качества реализации. Проектные решения изложены во второй главе, процедуры проверки и результаты апробации — в третьей; соответствие таблицы А.1, абзацев после таблицы А.2 и подраздела 1.7 содержанию глав 2–3 обеспечивает текстовое подтверждение каждого требования.

1.8 Выводы

В первой главе зафиксированы цель и задачи выпускной квалификационной работы и показано место разрабатываемой платформы в контексте онлайн-обучения программированию: акцент на связке теории, практики и контроля, необходимость оперативной автоматической проверки кода и учёта различий аудитории. На этом фоне обоснована значимость собственного решения, сочетающего структурированную траекторию, прозрачную логику адаптации, изоляцию исполнения пользовательских программ и развитые средства сопровождения обучения.

Сравнительный анализ распространённых платформ подтверждает, что комбинация перечисленных свойств в полном объёме в готовых продуктах не воспроизводится, что формулирует практическую мотивацию проекта. Сопоставлены подходы к адаптивности и к интеллектуальному сопровождению на базе больших языковых моделей; для первой версии платформы выбран управляемый правилами вариант адаптации и архитектурно обособленный контур работы с генеративными компонентами с акцентом на безопасность и управляемость контекста.

Для подсистемы проверки решений классифицированы способы автоматической верификации и угрозы исполнения недоверенного кода; для серверной среды Node.js и учебного JavaScript обоснован выбор изоляции на базе изолятов виртуальной машины V8. Параллельно согласован стек технологий клиента, сервера, хранилища и инструментов разработки.

В первой главе сформулированы функциональные и нефункциональные требования к платформе, требования по информационной безопасности и условия эксплуатации. На этой основе во второй главе описываются архитектура, модель данных и основные программные компоненты платформы.

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ПРОГРАММНОЙ ПЛАТФОРМЫ

2.1 Проектирование архитектуры платформы

Проектирование архитектуры разрабатываемой платформы выполнено на основании результатов анализа предметной области и требований, сформулированных в первой главе. Цель проектирования состоит в том, чтобы определить состав программных компонентов платформы, их функциональные назначения, способы взаимодействия и порядок развёртывания и тем самым обеспечить выполнение функциональных требований и соответствие установленным нефункциональным показателям. В настоящем подразделе рассматривается обобщённая архитектура платформы. Проектирование модели данных, серверной и клиентской частей, модуля изолированного исполнения и алгоритма формирования рекомендаций выполняется в последующих подразделах настоящей главы.

В качестве базового архитектурного стиля принят клиент-серверный стиль с разделением системы на клиентскую часть, исполняемую в среде браузера пользователя, и серверную часть, размещённую в управляемом окружении. Взаимодействие между ними строится по архитектурному стилю REST с передачей данных в формате JSON через защищённое соединение. Такой подход обеспечивает независимое развитие клиентской и серверной частей и упрощает сопровождение и тестирование [32]. Серверная часть организована как многоуровневая архитектура с уровнями представления, прикладной логики, доступа к данным и интеграции с внешними системами.

Платформа включает семь укрупнённых функциональных блоков. Это одностраничное клиентское приложение (материалы, задания, прогресс, интеллектуальное сопровождение), модульный API-сервер (логика, учебный процесс, учётные записи), PostgreSQL (контент, попытки, телеметрия, журналы обращений к языковой модели), модуль изолированного исполнения на

базе V8 (далее в тексте — песочница), шлюз интеллектуального сопровождения (Ollama или облачный поставщик), обратный прокси-сервер и подсистема наблюдаемости (Sentry, Prometheus/Grafana).

Обобщённая структурная схема разрабатываемой платформы и взаимосвязи между перечисленными компонентами представлены на рисунке 1.

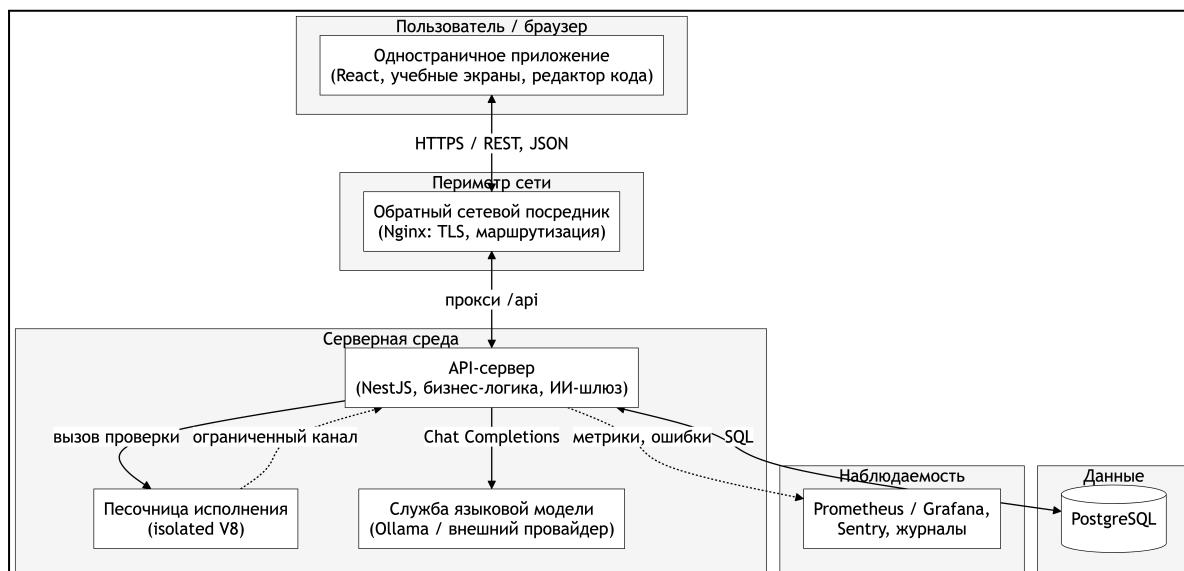


Рисунок 1 – Обобщённая архитектура разрабатываемой платформы

Серверная часть спроектирована как модульное приложение на NestJS [38]. Каждый модуль отвечает за отдельную область логики и взаимодействует с другими через явно объявленные зависимости. Состав модулей соответствует функциям платформы и включает управление учётными записями, каталог учебных материалов, теорию, практические задания, подсказки, контрольные проверки, учебный прогресс, рекомендации, журнал активности, шлюз языковой модели, а также администрирование и модерацию.

Клиентская часть проектируется как одностраничное приложение с маршрутизацией на стороне клиента. Состав маршрутов разделён на четыре группы. К ним относятся публичные маршруты без аутентификации, защищённые маршруты после входа, маршруты раздела преподавателя и маршруты административного раздела. Принцип явного разделения маршрутов по ролям согласуется с ролевой моделью управления доступом, сформулированной в подразделе 1.7. Управление сеансом пользователя реализовано на основе подписанных идентификаторов сессии, передаваемых в защищённых cookie-файлах с признаком HttpOnly, что обеспечивает защиту идентифи-

тора от чтения средствами клиентского кода и снижает риск его компрометации в случае внедрения вредоносного программного кода в среде браузера.

Развёртывание платформы проектируется на основе средств контейнеризации Docker и утилиты Docker Compose [45]. Каждый из компонентов (серверная часть, клиентская часть, система управления базами данных, обратный сетевой посредник, шлюз интеллектуального сопровождения и подсистема наблюдаемости) упаковывается в самостоятельный контейнер. Состав контейнеров, порядок их запуска, параметры сетевого взаимодействия и связанные постоянные тома описываются в декларативном файле формата Compose, поддерживаемом средствами системы контроля версий. Такое представление обеспечивает воспроизводимость окружения исполнения и поддерживает типовые сценарии развёртывания в средах разработки, опытной эксплуатации и промышленной эксплуатации. Схема развёртывания компонентов платформы в среде опытной эксплуатации представлена на рисунке 2. На ней приведены контейнеры узла и основные направления обращений между ними (внешний клиент на схеме не показан).

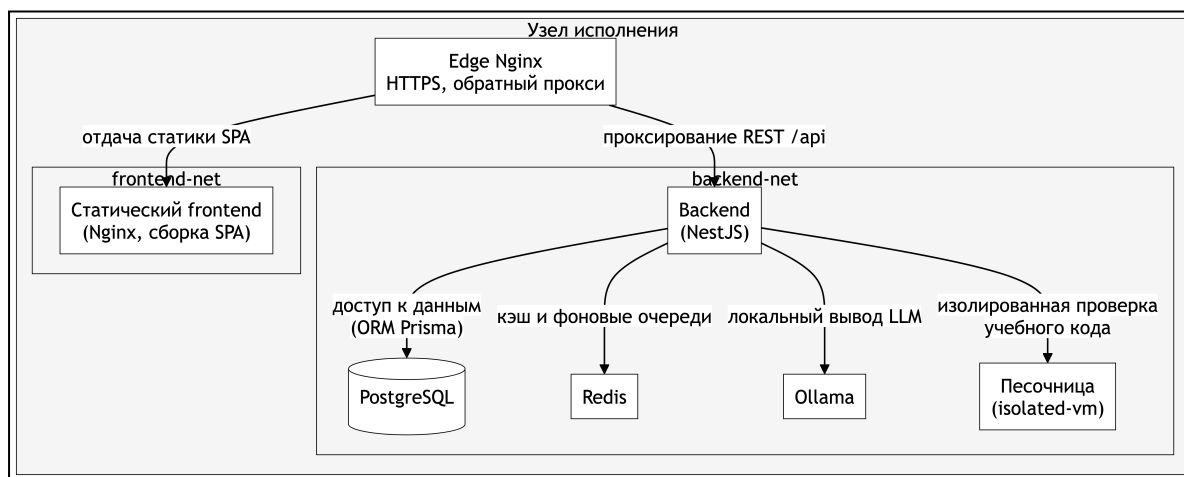


Рисунок 2 – Схема развёртывания платформы в среде опытной эксплуатации

Подсистема наблюдаемости включает регистрацию ошибок (Sentry), метрики (Prometheus) и панели (Grafana), а также структурированные журналы прикладной логики. Перечень событий безопасности согласован с подразделом 1.7 и ГОСТ Р 56939–2024 [17]. Для опытной эксплуатации принята одноузловая конфигурация, однако серверная часть не хранит состояние сес-

сии в оперативной памяти, взаимодействие компонентов сетевое, параметры вынесены во внешнюю конфигурацию, что упрощает переход к многоузловому развёртыванию [32].

Спроектированная архитектура удовлетворяет принципам разделения ответственности, согласуется со стеком программных технологий, обоснованным в первой главе, и обеспечивает выполнение функциональных требований. Детализация модели данных, серверной и клиентской частей, модуля изолированного исполнения и алгоритма формирования рекомендаций приводится в последующих подразделах настоящей главы.

2.2 Проектирование модели данных

Модель данных разрабатываемой платформы спроектирована на основании выявленных в подразделе 1.1 компонентов предметной области и сформулированных в подразделе 1.7 функциональных требований. Целью проектирования является получение целостной структуры хранилища, обеспечивающей согласованное хранение учебных материалов, сведений о пользователях, истории учебной активности и сопутствующих служебных сведений, а также эффективное выполнение основных запросов к данным. Реализация модели данных выполнена в среде системы управления базами данных PostgreSQL версии 16 [39] с использованием инструмента объектно-реляционного отображения Prisma [42]. Описание схемы данных оформлено на декларативном языке определения схемы Prisma и сопровождается набором версионизируемых миграций.

Сущности сгруппированы в шесть кластеров. К ним относятся учётные записи и аудит. Каталог (направление–раздел–тема). Учебные материалы (теория, фрагменты, задания, проверки, вопросы). Активность (попытки, подсказки, телеметрия). Адаптация (прогресс, навыки, параметры адаптации и обращений к языковой модели). Журнал обращений к языковой модели (запрос, ответ, метаданные).

Перечень основных сущностей модели данных и их назначения представлен в таблице А.3. Нормированные имена сущностей и их атрибутов в схеме данных, а также укрупнённая даталогическая модель по схеме Prisma, приведены в приложении Б(рис. В.4).

Концептуальная схема модели данных и взаимосвязи между сущностями представлены на рисунке 3. Иерархия учебных материалов имеет четырёхуровневое строение. Учебное направление включает разделы, разделы включают темы, а темы объединяют теоретические материалы, практические задания и контрольные проверки. Сущности учебной активности связаны с учётной записью пользователя по идентификатору пользователя, что обеспечивает накопление сведений об активности каждого обучающегося в разрезе всех изучаемых тем. Сущности группы адаптации образуют отдельный логический слой модели и используются модулем формирования рекомендаций при выборе следующего учебного действия. Сущности группы интеллектуального сопровождения образуют самостоятельный домен с журналированием обращений к языковой модели и связаны с прочими сущностями исключительно через идентификатор пользователя и идентификатор сущности, к которой относится обращение, что обеспечивает слабую связность данной подсистемы с основной прикладной логикой системы.

В качестве типа первичного ключа всех сущностей принят универсальный уникальный идентификатор стандарта UUID версии 4 (RFC 9562), а не устаревшие варианты версий 1–2 с привязкой к сетевому узлу. Версия 4 строится из случайных бит, поэтому идентификаторы в публичном API трудно угадать, а ключи можно выдавать на любом сервере без общего счётчика. В проекте они генерируются функцией `gen_random_uuid()` в PostgreSQL 16 и значением по умолчанию `@default(uuid())` в Prisma. Версия 7 (упорядочивание по времени, меньшая фрагментация B-дерева при массовых вставках) доступна в PostgreSQL 18 как `uuidv7()`, но для объёма данных учебной платформы переход не давал бы заметного эффекта при смене версии СУБД и схемы. Человекочитаемые URL задаются отдельным `slug`, а не типом UUID.

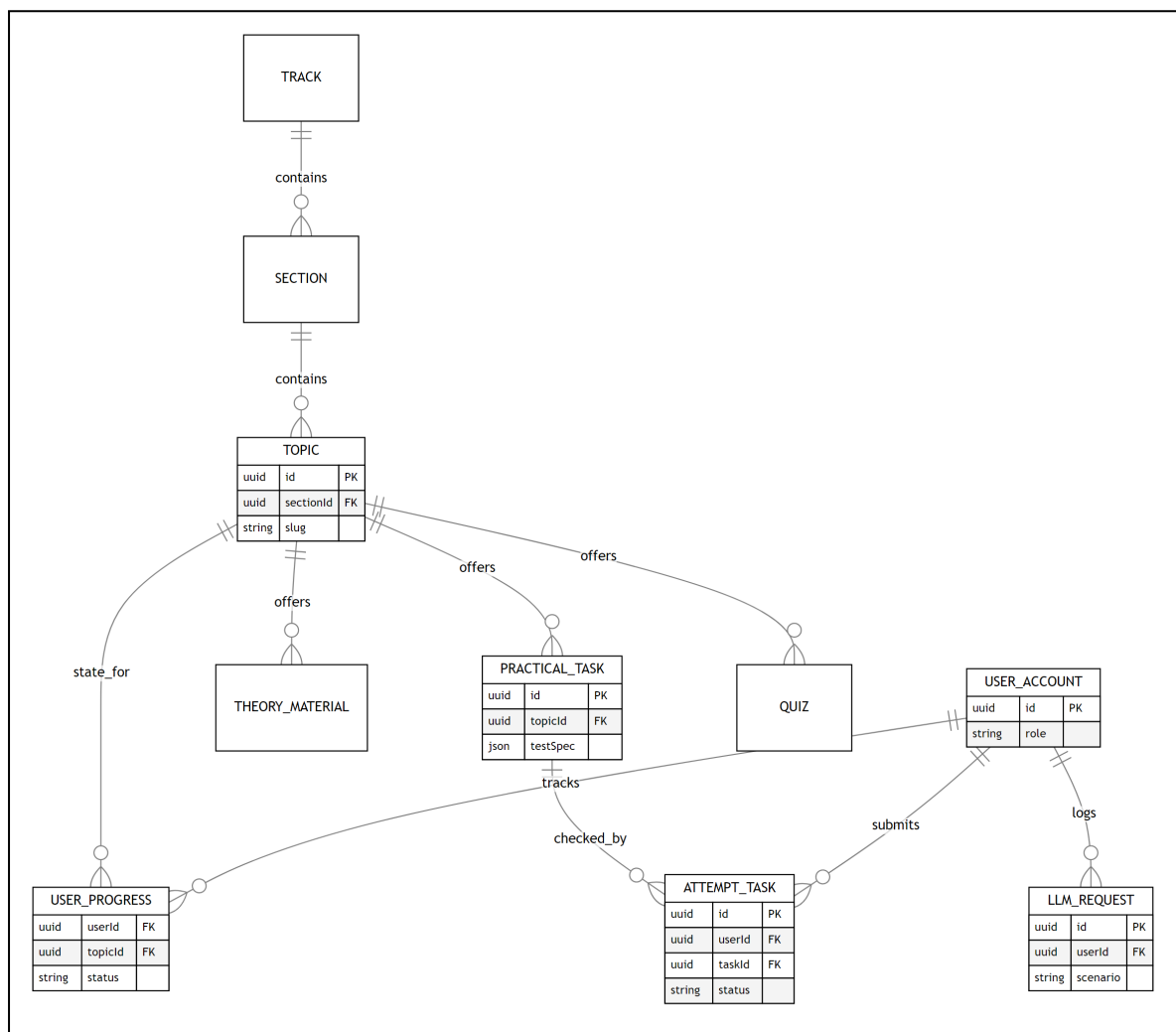


Рисунок 3 – Концептуальная схема модели данных разрабатываемой платформы

Для сущностей, обладающих читаемым уникальным признаком (учебное направление, раздел, тема, теоретический материал, практическое задание, контрольная проверка), дополнительно вводится атрибут «идентификатор для адресной строки», представляющий собой человекочитаемую краткую строку из латинских символов и дефисов и используемый для построения адресов страниц учебных материалов в клиентской части. Перечисленные сущности участвуют в процессе модерации содержимого. Для них в схеме данных предусмотрены атрибуты состояния согласования («черновик», «ожидание согласования», «согласовано», «отклонено») и атрибуты, фиксирующие сведения о сотруднике, выполнившем соответствующее действие, что обеспечивает прозрачность процесса публикации учебных материалов.

Для отдельных сущностей, содержащих структурированные сведения переменного состава (тест-спецификация практического задания, вариан-

ты ответов вопроса контрольной проверки, снимок ответов попытки контрольной проверки, полезная нагрузка события телеметрии), используется атрибут типа JSON, поддерживаемый системой управления базами данных PostgreSQL без обращения к внешним источникам [39]. Применение типа JSON ограничивается случаями, в которых структура содержимого определяется прикладной логикой и не требует регулярного поиска по отдельным полям. Для атрибутов, по которым выполняется поиск (идентификаторы, состояние, временные метки), используются типизированные столбцы с явным указанием типа данных. Подобный подход согласуется с положениями справочной литературы по проектированию реляционных баз данных [40, 41].

Сущности модели данных приведены к третьей нормальной форме. Состав таблиц, первичных и внешних ключей, по которому выполнялась проверка нормализации, приведён на рисунке В.5 (приложение В). Атрибуты каждой сущности зависят только от первичного ключа, неключевые атрибуты не образуют функциональных зависимостей друг от друга. Преднамеренное отступление от полной нормализации применено в двух случаях. Для сущности «вопрос контрольной проверки» атрибуты черновой и опубликованной версии хранятся в одной строке, что обеспечивает атомарную замену версии при согласовании. Для сущности «попытка выполнения практического задания» результат тестовых сценариев сохраняется в виде структурированного значения JSON, что упрощает отображение детальных сведений в пользовательском интерфейсе. В обоих случаях принятые решения обоснованы характером работы прикладной логики и не приводят к нарушению согласованности данных.

Для обеспечения эффективного выполнения наиболее частых запросов к данным в схеме предусмотрен набор вторичных индексов. К основным относятся индексы по идентификаторам внешних ключей, ускоряющие соединения между сущностями (например, по идентификатору раздела в составе сущности «тема»). Составные индексы по комбинациям идентификатора пользователя и идентификатора сущности, к которой относится дей-

ствие, обеспечивающие быстрый доступ к попыткам выполнения заданий конкретного пользователя по конкретной теме. Индексы по временной метке создания для сущностей телеметрии и журналов интеллектуального сопровождения, по которым типичной операцией является построение хронологических последовательностей событий. Индексы по состоянию согласования для сущностей, участвующих в процессе модерации, что ускоряет формирование очередей задач для администратора. Полный перечень индексов модели данных вместе с описанием схемы данных приведён в приложении Б.

Согласованность данных между связанными сущностями обеспечивается на уровне системы управления базами данных за счёт ограничений целостности по внешним ключам. Для каждого внешнего ключа определена политика поведения при удалении связанной сущности. Каскадное удаление применяется к сущностям учебной активности, тесно связанным с учётной записью пользователя или учебным материалом. Обнуление ссылки применяется к необязательным связям (например, ссылка попытки контрольной проверки на рекомендуемую тему не требует наличия темы и допускает её удаление). Принятая политика согласована с правилами обработки персональных данных в части права субъекта на удаление сведений [52]. При удалении учётной записи пользователя автоматически удаляются связанные с ней сведения учебной активности и обращений к языковой модели.

Изменения схемы данных оформляются в виде набора последовательно применяемых миграций, формируемых средствами Prisma [42]. Каждая миграция представлена SQL-инструкциями, поддерживаемыми системой контроля версий, и сопровождается описанием выполняемого изменения. Подобный подход обеспечивает воспроизводимость состояния схемы данных в различных средах развёртывания и поддерживает обратимость изменений схемы при необходимости.

Полученная модель данных удовлетворяет функциональным требованиям, сформулированным в подразделе 1.7, обеспечивает согласованное хранение сведений всех групп сущностей системы и формирует основу для по-

строения серверной части и клиентской части, рассматриваемого в подразделах 2.3 и 2.4 соответственно.

2.3 Проектирование серверной части

Серверная часть разрабатываемой платформы спроектирована как модульное приложение, реализующее многоуровневую архитектуру и предоставляющее прикладной программный интерфейс по архитектурному стилю REST. В качестве программного каркаса использован NestJS [38], изначально ориентированный на построение масштабируемых серверных приложений на языке TypeScript и реализующий принципы внедрения зависимостей и компонентной декомпозиции [32].

Серверная часть разделена на функциональные модули в соответствии с предметными областями системы. Каждый модуль обладает явно описанными границами, объединяет связанные программные компоненты и взаимодействует с другими модулями исключительно через объявленные экспортируемые компоненты и через системы хранения данных. Перечень функциональных модулей серверной части представлен в таблице 3.

Таблица 3 – Состав функциональных модулей серверной части

Модуль	Назначение
Аутентификация	Регистрация пользователей, аутентификация по подписанному идентификатору сессии, восстановление и смена пароля, интеграция с провайдером внешней аутентификации, ведение профиля пользователя
Учебные направления и разделы	Управление каталогом учебных направлений и разделов, контроль доступности разделов в зависимости от прогресса пользователя

Продолжение таблицы 3

Модуль	Назначение
Темы	Управление темами в составе разделов, формирование сведений о составе теории, заданий и проверок темы
Теоретические материалы	Управление материалами в формате Markdown, ведение их версий, регистрация просмотров
Практические задания	Управление учебными заданиями, обработка отправленных решений, координация с модулем изолированного исполнения
Изолированное исполнение	Безопасное исполнение пользовательского программного кода в обособленных контекстах виртуальной машины с заданием ограничений по времени и объёму памяти
Подсказки	Формирование контекстных подсказок к практическим заданиям с учётом сложности задания и истории попыток
Контрольные проверки	Управление проверочными заданиями с множественным выбором. Обработка результатов диагностической и тематических проверок

Продолжение таблицы 3

Модуль	Назначение
Вводное тестирование	Проведение диагностической проверки на этапе входа пользователя в систему, выбор начального учебного направления и темы
Адаптация	Управление параметрами адаптивного алгоритма, формирование рекомендуемого следующего действия по теме
Учебный прогресс	Ведение состояния прохождения тем, формирование сводных сведений о прогрессе для пользователя и преподавателя
Аналитика	Формирование агрегированных сведений об учебной активности, выявление типичных ошибок
Телеметрия	Регистрация событий учебной активности (просмотры, отправки, попытки) с фиксацией контекста
Шлюз обращений к языковой модели	Унифицированное обращение к языковой модели для пояснения теории, разбора ошибок в коде, формирования отзыва о решении. Журналирование запросов

Продолжение таблицы 3

Модуль	Назначение
Администрирование и модерация	Очередь модерации, журнал действий администратора, управление учётными записями, параметры системы, импорт учебного содержания из файловых источников

Каждый функциональный модуль серверной части построен по единому шаблону внутренней организации (контроллер, прикладная служба, доступ к данным через Prisma) и включает следующие уровни:

а) уровень представления. Контроллеры HTTP выполняют приём запроса, проверку прав доступа, валидацию входных данных, делегирование исполнения служебному компоненту и формирование ответа в формате JSON;

б) уровень прикладной логики. Служебные компоненты модуля реализуют сценарии использования, не связанные напрямую с конкретным транспортным протоколом и не зависящие от выбранной системы хранения данных;

в) уровень доступа к данным. Единый служебный компонент работы с системой хранения использует типизированный интерфейс инструмента объектно-реляционного отображения Prisma [42] и инкапсулирует все обращения к базе данных;

г) уровень типов и контрактов. Объекты передачи данных (DTO) декорированы правилами валидации и используются как для приёма входных данных, так и для формирования ответа клиенту, что обеспечивает согласованность типов между клиентской и серверной частями.

Подобная структура обеспечивает изолированность изменений в отдельных уровнях, упрощает модульное тестирование (служебные компоненты прикладной логики тестируются с использованием поддельных реализа-

ций уровня доступа к данным) и согласуется с принципом единственной ответственности классов [54–56].

Связи между модулями и компонентами внутри модулей реализуются на основе механизма внедрения зависимостей, предоставляемого программным каркасом NestJS. Каждый модуль явно описывает состав своих компонентов, экспортируемые им компоненты и зависимости от других модулей. Применение внедрения зависимостей обеспечивает явность графа зависимостей системы, возможность подмены реализаций при тестировании и поддержку различных режимов жизненного цикла компонентов (общий экземпляр на приложение, экземпляр на запрос, экземпляр на транзакцию). Подобная организация согласуется с принципами чистой архитектуры [54].

Защита серверной части описана тремя блоками: общие правила для всех запросов, вход пользователя и проверка прав, отдельные ограничения для наиболее уязвимых действий.

При запуске приложения (корневой модуль NestJS) для всех маршрутов задаются единые настройки. Указывается, с каких адресов сайта разрешено обращаться к API (CORS). Средствами Helmet в ответы добавляются служебные заголовки, в том числе политика безопасности для JSON-ответов API. Тело каждого запроса проверяется по описанию объекта передачи данных, поля, не предусмотренные контрактом, отбрасываются. Если в коде возникает ошибка, её перехватывает один общий обработчик на всё приложение. Клиент получает ответ в едином виде (код ошибки, текст и при сбое проверки полей список замечаний), а сведения об ошибке записываются в журнал и при серьёзных сбоях передаются в Sentry. Параметры перечисленных настроек задаются через переменные окружения и проверяются при старте.

Вход в систему выполняется по подписанному токenu сессии JWT [43]. Токен хранится в cookie-файле с признаком HttpOnly (недоступен скриптам страницы) или передаётся в заголовке HTTP при программном доступе. Перед выполнением метода API срабатывают три проверки (guards). Проверяется, что пользователь вошёл в систему. Проверяется его роль (обучающийся,

преподаватель, администратор). При необходимости проверяется, что запрашиваемый ресурс принадлежит этому пользователю. Ролевая модель согласована с подразделом 1.7.

Дополнительно ограничивается число запросов в минуту к входу, восстановлению пароля, отправке решения задания и обращению к языковой модели. Обращения к базе данных идут только через Prisma с подстановкой параметров, без сборки SQL из строк. На стороне клиента экранируется вывод в интерфейсе. Совокупность мер соответствует рекомендациям каталога OWASP Top 10 [26, 27] и снижает риск внедрения чужого кода на странице, подделки запроса и внедрения команд в запросы к базе данных.

Единый формат ответов об ошибках и запись сбоев в журнал и Sentry упрощают сопровождение системы.

Описание прикладного программного интерфейса серверной части формируется в соответствии со спецификацией OpenAPI и публикуется в виде интерактивной справки, доступной разработчикам клиентской части. Описание формируется частично автоматически на основании декораторов программного каркаса и аннотаций объектов передачи данных и дополняется текстовыми пояснениями к назначению каждого ресурса. Применение OpenAPI обеспечивает согласованность реализации прикладного программного интерфейса с его описанием и согласуется с положениями принятого в настоящей работе спецификационно-ориентированного подхода, изложенными в подразделе 1.4.

Спроектированная серверная часть удовлетворяет функциональным требованиям, сформулированным в подразделе 1.7, обладает явной модульной структурой, согласуется с принципами многоуровневой архитектуры и предоставляет необходимые механизмы аутентификации, управления доступом и защиты от типовых атак. Особенности её программной реализации рассматриваются в подразделе 2.7.

2.4 Проектирование клиентской части

Клиентская часть разрабатываемой платформы спроектирована как одностраничное приложение, исполняемое в среде браузера пользователя и обеспечивающее представление учебного материала, выполнение учебных заданий, отображение прогресса и взаимодействие с интеллектуальным сопровождением. Основная библиотека построения пользовательского интерфейса React [34], язык программирования TypeScript [35], сборщик и среда разработки Vite [36]. Обоснование выбора этих средств приведено в подразделе 1.6. Объявленные версии ключевых зависимостей клиентской части сведены в таблице 2.

Состав маршрутов клиентской части разделён на четыре группы, соответствующие категориям функциональных возможностей системы и принятой в подразделе 1.7 ролевой модели управления доступом. Перечень групп маршрутов и их назначение представлены в таблице 4.

Таблица 4 – Группы маршрутов клиентской части

Группа	Доступ	Назначение
Публичные маршруты	Без аутентификации	Вводная страница, страницы регистрации и аутентификации, восстановления пароля, сведения о платформе, политика конфиденциальности, контакты

Продолжение таблицы 4

Группа	Доступ	Назначение
Защищённые маршруты	Аутентифицированный пользователь	Каталог учебных направлений, темы, теоретические материалы, практические задания, контрольные проверки, прогресс пользователя, профиль учётной записи
Маршруты раздела преподавателя	Роль «преподаватель»	Управление учебными материалами, формирование заданий и проверок, аналитика по обучающимся
Маршруты раздела администратора	Роль «администратор»	Очередь модерации, журнал действий, управление учётными записями, параметры системы, средства интеллектуального генерирования содержимого

Маршруты защищённых групп заключены в макетный компонент, выполняющий проверку наличия активного сеанса пользователя и его роли до отображения дочерних элементов. При отсутствии активного сеанса пользователь перенаправляется на страницу аутентификации с сохранением целевого адреса. При недостаточности прав доступа отображается страница «доступ запрещён». Дополнительно для группы защищённых маршрутов применяется

ся компонент-защитник вводного тестирования, контролирующий факт прохождения диагностической проверки и обеспечивающий перенаправление на её прохождение при первичном входе пользователя в систему. Подобное разделение ответственности между компонентами защищённых маршрутов соответствует принципам компонентной декомпозиции пользовательского интерфейса [34].

Компоненты пользовательского интерфейса спроектированы по принципу иерархической декомпозиции и разделены на три уровня детализации:

а) уровень страниц. Крупные компоненты, привязанные к маршрутам и реализующие отдельные сценарии использования (страница учебного направления, страница темы, страница задания, страница контрольной проверки, панель прогресса пользователя);

б) уровень функциональных областей. Составные компоненты, объединяющие связанные элементы интерфейса в самостоятельные блоки (область отображения теоретического материала, область выполнения практического задания, область интеллектуального сопровождения, область просмотра ошибок при выполнении задания);

в) уровень атомарных компонентов. Повторно используемые элементы пользовательского интерфейса (кнопки, поля ввода, метки, диалоговые окна, панели уведомлений), оформленные в соответствии с общей системой визуального оформления.

Подобная декомпозиция упрощает повторное использование компонентов, локализует изменения визуального оформления и обеспечивает согласованность пользовательского интерфейса. Принципы оформления элементов интерфейса согласованы с положениями работ Д. Нормана в части ясности доступных действий и обратной связи о результатах действий [57].

Управление состоянием клиентской части построено по принципу минимизации общего состояния. Состояние, существенное для нескольких компонентов одного сценария, размещается в общем родительском компоненте и передаётся дочерним посредством явных параметров. Состояние, суще-

ственное для всего приложения (выбранное учебное направление, сведения о текущем пользователе), хранится в небольшом числе сквозных контекстов библиотеки React. Применение специализированных библиотек управления глобальным состоянием в проекте не предусмотрено по причине ограниченного объёма общего состояния и для упрощения сопровождения. Подобный подход согласуется с принципом избегания преждевременного усложнения, описанным в литературе по практике разработки программного обеспечения [58].

Все обращения клиентской части к серверной части сосредоточены в выделенном служебном слое, реализованном с использованием библиотеки HTTP-клиента Axios. Слой обеспечивает единую точку настройки базового адреса серверной части и режима передачи защищённых cookie-файлов, перехватчик запроса для установки резервного заголовка авторизации во внешних окружениях и перехватчик ответа для унифицированной обработки ошибок прикладного программного интерфейса (потеря сеанса, недостаточность прав, нарушение валидации, ограничение частоты запросов) с формированием пригодных для отображения уведомлений. Сценарии работы с отдельными ресурсами прикладного программного интерфейса оформлены в виде самостоятельных служебных модулей (служебные модули учётных записей, учебных материалов, заданий, проверок, интеллектуального сопровождения), что обеспечивает локализацию изменений при развитии прикладного программного интерфейса.

Идентификатор сеанса пользователя передаётся между клиентом и сервером в защищённом cookie-файле с признаком HttpOnly, что обеспечивает защиту идентификатора от чтения средствами клиентского кода и снижает риск его компрометации в случае внедрения вредоносного программного кода в среде браузера [26]. Состояние входа пользователя в систему отображается в клиентской части на основании ответа отдельного запроса проверки текущего пользователя. При потере сеанса перехватчик ответа очищает локальное состояние и перенаправляет пользователя на страницу аутенти-

фикации с сохранением целевого адреса. В резервном сценарии (например, при использовании клиентской части в среде, не поддерживающей передачу cookie-файлов между подобластями) идентификатор сеанса передаётся в заголовке HTTP «Authorization». Решение о сценарии передачи принимается слоем обращения к прикладному программному интерфейсу автоматически.

В составе страниц практических заданий используется встроенный редактор программного кода, построенный на основе библиотеки CodeMirror версии 6 [37]. Редактор настроен на подсветку синтаксиса языков JavaScript, TypeScript и языка разметки HTML. В составе настроек редактора предусмотрены режимы автоматического сохранения промежуточного решения в локальном хранилище браузера и восстановления решения при возврате пользователя к заданию. Размеры редактора и состав отображаемых элементов адаптированы к различным размерам экрана пользователя, что соответствует требованию NFR-06 в части поддержки адаптивной вёрстки.

Для учебных заданий с использованием языка разметки HTML и языка стилизации CSS в составе клиентской части предусмотрен модуль предварительного просмотра, отображающий результат выполнения задания во встроенном изолированном кадре (iframe) со строгим набором атрибутов изоляции. Подход опирается на штатные механизмы браузера и обеспечивает достаточный уровень изоляции отображаемого содержимого для учебных целей. Окончательная проверка решения выполняется на стороне серверной части в составе модуля изолированного исполнения, описанного в подразделе 2.9.

Пользовательский интерфейс полностью локализован на русский язык. Формат отображения дат, чисел и временных интервалов соответствует правилам русской локали. Цветовая схема, типографика, размеры элементов управления и отступов оформлены в виде системы переменных каскадных стилей и одноимённого объекта на языке TypeScript, что обеспечивает согласованность визуального оформления и упрощает построение альтернатив-

ных тем. Поддерживается переключение между светлой и тёмной темами оформления по выбору пользователя.

Клиентская часть проектируется с учётом основных положений руководства по доступности веб-содержимого WCAG 2.1 уровня AA. Среди требований соблюдение требуемых соотношений контрастности текста и фона, поддержка управления интерфейсом с использованием клавиатуры, явная разметка элементов формы, поддержка средств экранного чтения. Адаптивная вёрстка обеспечивает корректное отображение пользовательского интерфейса на устройствах с шириной экрана от 320 пикселей в соответствии с требованием NFR-06.

Регистрация ошибок клиентской части выполняется средствами подсистемы Sentry [27]. При возникновении необработанной ошибки или отказа сетевого запроса средства подсистемы фиксируют сведения об ошибке, идентификатор сессии и выборочный контекст пользовательских действий. Передача персональных данных в подсистему регистрации ошибок ограничена на уровне настройки клиента подсистемы, что соответствует требованиям обработки персональных данных [52].

Спроектированная клиентская часть (1.7, три роли, редактор, интеллектуальное сопровождение) реализуется в 2.8. Обращение к языковой модели выполняется по тем же принципам программного интерфейса, что и к остальному серверному контуру. Ниже рассматриваются архитектура агентов сопровождения, подключаемые навыки и разработка по спецификации. Эксплуатационные риски обсуждаются в главе 3.

2.5 Проектирование архитектуры агентов сопровождения обучения

Теоретические основания интеллектуального сопровождения изложены в подразделе 1.4 первой главы [9, 59]. Во второй главе описывается, как эти идеи перенесены в серверный модуль платформы. Модуль выступает еди-

ным шлюзом к языковой модели и не дублирует учебную логику (проверку заданий, прогресс, каталог), а только формирует текстовые подсказки и черновики контента на основании данных, уже хранящихся в платформе.

По назначению обращения к модели разделены на два класса. К первому относятся интерактивные запросы обучающегося и преподавателя, выполняемые синхронно в рамках одного HTTP-обращения (пояснение выделенного фрагмента теории, разбор ошибок в коде, упрощённое изложение условия задания, пошаговый разбор эталонного решения). Второй класс составляет административная генерация контента по теме, инициируемая преподавателем или администратором и реализуемая как фоновый конвейер из нескольких шагов (краткий план темы, набор практических заданий, одна или несколько контрольных проверок). Такое разделение обусловлено различной допустимой задержкой ответа. Интерактивные подсказки должны формироваться за секунды, тогда как полная генерация темы нагружает локальную модель Ollama и выполняется в очереди по одной задаче.

Все сценарии доступны только авторизованному пользователю по префиксу /api/ai с проверкой JWT и ограничением числа запросов в минуту. Перечень сценариев и соответствующих программных компонентов приведён в таблице 5.

Таблица 5 – Сценарии интеллектуального сопровождения и программные компоненты

Сценарий	Назначение и реализация
Пояснение выделенного фрагмента теории	Формирование краткого пояснения к выделенному пользователем тексту с учётом окружающего контекста и учебного направления (JavaScript или TypeScript). Служебный компонент AiService, сценарий theory_explain_selection.

Продолжение таблицы 5

Сценарий	Назначение и реализация
Разбор ошибок в учебном коде	Подсказки по исправлению кода с учётом формулировки задания, сводки результатов автоматических тестов и учебного направления. Служебный компонент AiService, сценарий task_debug_user_code.
Упрощённое изложение формулировки задания	Перефразирование условия без выдачи готового решения. Служебный компонент AiService, сценарий task_explain_task_description.
Пояснение эталонного решения	Пошаговое объяснение предоставленного эталонного кода в связке с условием задания. Служебный компонент AiService, сценарий task_explain_reference_solution.
Конвейер генерации по теме	Последовательное выполнение шагов (краткий план темы, генерация набора практических заданий в структурированном виде, одна или несколько контрольных проверок с настраиваемым числом вопросов и распределением сложности). Служебный компонент AiOrchestratorService совместно с AiGenerateService. Сценарии generate_plan, generate_tasks, generate_quiz.

Схема взаимодействия компонентов представлена на рисунке 4.

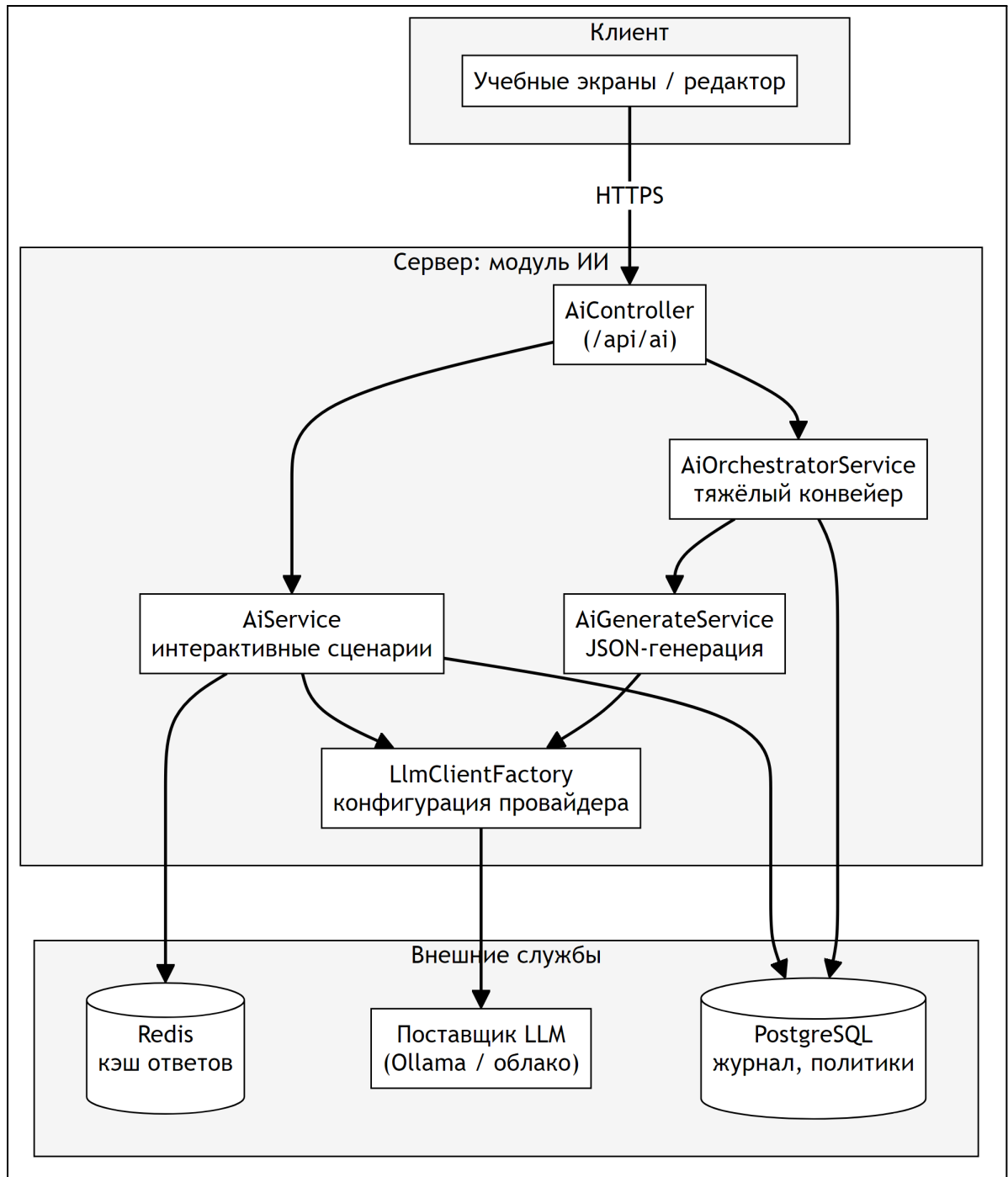


Рисунок 4 – Архитектура агентов
сопровождения обучения (обобщённая схема)

AiController принимает HTTP-запросы и проверяет входные объекты передачи данных. AiService обслуживает четыре интерактивных сценария. Он собирает промпт, применяет политики безопасности, обращается к кэшу Redis, записывает журнал в таблицы AiRequest и AiResponse, учитывает суточную квоту для ролей обучающегося и преподавателя. AiOrchestratorService создаёт фоновую задачу (jobId), ведёт статус ша-

гов и последовательно вызывает генерацию. AiGenerateService запрашивает у модели структурированный JSON (задания, вопросы проверки, план) и при ошибке разбора повторяет запрос с уточнёнными инструкциями. LlmClientFactory скрывает различия поставщиков (локальный Ollama или облачный API в формате Chat Completions, по аналогии с унификацией инструментов MCP [60]) и поддерживает два профиля настроек. Профиль interactive использует более лёгкую модель для быстрых подсказок, профиль generate предназначен для длинных ответов при наполнении темы. Параметры модели, температуры, тайм-аута и лимитов текста подставляются в порядке приоритета. Сначала из таблицы AiAgentSettings, затем из переменных окружения, при отсутствии записей используются значения по умолчанию для выбранного режима развёртывания.

Интерактивный сценарий для всех четырёх видов помощи реализован единой цепочкой обработки запроса. Клиент передаёт идентификатор материала или задания и фрагмент текста либо кода. Сервер проверяет размер входа (по умолчанию не более 8000 символов суммарно), при необходимости отклоняет слишком длинный запрос. Из базы подгружаются условие задания, результаты последних попыток, учебное направление (JavaScript или TypeScript). По нормализованному входу вычисляется хэш SHA-256. При совпадении с недавним ответом в Redis возвращается закешированный Markdown без повторного вызова модели. Иначе формируется пара сообщений (системная инструкция и пользовательский контекст) из файла промпта соответствующего сценария, выполняется запрос к профилю interactive, ответ сохраняется в журнале. В журнал попадают не полные тексты, а длины и хэши полей, статус, число токенов и признаки безопасности (например, очень длинный блок кода). Клиент получает Markdown, идентификатор записи requestId и признак cached. Позже пользователь может отправить оценку полезности ответа. Для обучающихся и преподавателей действует суточная квота интерактивных запросов (настраивается в AiAgentSettings, по умолчанию 10 и 20 соответственно), администратор квоте не подчиняется.

Конвейер генерации по теме запускается отдельным запросом и сразу возвращает идентификатор задачи `jobId`. Дальнейший опрос статуса показывает текущий шаг (план, задания, проверка) и итог или текст ошибки. Оркестратор читает из базы название темы, структуру раздела и при наличии опубликованный фрагмент теории. Сначала генерируется краткий план, затем массив заданий с полями заголовка, описания, начального кода и тест-спецификации, затем один или несколько наборов вопросов проверки с вариантами ответов и тегами навыков. Сбой шага плана не останавливает остальные шаги, а критическая ошибка при генерации заданий или вопросов завершает задачу со статусом ошибки. Каждый шаг сопровождается записью в журнале `AiRequest`. Состояние задач хранится в памяти процесса сервера с автоматическим удалением через 30 минут, что достаточно для интерфейса преподавателя и не требует отдельной очереди сообщений на этапе MVP.

Таким образом, в подразделе зафиксированы сценарии (таблица 5), разделение быстрых и фоновых обращений, состав модуля и правила журналирования. Тексты инструкций для модели и согласование разработки с артефактами спецификации рассматриваются в подразделе 2.6. Ограничения по безопасности и персональным данным рассматриваются в подразделе 3.1 третьей главы.

2.6 Применение подходов подключаемых навыков и разработки по спецификации

В литературе по построению агентов подключаемые навыки (skills) понимаются как оформленные пакеты инструкций и требований к формату ответа для отдельных сценариев [10, 12, 13]. В разрабатываемой платформе отдельный внешний каталог навыков не применяется. Каждый сценарий из подраздела 2.5 сопоставлен трём элементам реализации. Это файл промпта в каталоге `backend/src/ai/prompts/`, значение перечисления `AiScenario` и общие правила класса `AiPolicy` (нормализация текста, ограничение длины вхо-

да, вычисление хэша для кэширования, признаки длинного блока кода, запись в журнал длин и хэшей вместо полных текстов).

В каталоге промптов для интерактивных сценариев заданы системные правила на русском языке. При разборе кода запрещается выдавать готовое решение целиком, допускаются указания на типичную ошибку. При пояснении теории модель должна опираться на выделенный фрагмент и выбранный учебный трек. Для административной генерации в отдельных файлах зафиксирован ожидаемый формат JSON (поля задания, вопроса, вариантов ответа), что позволяет AiGenerateService проверять структуру ответа и при ошибке разбора повторить запрос. Подобная организация обеспечивает воспроизводимость сценариев в пределах одного репозитория и согласованное версионирование вместе с исходным кодом серверной части.

Параллельно используется разработка по спецификации. В каталоге specs/001-adaptive-learning-mvp/ хранятся согласованные артефакты минимально жизнеспособной версии платформы [61]. Файл spec.md содержит пользовательские истории и критерии приёмки. Файл data-model.md описывает сущности предметной области и согласован с schema.prisma. В подкаталоге contracts/ для групп REST-ресурсов указаны метод, путь, форматы тел запроса и ответа, коды ошибок и ограничения частоты. Файл tasks.md фиксирует этапы реализации, файлы plan.md и research.md содержат архитектурные решения и обоснование выбора технологий. Сводное соответствие уровней спецификации артефактам репозитория приведено в таблице 6.

При изменении функции сначала уточняется формулировка в spec.md и при необходимости контракт в contracts/, затем обновляются схема Prisma и миграция, после чего вносятся изменения в контроллеры, служебные компоненты и страницы клиентской части с отметкой в tasks.md. Учебный контент в формате Markdown ведётся в каталоге materials/ по правилам CONTENT_GENERATION_GUIDE.md и импортируется в базу данных отдельно от кода API. Для модуля интеллектуального сопровождения имена полей в промптах генерации совпадают с полями сущностей в Prisma и в фи-

зической модели (приложение Б), что исключает ручное согласование столбцов при сохранении сгенерированных данных.

Таблица 6 – Уровни спецификации и соответствующие артефакты репозитория проекта

Уровень	Артефакт и назначение
Требования и приёмочные сценарии	Файл specs/001-adaptive-learning-mvp/срес.md. Формулировки пользовательских историй, критерии приёмки, уточнения по ограничениям (например, политики ограничения частоты запросов, наблюдаемость на этапе MVP).
Контракты интерфейса прикладного программирования	Файлы specs/001-adaptive-learning-mvp/contracts/*.md. Для каждой группы конечных точек заданы метод, путь, формат тел запроса и ответа в нотации JSON, коды ошибок и ограничения по частоте. Пути согласованы с префиксом /api/ в реализации.
Модель хранимых данных	Файл backend/prisma/schema.prisma. Декларативное описание сущностей, связей, ограничений целостности и перечислений. Миграции схемы оформляются средствами Prisma и воспроизводят состояние базы данных в различных средах [42].

Продолжение таблицы 6

Уровень	Артефакт и назначение
Проверка входных данных при исполнении	Объекты передачи данных с декораторами class-validator и глобальная труба валидации каркаса NestJS [38]. Обеспечение соответствия фактических запросов контрактам по составу полей и типам на стороне сервера.
Параметры среды исполнения	Файл backend/src/config/env.schema.ts и подключение в точке входа приложения. Явные правила для обязательных и необязательных переменных окружения, предотвращение запуска с неполной конфигурацией.
Инструкции для языковой модели	Каталог backend/src/ai/prompts/. Изолированные сценарные «навыки» (системный текст, сборка пользовательского сообщения, параметры трека JavaScript/TypeScript).
Исходные учебные материалы	Каталог materials/. Соглашения по структуре Markdown-файлов для импорта теории, заданий и проверок в систему хранения данных.

Нефункциональные требования допускают в перспективе публикацию описания API в формате OpenAPI [62]. В рамках дипломного проекта контракты ведутся в Markdown в каталоге contracts/. Согласованное использо-

вание каталога `specs/` уменьшает расхождение между требованиями, реализацией REST, схемой базы данных и текстами промптов оркестратора [9]. Оценка рисков применения языковой модели приведена в подразделе 3.1.

2.7 Реализация серверной части

Программная реализация серверной части выполнена на языке программирования TypeScript с использованием программного каркаса NestJS. Номера версий каркаса и связанных зависимостей приведены в таблице 2. Структура исходного кода соответствует принципам организации, рассмотренным в подразделе 2.3. Каждый функциональный модуль размещён в отдельном каталоге, объединяющем составляющие его контроллер, служебные компоненты, объекты передачи данных и связанные элементы (защитники, стратегии аутентификации, перехватчики). Обращение к системе управления базами данных сосредоточено в выделенном служебном модуле и осуществляется через типизированный программный интерфейс инструмента объектно-реляционного отображения Prisma.

Корневой каталог исходного кода серверной части содержит точку входа приложения, корневой модуль и подкаталоги функциональных модулей. Корневой модуль декларативно описывает состав приложения. Подключаются модуль конфигурации, выполняющий проверку параметров окружения по схеме, Подключение модуля кэширования. Подключение модуля ограничения частоты запросов с тремя именованными политиками (короткая политика для операций аутентификации, средняя для приёма решений учебных заданий, длительная для обращений к подсистеме интеллектуального сопровождения). Подключение модуля доступа к базе данных и подключение функциональных модулей серверной части в порядке, определяемом графом зависимостей. Глобальный перехватчик аудита изменений административных сущностей подключён на уровне корневого модуля, что обеспечивает регистрацию операций модификации без распространения соответствующего поведе-

ния на уровень контроллеров. Фрагмент описания корневого модуля приведён в листинге 2.1.

Листинг 2.1 – Структура корневого модуля серверной части

```
1 @Module({
2   imports: [
3     ConfigModule.forRoot({ isGlobal: true, validate: (c) => envSchema.parse(c) }),
4     CacheModule.register({ isGlobal: true, ttl: 5 * 60 * 1000 }),
5     ThrottlerModule.forRoot([
6       { name: 'short', ttl: 60000, limit: 5 },
7       { name: 'medium', ttl: 60000, limit: 30 },
8       { name: 'long', ttl: 60000, limit: 10 },
9     ]),
10    PrismaModule, AuthModule, TopicsModule, SectionsModule,
11    OnboardingModule, TheoryModule, SandboxModule, TasksModule,
12    HintsModule, QuizzesModule, AdaptationModule, ProgressModule,
13    TelemetryModule, AnalyticsModule, AdminModule, AiModule, HealthModule,
14  ],
15  controllers: [AppController],
16  providers: [
17    AppService,
18    { provide: APP_INTERCEPTOR, useClass: AdminMutationAuditInterceptor },
19  ],
20 })
21 export class AppModule implements NestModule {
22   configure(consumer: MiddlewareConsumer) {
23     consumer.apply(LoggingMiddleware).forRoutes('*');
24   }
25 }
```

Приложение поднимается с глобальным whitelist-преобразователем полей, единым обработчиком ошибок (идентификатор запроса в журнале), CORS, Helmet и Sentry. Методы контроллеров принимают DTO с class-validator. Типы согласованы между уровнями за счёт общего TypeScript.

Защитник аутентификации реализован на основе встроенной стратегии разбора подписанных идентификаторов сессии в формате JWT [43], расширенной поддержкой выборочно-открытых маршрутов. Маршруты, не требующие активного сеанса (главная страница, регистрация, аутентификация, восстановление пароля), помечаются специальным декоратором, и защитник пропускает запросы к ним без проверки идентификатора сессии. В остальных случаях защитник проверяет наличие подписанного идентификатора сессии в защищённом cookie-файле или в заголовке HTTP «Authorization» и при

успешной проверке записывает разобранные сведения о пользователе в объект запроса. Реализация защитника представлена в листинге 2.2.

Листинг 2.2 – Защитник аутентификации с поддержкой выборочно-открытых маршрутов

```
1 @Injectable()
2 export class JwtAuthGuard extends AuthGuard('jwt') {
3   constructor(private reflector: Reflector) { super(); }
4
5   canActivate(context: ExecutionContext) {
6     const isPublic = this.reflector.getAllAndOverride<boolean>(
7       IS_PUBLIC_KEY, [context.getHandler(), context.getClass()],
8     );
9     if (isPublic) return true;
10    return super.canActivate(context);
11  }
12
13  handleRequest<TUser>(err: Error | null, user: TUser | false): TUser {
14    if (err || !user) {
15      throw err ?? new UnauthorizedException('Invalid or missing token');
16    }
17    return user;
18  }
19 }
```

Сессии, Prisma, тесты и наблюдаемость. JWT выдаётся в httpOnly-cookie с политикой, зависящей от среды. Срок cookie совпадает с exp токена. Доступ к БД выполняется через один Prisma-сервис и транзакции. Строковая конкатенация SQL исключена [26]. Тестирование выполняется средствами Jest (модули с моками и e2e-профиль, 12 файлов .spec.ts, 136 модульных тестов, npm test проходит без ошибок, NFR-04, 1.7). Журналы запросов/валидации и Sentry с минимизацией ПДн [27]. Переменные окружения валидируются схемой при старте (2.3).

Реализованная серверная часть удовлетворяет требованиям, сформулированным в подразделе 1.7, и содержит средства, необходимые для даль-

нейшего внедрения и сопровождения платформы. Особенности реализации модуля изолированного исполнения программного кода и алгоритма формирования персональных рекомендаций рассматриваются в подразделах 2.9 и 2.10.

2.8 Реализация клиентской части

Программная реализация клиентской части выполнена на языке программирования TypeScript с использованием библиотеки построения пользовательского интерфейса React и сборщика Vite. Объявленные версии указаны в таблице 2. Структура исходного кода клиентской части соответствует принципам организации, рассмотренным в подразделе 2.4. Компоненты пользовательского интерфейса разделены по уровням детализации, обращения к серверной части сосредоточены в выделенном служебном слое, а сквозные сведения хранятся в небольшом числе сквозных контекстов библиотеки React.

Корневой каталог исходного кода клиентской части содержит точку входа приложения, корневой компонент маршрутизатора и подкаталоги отдельных тематических групп. Подкаталог компонентов уровня страниц содержит крупные компоненты, привязанные к маршрутам. Подкаталог общих компонентов объединяет повторно используемые элементы пользовательского интерфейса (макетные компоненты, элементы формы, диалоговые окна, граничный обработчик ошибок). Подкаталог служебных модулей содержит слой обращения к прикладному программному интерфейсу, разделённый по тематическим ресурсам (учётные записи, учебные направления, темы, задания, проверки, интеллектуальное сопровождение). Подкаталог утилит включает вспомогательные функции преобразования данных, выполнения программного кода в изолированном кадре и предварительного просмотра разметки документа. Подкаталог сквозных контекстов задаёт глобальные сведения о выбранном учебном направлении и сведения о текущем пользователе.

Подкаталог настроек темы оформления содержит систему переменных каскадных стилей и одноимённый объект на языке TypeScript.

Точка входа приложения выполняет инициализацию подсистемы регистрации ошибок Sentry, монтирование корневого элемента в страницу-каркас, поставляемую сборщиком Vite, и применение строгого режима библиотеки React. Корневой компонент описывает состав маршрутов с использованием функционального построителя маршрутов библиотеки react-router-dom (версия в таблице 2). Маршруты разделены на четыре группы, соответствующие группам, описанным в таблице 4, и заключены в макетные компоненты, выполняющие проверку активного сеанса и состояния прохождения вводного тестирования.

Защита защищённых маршрутов реализована в макетном компоненте, выполняющем запрос сведений о текущем пользователе при первом монтировании и переходящем в одно из трёх состояний («загрузка», «пользователь аутентифицирован», «пользователь не аутентифицирован»). В первом случае отображается индикатор загрузки. Во втором отображается макет страницы пользователя с навигационной панелью и слотом для дочернего маршрута. В третьем выполняется перенаправление на страницу аутентификации с сохранением целевого адреса. Сокращённая реализация компонента защищённых маршрутов представлена в листинге 2.3.

Листинг 2.3 – Сокращённая реализация компонента защищённых маршрутов

```
1 type AuthState = 'loading' | 'authenticated' | 'unauthenticated';
2
3 export default function ProtectedLayout() {
4   const location = useLocation();
5   const [authState, setAuthState] = useState<AuthState>('loading');
6   const [user, setUser] = useState<AuthUser | null>(null);
7
8   useEffect(() => {
9     let cancelled = false;
10    getMe()
11      .then((u) => { if (!cancelled) { setUser(u); setAuthState('authenticated'); } })
12      .catch(() => { if (!cancelled) setAuthState('unauthenticated'); });
13    return () => { cancelled = true; };
14  }, []);
15
16  if (authState === 'loading') return <div className="guard-loading">Загрузка...</div>;
17  if (authState === 'unauthenticated') return <Navigate to="/login" replace />;
18
19  return (
20    <div className="protected-layout">
21      <Sidebar user={user} pathname={location.pathname} />
22      <main className="protected-layout__main"><Outlet /></main>
23    </div>
24  );
25 }
```

Слой обращения к прикладному программному интерфейсу. Все обращения клиентской части к серверной части сосредоточены в выделенном служебном слое, реализованном на основе библиотеки HTTP-клиента Axios. Создание экземпляра клиента сопровождается определением базового адреса серверной части, согласованного со средой исполнения, и установкой признака передачи защищённых cookie-файлов. Перехватчики запроса выполняют добавление резервного заголовка авторизации в случае отсутствия cookie-

файла. Перехватчики ответа выполняют унифицированную обработку ошибок прикладного программного интерфейса. Фрагмент создания типизированного клиента HTTP представлен в листинге 2.4.

Листинг 2.4 – Фрагмент создания типизированного клиента HTTP клиентской части

```
1 export function createApiClient(): AxiosInstance {
2   const client = axios.create({
3     baseURL: getBaseURL(),
4     headers: {
5       'Content-Type': 'application/json; charset=utf-8',
6       Accept: 'application/json; charset=utf-8',
7     },
8     withCredentials: true,
9   });
10
11   client.interceptors.request.use((config: InternalAxiosRequestConfig) => {
12     if (!config.headers['Authorization']) {
13       try {
14         const token = localStorage.getItem('access_token');
15         if (token) config.headers['Authorization'] = 'Bearer ${token}';
16       } catch { /* private mode / SSR */ }
17     }
18     return config;
19   });
20
21   client.interceptors.response.use(
22     (response) => response,
23     (error) => { /* унифицированная обработка ошибок API */ throw error; },
24   );
25   return client;
26 }
```

API-слой, состояние, редактор и сборка. По одному модулю на REST-ресурс, типы совпадают с серверными DTO. Глобальные сведения задаются через React Context (пользователь, направление из localStorage). Состоя-

ние форм держится в локальном `useState`. Редактор построен на `@uiw/react-codemirror` (CM6, JS/TS/HTML) с автосохранением черновика [37]. HTML-превью выполняется во `iframe` с жёсткой изоляцией. Итоговая проверка выполняется на сервере (2.9). `Error boundary` и сеть уходят в Sentry с минимизацией ПДн [27,52]. Сборщик Vite выполняет `code-splitting` по ролям и тяжёлым плагинам [36].

Реализованная клиентская часть удовлетворяет функциональным требованиям, сформулированным в подразделе 1.7, обеспечивает поддержку всех ролей пользователей и согласована со спроектированной серверной частью, рассмотренной в подразделах 2.3 и 2.7. Особенности реализации модуля изолированного исполнения программного кода и алгоритма формирования персональных рекомендаций рассматриваются в подразделах 2.9 и 2.10.

2.9 Реализация модуля безопасного исполнения программного кода

Модуль безопасного исполнения программного кода является ключевой составляющей подсистемы автоматической проверки решений. Его назначение состоит в выполнении пользовательского программного кода в изолированной вычислительной среде, ограниченной по времени и объёму памяти, с последующим сопоставлением результатов работы кода с эталонными значениями, описанными в составе тест-спецификации учебного задания. Реализация модуля выполнена с учётом обоснований, приведённых в подразделах 1.5 и 1.6.

Модуль реализован в виде самостоятельного служебного компонента серверной части, принимающего на вход исходный программный код пользователя и описание тест-спецификации в формате JSON и формирующего на выходе структурированный результат, содержащий обобщённый статус проверки, количество пройденных и не пройденных тестовых сценариев, перечень детальных сведений по каждому сценарию и при необходимости тек-

стовое описание ошибки. Модуль поддерживает два класса учебных заданий. Задания на языке программирования JavaScript выполняются в обособленных контекстах виртуальной машины V8 средствами модуля `isolated-vm` [29]. Задания на языке разметки HTML разбираются и сопоставляются с тест-спецификацией с использованием программной библиотеки `linkedom`, эмулирующей в среде серверного исполнения объектную модель документа. Выбор ветви исполнения определяется значением соответствующего поля тест-спецификации, что обеспечивает единообразный программный интерфейс модуля для различных типов учебных заданий.

Цикл проверки включает следующие этапы. Валидация JSON-спеки (не более 100 КБ). Выбор ветви JS или HTML. Для JS используется изолят V8 с лимитами памяти и времени, глобальные `eval` и `Function` отключены. Компиляция и запуск пользовательского кода. Поиск имени целевой функции по правилам идентификатора. Прогон тестов с тайм-аутами на каждый сценарий. Агрегированный результат без раскрытия эталона. В блоке `finally` выполняется `dispose` изолята.

Сокращённая реализация ключевого этапа (исполнение программного кода на языке JavaScript в обособленном контексте виртуальной машины) представлена в листинге 2.5.

Листинг 2.5 – Исполнение программного кода в обособленном контексте виртуальной машины V8

```
1 let isolate: ivm.Isolate | null = null;  
2 try {  
3   isolate = new ivm.Isolate({ memoryLimit: memoryLimitMb });  
4   const context = isolate.createContextSync();  
5   const jail = context.global;  
6   jail.setSync('eval', undefined);  
7   jail.setSync('Function', undefined);  
8  
9   const userScript = isolate.compileScriptSync(userCode);  
10  userScript.runSync(context, { timeout: timeoutMs });  
11  
12  let passed = 0;  
13  const details = [];  
14  for (const test of tests) {  
15    const argsStr = (test.input ?? []).map(serializeArg).join(', ');  
16    const callCode = `${functionName}(${argsStr})`;  
17    try {  
18      const actual = context.evalSync(callCode, { timeout: timeoutMs, copy: true });  
19      const eq = test.checkType  
20        ? typeof actual === test.expected  
21        : valuesEqual(actual, test.expected);  
22      if (eq) passed++;  
23      details.push({ input: test.input, expected: test.expected, actual });  
24    } catch (err) {  
25      details.push({ input: test.input, expected: test.expected, actual: String(err) });  
26    }  
27  }  
28  return { status: passed === tests.length ? 'passed' : 'failed',  
29    testResult: { passed, failed: tests.length - passed, details },  
30    errorMessage: null };  
31 } finally {  
32   isolate?.dispose();  
33 }
```

Параметры ограничения ресурсов модуля вынесены во внешние переменные окружения и принимают значения по умолчанию, согласованные с требованием NFR-02 (подраздел 1.7). Максимальная продолжительность исполнения равна 5 секундам, максимальный объём памяти обособленного контекста равен 128 МБ. Указанные значения являются достаточными для выполнения учебных программ ограниченного объёма и одновременно ограничивают потенциальное воздействие на серверную часть в случае попыток исчерпания вычислительных ресурсов. При необходимости значения ограничений изменяются администратором без модификации исходного кода. В дополнение к ограничениям продолжительности и объёма памяти модуль ограничивает количество тестовых сценариев одного учебного задания значением 100, что предотвращает чрезмерное увеличение совокупной продолжительности проверки.

Помимо штатных средств изоляции, предоставляемых модулем `isolated-vm`, в составе разработанного модуля предусмотрены дополнительные меры защиты от обхода ограничений изолированного контекста:

- а) обязательная проверка имени проверяемой функции по правилу допустимого идентификатора (только латинские буквы, цифры, символы подчёркивания и доллара, начало с буквы или специального символа). Попытки указания иного имени отклоняются на этапе предварительной проверки;

- б) замена глобальных функций `eval` и `Function` значением `undefined` внутри обособленного контекста, что устраняет возможность динамического исполнения произвольных строк программного кода;

- в) ограничение совокупного объёма тест-спецификации значением 100 КБ для предотвращения чрезмерного потребления оперативной памяти на этапе разбора;

- г) использование явного режима копирования возвращаемых значений между обособленным контекстом и основным процессом, что устраняет возможность совместного использования объектов, способных привести к утечке состояния обособленного контекста.

Перечисленные меры согласуются с положениями ГОСТ Р 56939–2024 [17] и каталога OWASP Top 10 в части обработки недоверенного содержимого [26].

Учебные задания на языке разметки HTML обрабатываются с использованием программной библиотеки `linkedom`, обеспечивающей разбор разметки в среде серверного исполнения без обращения к графическому подсистему рендеринга. Для каждого тестового сценария применяется выборка элементов разметки по селектору и сравнение значения указанного свойства с эталонным значением. В составе модуля поддерживается ограниченный белый список читаемых свойств элементов разметки (текстовое содержимое, идентификатор, имя класса, имя тега, значение атрибута и иные читаемые без побочных эффектов свойства), что устраняет необходимость интерпретации произвольных свойств и снижает класс возможных ошибок. Дополнительно ограничены длина селектора (не более 500 символов), общее количество элементов разметки документа (не более 1000) и количество тестовых сценариев (не более 100).

Передача аргументов из тест-спецификации в обособленный контекст осуществляется посредством явной сериализации в форме литералов языка JavaScript. Большинство аргументов сериализуется в формате JSON. Для случаев, в которых аргументом является функция (например, в учебных заданиях, связанных с использованием функций высшего порядка), предусмотрена специальная форма представления, при которой исходный код функции включается в сериализованное представление с заданным префиксом. Подобная схема обеспечивает возможность тестирования функций высшего порядка без изменения общей архитектуры модуля.

Автотесты модуля песочницы покрывают успешные кейсы, синтаксис, тайм-аут, давление на память, вызовы запрещённых глобалов и HTML-селекторы. Гоняются в CI (NFR-04).

Реализованный модуль изолированного исполнения программного кода обеспечивает безопасное выполнение пользовательских решений учеб-

ных заданий в пределах установленных ограничений по времени и объёму памяти, формирует структурированный результат проверки, пригодный для отображения в пользовательском интерфейсе, и удовлетворяет требованиям, сформулированным в подразделах 1.5 и 1.7.

2.10 Реализация алгоритма формирования персональных рекомендаций

Алгоритм формирования персональных рекомендаций реализует обоснованный в подразделе 1.3 подход на основе явных правил, обеспечивающий прозрачность принятия решений, ограниченные требования к объёму исходных данных и возможность последовательной отладки. Алгоритм выполняет две основные функции. Первая состоит в формировании рекомендуемого следующего действия пользователя по теме на основании совокупности результатов учебной активности. Вторая обеспечивает адаптивный подбор вопросов контрольной проверки и сложности учебных заданий с учётом численной оценки освоенности отдельных учебных навыков пользователем [5, 6].

Параметры алгоритма формирования рекомендаций вынесены в отдельную группу настраиваемых сведений модели данных и доступны для изменения администратором без модификации исходного кода. К основным параметрам относятся следующие величины:

а) пороговое значение результата контрольной проверки, при достижении которого пользователю рекомендуется переход к следующей теме (по умолчанию 80 %);

б) пороговое значение результата контрольной проверки, ниже которого пользователю рекомендуется повторное изучение теоретического материала (по умолчанию 50 %);

в) максимальное количество подсказок, после превышения которого пользователю рекомендуется выполнение упрощённых вариантов практических заданий (по умолчанию 5).

Установленные значения параметров согласованы с распространёнными в литературе по педагогическим измерениям рекомендациями к проходным баллам контрольных проверок и допускают уточнение по результатам опытной эксплуатации системы [3].

Для текущей темы пользователя алгоритм рассчитывает рекомендуемое следующее действие на основании четырёх независимо собираемых показателей учебной активности. Показатель S означает результат последней пройденной тематической контрольной проверки в процентах (или значение «не определено», если проверка не пройдена). P задаёт число успешно решённых практических заданий темы. F задаёт число практических заданий темы, в которых пользователь допустил неуспешные попытки. H задаёт число запрошенных пользователем подсказок по теме. На основании указанных показателей и параметров алгоритма θ_{adv} (порог перехода), θ_{th} (порог теории) и H_{max} (максимум подсказок) определяется одно из пяти возможных рекомендуемых действий по правилу.

$$R = \begin{cases} \text{следующая тема (тема завершена),} & \text{если } S \neq \emptyset, S \geq \theta_{adv}; \\ \text{повторное изучение теории,} & \text{если } S \neq \emptyset, \theta_{th} \leq S < \theta_{adv}; \\ \text{упрощённая практика,} & \text{если } F > 0, H \geq H_{max}; \\ \text{следующая тема,} & \text{если } P > 0, F = 0; \\ \text{повторное изучение темы,} & \text{в иных случаях.} \end{cases}$$

Сокращённая реализация алгоритма представлена в листинге 2.6.

Листинг 2.6 – Алгоритм формирования рекомендуемого следующего действия по теме

```
1  const settings = await this.getSettings();
2  const [lastQuiz, attempts, hintsCount] = await Promise.all([ /* quiz, attempts, hints */ ]);
3
4  const score = lastQuiz?.scorePercent !== null ? Number(lastQuiz.scorePercent) : null;
5  const passed = attempts.filter((a) => a.status === 'passed').length;
6  const failed = attempts.filter((a) => a.status === 'failed').length;
7
8  let status: LearningStatus = 'in_progress', next: RecommendedNext;
9  if (score !== null && score >= settings.quizMinScoreToAdvance) { status = 'completed';
    next = 'next_topic'; }
10 else if (score !== null && score >= settings.quizMinScoreTheory) { next = 'more_theory';
    }
11 else if (failed > 0 && hintsCount >= settings.maxHintsBeforeEasier) { next = '
    easier_practice'; }
12 else if (passed > 0 && failed === 0) { next = 'next_topic'; }
13 else { next = 'repeat_topic'; }
14 // ... upsert learningProgress (status, recommendedNext)
```

Состояние пользователя по теме принимает одно из трёх значений («не начато», «в процессе», «завершено») и однозначно определяется по результату работы алгоритма. Значение «завершено» устанавливается при выборе действия «следующая тема» на основании достигнутого результата контрольной проверки. Значение «в процессе» сохраняется во всех прочих случаях. Значение «не начато» ставится при отсутствии указанной темы либо учебной активности по ней. Вычисленные значения сохраняются в строке сущности учебного прогресса с использованием операции «обновить или добавить», что обеспечивает атомарность изменения состояния.

В дополнение к алгоритму рекомендуемого следующего действия в составе системы реализован алгоритм адаптивного подбора вопросов тематической контрольной проверки на основе численной оценки освоенности учебных навыков пользователем. Каждый вопрос сопровождается набором тегов (учебных навыков) и численной сложностью d по пятибалльной шкале. Для

каждой пары «пользователь, навык» в хранилище ведётся показатель освоенности m в диапазоне от 0 (навык не освоен) до 1 (навык считается освоенным). Значение m обновляется после каждого ответа на вопрос, в тегах которого фигурирует данный навык.

При формировании очередной попытки контрольной проверки вопросы отбираются взвешенной случайной выборкой. Для вопроса вычисляется средняя освоенность по его тегам $\overline{m}$ (среднее арифметическое всех m по навыкам, привязанным к вопросу). Вес вопроса в выборке задаётся формулой:

$$w = b \cdot k_d, \quad (1)$$

$$b = \max(0,05, 1 - \overline{m}), \quad (2)$$

$$k_d = 1 + 0,15 \cdot \min(4, d - 1). \quad (3)$$

Чем ниже $\overline{m}$, тем больше коэффициент b и тем чаще попадают вопросы по слабо освоенным навыкам. Нижняя граница $b \geq 0,05$ не обнуляет шанс вопроса полностью. Коэффициент k_d слегка повышает вес более сложных вопросов [6].

После ответа пользователя для каждого затронутого навыка старое значение m заменяется новым m' по правилу:

$$m' = \begin{cases} m + 0,1 \cdot (1 - m), & \text{при верном ответе;} \\ 0,8 \cdot m, & \text{при неверном ответе.} \end{cases}$$

То есть m' это освоенность навыка *после* данного ответа, а m в правой части значение *до* ответа. При верном ответе m увеличивается, но не выше единицы. При неверном уменьшается умножением на 0,8. Ведётся streak по тегам для аналитики. Интерпретация близка к упрощённому ВКТ [63]. На практике задания упорядочены по сложности. Режим «повышенная сложность» даёт мягкую подсказку без блокировок (1.3). Возможные расширения включают ВКТ, коллаборативную фильтрацию и DKT [8, 63, 64] без ломки

архитектуры. Качественный анализ приёмлемости и замечаний пользователей по итогам апробации приведён в третьей главе.

2.11 Выводы

Во второй главе результаты анализа и требований первой главы переведены в проектные решения. Определены границы системы и клиент-серверная декомпозиция, описана модель данных (PostgreSQL, ORM Prisma), спроектированы модульная серверная часть на NestJS, клиентское приложение на React с разграничением доступа по ролям, механизм безопасного исполнения пользовательского кода, шлюз обращений к языковой модели и алгоритмы персонализации на основе явных правил над показателями учебной активности. Сформулированные решения задают состав платформы и принципы её работы на уровне архитектуры и данных и служат основой для реализации, рассмотренной в последующих подразделах настоящей главы.

Сопоставление разработанных решений с требованиями, результаты тестирования, учебной апробации и развёртывания приведены в третьей главе.

3 ОБЕСПЕЧЕНИЕ КАЧЕСТВА, ТЕСТИРОВАНИЕ, БЕЗОПАСНОСТЬ И РАЗВЁРТЫВАНИЕ ПЛАТФОРМЫ

Третья глава отчёта посвящена обеспечению качества и безопасности реализации, проверкам и опытной эксплуатации платформы, а также воспроизводимому развёртыванию в серверной среде. В ней рассматриваются меры снижения рисков при интеграции языковых моделей и учёт объёма обращений к ним, контур статического анализа и сопровождения исходного кода, функциональное и обзорное тестирование безопасности, учебная апробация на ограниченной группе обучающихся и вывод платформы в целевое окружение исполнения на основе контейнерной конфигурации.

3.1 Безопасность интеллектуальных агентов и работа с языковыми моделями

Интеграция больших языковых моделей в учебную платформу расширяет поверхность атаки по сравнению с классическим веб-приложением. Наряду с общими угрозами прикладного уровня (несанкционированный доступ, инъекции, злоупотребление ресурсами) появляются специфические риски, связанные с подменой инструкций, утечкой конфиденциальных данных в запрос к модели, недостоверными или небезопасными ответами и злоупотреблением вычислительными ресурсами поставщика модели [26, 65]. В настоящем подразделе систематизированы принятые в разрабатываемой платформе меры снижения указанных рисков и их соотнесение с требованиями по информационной безопасности, сформулированными в подразделе 1.7 первой главы.

При классификации угроз и определении целевых свойств защиты ориентиром служит перечень OWASP Top 10 for LLM Applications (указан выше). Для разрабатываемой платформы выделены следующие группы угроз, требующих явного учёта при проектировании:

- а) попытки внедрения вредоносных инструкций в пользовательский ввод, воспринимаемые моделью как часть задания;
- б) несанкционированный вызов дорогостоящих сценариев генерации и сопутствующее истощение ресурсов;
- в) утечка персональных или учебных данных через журналы и метаданные запросов;
- г) выдача моделью избыточного объёма исполняемого кода или рекомендаций, противоречащих педагогической политике платформы;
- д) недостоверность автоматически сгенерированного учебного содержания с точки зрения корректности формулировок и эталонов.

Целевые свойства защиты включают ограничение доверия к выходу модели, минимизацию объёма сохраняемых чувствительных данных, разграничение доступа по ролям, ограничение частоты и объёма запросов, а также наблюдаемость обращений к модели.

Контур доступа к модели сочетает ограничение частоты вызовов (throttling на уровне HTTP), настраиваемые суточные лимиты числа обращений к интерактивному помощнику для обучающегося и преподавателя (типовые значения 10 и 20 запросов за календарные сутки по UTC, значение ноль в настройке отключает квоту для роли), отображение остатка нормы в личном профиле пользователя и правила промпт-безопасности. Конечные точки /api/ai защищены сессией; генерация и сохранение артефактов ограничены административным контуром (AdminAiGenerateController). На интерактивные вызовы действуют отдельные квоты частоты. Перед обращением к модели проверяется суммарный объём полей, заданы лимиты токенов и таймауты, тяжёлые конвейеры сериализуются очередью. Системные инструкции запрещают следовать вредоносным вставкам в учебном тексте; пользовательский контекст передаётся в размеченных блоках (<task_description>), что снижает риск смешения инструкций и данных. Секреты и параметры поставщика изолированы в серверной фабрике и конфигурации.

В журналах интерактивных сценариев реализованы минимизация сохраняемых данных и контролируемая публикация результатов генерации. Сохраняются длины и хэши полей вместо полных текстов; к ответам добавляются признаки безопасности для модерации [52]. Сгенерированные задания и проверки сохраняются как неопубликованные до ручной приёмки; длины полей ограничиваются на сервере [59]. Реализованные меры не заменяют полноценный пентест и политику обработки данных у поставщика модели; дополнительные проверки описаны в подразделе 3.4.

Реализованные меры безопасности и журналирования (подраздел 3.1) дополняются количественной оценкой нагрузки на языковую модель. Ниже рассматриваются ограничения на объём пользовательского ввода и ответа, а также фактический расход токенов по сценариям; денежная оценка стоимости вызовов в отчёте не приводится, поскольку для сопоставления конфигураций достаточно показателей в токенах.

Учёт токенов выполняется на стороне поставщика модели в рамках протокола, совместимого с OpenAI Chat Completions. В ответе передаются поля `usage.prompt_tokens` и `usage.completion_tokens`, которые сохраняются в журнале запросов как `tokensIn` и `tokensOut` сущности `AiRequest`. Таким образом, наблюдаемость обращений к модели опирается на единые для всех сценариев метрики и не требует дополнительной эвристики на стороне платформы.

Ограничение длины пользовательского контекста для интерактивных сценариев задаётся в символах. Перед вызовом модели проверяется суммарный объём полей ввода; по умолчанию действует порог 8000 символов (параметр `ai_safety_max_input_chars` в базе данных и то же значение по умолчанию в серверной реализации). Данный порог задаёт верхнюю границу размера текста, включаемого в промпт, и не эквивалентен числу токенов; отображение «символы – токены» определяется токенизатором и языком исходного текста.

Верхняя граница длины генерируемого ответа задаётся параметром `max_tokens` запроса к API. В конфигурации по умолчанию для интерактивных сценариев установлены следующие значения — 2048 (пояснение выделенного фрагмента теории), 1024 (пояснение условия задачи), 1536 (подсказка по коду обучающегося), 2048 (разбор эталонного решения). В административном конвейере для шага составления плана темы используется 600 токенов; для генерации набора заданий и генерации теста по умолчанию допускается до 8192 токенов на каждый вызов, при этом величины могут быть изменены настройками `ai_generate_tasks_max_tokens` и `ai_generate_quiz_max_tokens`. Положительное значение глобальной настройки `ai_max_tokens` переопределяет перечисленные лимиты для интерактивных сценариев.

Фактический расход токенов зависит от объёма теоретического материала, числа генерируемых заданий и вопросов, режима сложности и стохастичности ответа модели. В контуре генерации структурированного JSON при ошибке разбора ответа допускается повторный вызов API, что в худшем случае увеличивает суммарный расход. Для интерактивных сценариев при попадании в кэш повторный вызов поставщика может не выполняться, и учёт новых токенов не производится.

Оценка по экспериментальным данным выполнена по журналу административного интерфейса за 16.05.2026 для облачной модели `qwen/qwen3-coder-next` (провайдер `cloud`, отсутствие попадания в кэш). В таблицах 7–9 на одну строку приходится один успешный запрос без учёта повторов.

Таблица 7 – Расход токенов в интерактивных сценариях (связка из четырёх вызовов — теория и одна практическая задача по JavaScript)

Сценарий	Назначение	Лим.	Вх.	Исх.	Симв.
<code>theory_explain_selection</code>	Пояснение выделенного фрагмента теории	2048	612	221	выд. 1194
<code>task_explain_task_description</code>	Пояснение условия задачи	1024	416	282	усл. 340

Продолжение таблицы 7

Сценарий	Назначение	Лим.	Вх.	Исх.	Симв.
task_debug _user_code	Подсказка по коду обучающегося	1536	551	243	к. 62; у. 340
task_explain _reference_s olution	Разбор эталонного решения	2048	498	387	эт. 193; у. 340

В столбце «Симв.» указаны длины пользовательских фрагментов (симв.) по метаданным журнала: выд. — выделенный фрагмент теории, усл. — условие задачи, к. — код обучающегося, эт. — эталонное решение, у. — условие задачи (при совместной передаче с кодом или эталоном). Величина «Вх.» дополнительно включает системные инструкции и разметку запроса, учитываемые поставщиком в `prompt_tokens`. Суммарно по четырём вызовам получено 2077 токенов на входе и 1133 на выходе.

Таблица 8 – Расход токенов при отдельных административных генерациях (облако, qwen/qwen3-coder-next)

Сценарий	Условия	Вх.	Исх.
generate_tasks	10 заданий, hard	1006	2003
generate_quiz	15 вопросов, hard (один из серии)	1160	3918

Таблица 9 – Расход токенов в административном конвейере (тема «Event loop»; 10 заданий; три теста по 20 вопросов)

№	Сценарий	Вх.	Исх.
1	generate_plan	700	446
2	generate_tasks	656	2155
3.1	generate_quiz (тест 1)	875	3534
3.2	generate_quiz (тест 2)	875	4192
3.3	generate_quiz (тест 3)	875	3765

Суммарно по всем шагам конвейера в таблице 9 (позиции 1, 2 и 3.1–3.3) получено 3981 токенов на входе и 14092 на выходе.

По входу третий тест совпадает с первыми двумя (875 токенов); по выходу величина 3765 находится между результатами тестов 1 и 2 при той же конфигурации (20 вопросов, сложность hard).

3.2 Качество программного кода и инструменты статического анализа

Обеспечение качества исходного кода клиентской и серверной частей платформы согласуется с моделью качества программного обеспечения ГОСТ Р ИСО/МЭК 25010–2015 в части сопровождаемости, надёжности и безопасности за счёт раннего выявления дефектов, единообразия оформления и снижения связности между модулями [51]. Набор инструментальных средств статического анализа и форматирования обоснован в первой главе; в настоящем подразделе фиксируется фактическая организация проверок в репозитории проекта и их связь с требованиями к платформе (в том числе нефункциональным требованием NFR-05 в подразделе 1.7 первой главы).

Статический анализ и форматирование строятся на связке TypeScript, ESLint и Prettier. Сборка клиента (`npm run build`) и сервера проверяет типы и синтаксис [35]. ESLint 9 с `eslint.config.mjs`; для сервера подключены `recommendedTypeChecked` и Prettier-плагин; для клиента задействованы React, react-hooks и `eslint-config-prettier`; `npm run lint` на клиенте без допуска предупреждений [46]. `npm run format` применяет Prettier к TS/TSX и тестам [48].

Дополнительно в контуре качества зарезервированы Stylelint, jscpd и Knip в обосновании стека первой главы; они подключаются при росте стилей или ужесточении CI [47, 49, 50]. Сценарии Snyk в `package.json` обеих частей ищут уязвимости зависимостей и выполняют SAST с порогом «высокая» по расписанию или в конвейере.

Сводное сопоставление инструментов и целей приведено в таблице 10.

Таблица 10 – Инструменты обеспечения качества исходного кода в репозитории проекта

Инструмент	Часть системы	Назначение
TypeScript (компилятор)	клиентская и серверная	Статическая типизация, проверка на этапе сборки
ESLint + typescript-eslint	клиентская и серверная	Правила корректности; на клиенте дополнительно правила React и хуков
Prettier	клиентская и серверная	Автоматическое форматирование, согласование с ESLint
Snyk (сценарии CLI)	клиентская и серверная	Поиск уязвимостей зависимостей и статический анализ кода по расписанию или в конвейере
Stylelint (план)	клиентская (таблицы стилей)	Контроль соглашений по каскадным таблицам стилей при росте объёма стилей
jscpd, Knip (план)	монорепозиторий	Выявление дубликатов и неиспользуемых зависимостей и файлов в контуре непрерывной интеграции

Совокупность описанных практик обеспечивает воспроизводимый минимальный уровень контроля качества исходного кода на рабочих местах разработчиков и в конвейере сборки. Динамическая верификация поведения системы средствами автоматизированного тестирования рассматривается в подразделе 3.3.

3.3 Функциональное тестирование системы

Функциональные проверки опираются на общие подходы к тестированию [66]. Критические пользовательские сценарии согласованы со специфи-

кацией `specs/001-adaptive-learning-mvp/` (подраздел 2.6) и изложены в подразделе 3.3.

Сопоставление результатов испытаний с требованиями первой главы выполняется по таблице A.1 (функциональные требования), абзацу после таблицы A.2 (нефункциональные показатели) и подразделу 1.7 (информационная безопасность). Подтверждение для функциональных и нефункциональных показателей приведено в настоящем разделе; проверки по информационной безопасности рассмотрены в подразделе 3.4. Апробация платформы с участием обучающихся описана в подразделе 3.5.

Под критическим путём понимается минимальная цепочка действий, без которой платформа не выполняет своё назначение с точки зрения конечного пользователя; проверка по такому пути позволяет при ограниченном объёме испытаний рано выявить дефекты в архитектуре и интеграции компонентов, что согласуется с общими положениями о приоритизации тестов, изложенными в начале настоящего подраздела. Роли пользователей заданы в подразделе 1.7. Для обучающегося критическая цепочка включает регистрацию и установление сессии, прохождение вводной диагностики, выбор учебной темы, изучение теоретического материала, выполнение практического задания во встроенном редакторе кода и получение результата автоматической проверки во взаимодействии с изолированной серверной средой; далее возможны обращение к интеллектуальному сопровождению и использование персональных рекомендаций. Для преподавателя цепочка включает аутентификацию, работу с учебным контентом и тест-спецификациями, а также просмотр аналитики по группе. Для администратора цепочка включает управление учётными записями и ролями, настройку параметров песочницы и просмотр журналов событий, связанных с безопасностью. Перечисленные цепочки согласованы с артефактами спецификации в каталоге `specs/001-adaptive-learning-mvp/` и покрываются модульными и сквозными автоматизированными тестами серверной части, дополняемыми ручной проверкой по ролям; инструментарий автоматизации описан ниже в настоящем разделе.

В качестве инструментария на сервере применяются Jest (ts-jest), изоляция сервисов через @nestjs/testing, сквозной профиль test/jest-e2e.json и supertest. Клиент без отдельного набора UI-тестов; вместо автотестов UI используются сборка, линтер (подраздел 3.2), ручные сценарии и опытная эксплуатация.

Модульные тесты серверной части сосредоточены в каталоге backend/src. Двенадцать файлов .spec.ts, 141 модульный тест (Jest). В их числе проверки по аутентификации (регистрация, сессия, восстановление пароля с подменой внешних служб), заданиям и проверкам (валидация, адаптивные веса), онбордингу (теги track:* и выбранный путь), разделам и адаптации, интеллектуальное сопровождение (политики ввода, хэши, разбор ошибок языковой модели, построение промптов). Сводка по модулям приведена в таблице 11. Команда npm test на момент написания завершается успешно.

Таблица 11 – Модули серверной части, охваченные автоматизированными модульными тестами

Модуль / область	Содержание проверок (кратко)
Аутентификация (AuthService)	Регистрация, вход, ошибки клиента, выдача подписи сессии, восстановление пароля, граничные случаи валидации
Практические задания (TasksService)	Правила приёма решений, сообщения об ошибках, согласованность с моделью данных
Контрольные проверки (QuizzesService)	Обработка ответов, вычисление результата, ограничения по повторным попыткам
Адаптивный подбор вопросов (adaptive.spec.ts)	Формулы веса и обновления оценки освоенности навыков (изолированная проверка математики)

Продолжение таблицы 11

Модуль / область	Содержание проверок (кратко)
Вводное тестирование (OnboardingService)	Выбор учебного направления, сохранение ответов, расчёт результата с учётом тегов пути, рекомендация стартовой темы
Учебные разделы (SectionsService)	Доступность разделов, связь с прогрессом пользователя
Адаптация (AdaptationService)	Загрузка порогов из настроек, рекомендуемое следующее действие по теме, запись строки учебного прогресса
Интеллектуальное сопровождение	Политики AiPolicy, построение промптов, разбор ошибок тайм-аута; сценарии AiService с подменой зависимостей
Корневой контроллер (AppController)	Контрольная точка доступности приложения

Сквозное тестирование, оценка покрытия и ручные сценарии дополняют модульные тесты. Профиль `test:e2e` расширяет цепочки вроде регистрации, установления сессии и обращения к защищённому API при отдельной БД; `npm run test:cov` оценивает покрытие; `npm test` в CI ловит регрессии. Markdown, `allu` и визуальная согласованность проверяются вручную по ролям из спецификации. Верификация уязвимостей описана в 3.4.

3.4 Тестирование безопасности системы

Тесты информационной безопасности дополняют функциональные проверки (конфиденциальность, целостность и доступность, конфигурация, пользовательский код и обращения к языковой модели). Методологическая база и перечень нормативных ориентиров приведены в подразделе 1.7 пер-

вой главы; меры для интеллектуального сопровождения систематизированы в 3.1; модуль изолированного исполнения описан в главе 2.

Выполняются анализ зависимостей и SAST, проверка заголовков, троттлинга, лимитов и модуля изолированного исполнения, а также регрессионные тесты (подраздел 3.3). Пентест в рамках ВКР не выполнялся; контур может быть усилен на промышленной эксплуатации.

Сценарии Snyk и строгий TypeScript дополняют ESLint (подраздел 3.2). В точке входа задаются лимит тела запроса, cookie, Helmet для JSON-ответов, CORS с белым списком; проверка выполняется через инспекцию и curl. На критичных маршрутах действует отдельный троттлинг; тесты аутентификации и ручные сценарии по ролям подтверждают отказ при неверных данных и разграничение администрирования.

На уровне данных, исполнения пользовательского кода и разметки Prisma и DTO-валидация снижают риск SQL-инъекций; учебный код выполняется в модуле изолированного исполнения с лимитами по времени и памяти (подраздел 2.9). Markdown рендерится с rehype-sanitize; предпросмотр кода выполняется во встроенном изолированном кадре (iframe), вручную на типовых XSS-вставках.

Сопоставление типовых угроз и выполненных в рамках работы проверок приведено в таблице 12.

Таблица 12 – Типовые угрозы и способы их верификации в разрабатываемой платформе

Класс угрозы	Реализованная мера	Проверка
Нарушение контроля доступа	Подписанный идентификатор сессии, роли, ограничение частоты	Модульные тесты аутентификации, ручные сценарии по ролям
Инъекции и манипуляции данными	Prisma, валидация DTO	Анализ кода, регрессионные тесты API

Продолжение таблицы 12

Класс угрозы	Реализованная мера	Проверка
Отказ в обслуживании	Лимиты тела запроса, троттлинг, очередь тяжёлых вызовов ИИ	Нагрузочные пробные запросы, инспекция конфигурации
Небезопасное исполнение кода	Модуль изолированного исполнения, лимиты времени и памяти	Модульные тесты заданий, ручные попытки обхода (подраздел 2.9)
Межсайтовое выполнение сценариев (в теории)	Санитизация Markdown, изолированный кадр предпросмотра	Ручная проверка вредоносных вставок в учебный текст
Риски больших языковых моделей	Политики ввода-вывода, журналирование, роль администратора для генерации	Подраздел 3.1

Перечисленные меры не гарантируют отсутствие дефектов при изменении зависимостей и конфигурации среды развёртывания; для промышленной эксплуатации рекомендуется закрепить периодичность запуска Snyk, журналирование инцидентов и план реагирования на уязвимости в соответствии с требованиями подраздела 1.7. Практика развёртывания платформы в защищённом контуре с обратным прокси-сервером и наблюдаемостью рассматривается в подразделе 3.6.

3.5 Апробация платформы с участием обучающихся

После функционального и обзорного тестирования безопасности (разделы 3.3 и 3.4) целесообразна проверка платформы в условиях реального учебного процесса. Оцениваются не только отсутствие дефектов, но и удоб-

ство интерфейса, согласованность прогресса с логикой доступа к этапам обучения и приемлемость темпа отклика вспомогательных сервисов. Ниже фиксируется апробация в составе занятия с обучающимися первого курса.

Целью апробации является оценка пригодности платформы для обучения по дисциплине выявление ошибок интерфейса и прикладной логики, а также сбор субъективных оценок удобства и полноты учебного материала при работе вне лаборатории разработчика.

Занятие по курсу клиентского веб-программирования прошло 23 апреля 2026 г.; в тестировании платформы участвовали 16 студентов первого курса. Обратная связь в письменном виде (в свободной форме, в электронной переписке) предоставили 7 респондентов, что составляет 43,8 % от числа участников; такая выборка отражает мнение наиболее активной части аудитории, но не заменяет полного опроса всех присутствовавших.

После работы с платформой обучающимся было предложено описать впечатления, отметить удобство интерфейса, обнаруженные сбои и пожелания; к ряду сообщений прилагались скриншоты (в том числе мобильный доступ в браузере на базе Chromium). Это обеспечило качественный анализ открытых ответов и воспроизводимость замечаний по визуальным примерам. Параллельно использовались объективные данные встроенной панели аналитики администратора и журналов обращений к интеллектуальному сопровождению (агрегаты по учётным записям обучающихся без персональных идентификаторов в таблицах настоящей работы).

Сводные формальные сведения о составе апробации приведены в таблице 13; количественные показатели активности и обобщённые метрики по теории, практике, тестам и ИИ изложены в подразделе 3.5.

Таблица 13 – Сводные сведения об апробации платформы (23.04.2026)

Показатель	Значение
Число участников апробации	16

Продолжение таблицы 13

Показатель	Значение
Число письменных откликов	7 (43,8 % от группы)
Формат обратной связи	текстовые сообщения, частично со скриншотами интерфейса
Среда доступа	веб-клиент; настольные и мобильные браузеры (сообщения респондентов)

По выгрузке панели администратора (состояние после прохождения апробации 23.04.2026) зафиксированы сводные величины, согласованные с числом учётных записей обучающихся $N=16$. Они характеризуют фактическую нагрузку на подсистемы теории, практики, тестирования и интеллектуального сопровождения и дополняют качественные отклики.

Таблица 14 – Сводные показатели использования платформы по данным панели аналитики ($N=16$)

Раздел	Показатель	Значение
Общее	Зарегистрировано обучающихся	16
ИИ (локальная модель ollama/qwen2.5:3b)	Число запросов к модели	58
	Ошибки / тайм-ауты	2 / 0
	Попадания в кэш ответа	6
	Оценки полезности («лайк» / «дизлайк»)	4 / 2

Продолжение таблицы 14

Раздел	Показатель	Значение
	Средняя задержка ответа (по панели)	≈ 151 с ($150,8 \times 10^3$ мс)
ИИ — обратная связь по кнопкам	«Объяснить задание» (лайк / дизлайк / всего)	2 / 2 / 4
ИИ — обратная связь по кнопкам	«Разобрать мой код», «Объяснить решение»	0 / 0 / 0 (нет обращений)
Теория	Просмотры / завершённые чтения / среднее время чтения одного фрагмента	63 / 31 / 58 с
Практика	Проверок за 7 суток / за всё время	155 / 156
	Просмотров эталонных решений; среднее время сессии практики	58; 7 мин 15 с
	Подсказки (7 суток / всего)	0 / 0
Тесты	Среднее время прохождения; средний балл по попыткам	1 мин 30 с; 85,8 %

Высокая средняя задержка ответа ИИ на локальной конфигурации согласуется с субъективными откликами о длительном ожидании; часть запросов при повторении одного и того же сценария обслуживалась из кэша с задержкой порядка миллисекунд. Показатели «за 7 суток» и «за всё время» по практике близки, что указывает на концентрацию активности в период апробации.

Результаты контрольных тестов по учебным темам (девять тем с опубликованными тестами) приведены в таблице 15. Наиболее низкий средний балл наблюдается по теме «Среда выполнения JavaScript»; наибольшие по-

казатели соответствуют темам с акцентом на объекты, асинхронность и колбэки.

Таблица 15 – Средние результаты тестов по темам (панель «Результаты тестов по темам»)

Тема	Средний балл	Среднее время
Объекты, функции-конструкторы и this	100,0 %	18 с
Асинхронность и цикл событий (Event Loop)	100,0 %	2 мин 4 с
Функции обратного вызова (колбэки)	100,0 %	2 мин 6 с
Документ, дерево DOM и выборка элементов	86,7 %	17 с
Условные конструкции	86,7 %	2 мин 43 с
Performance API и замеры времени	80,0 %	1 мин 29 с
Замыкания и таймеры в браузере	80,0 %	46 с
Типы данных, переменные, операторы	76,0 %	1 мин 18 с
Среда выполнения JavaScript	62,9 %	1 мин 37 с

Агрегированные показатели по практическим заданиям в разрезе глав курса приведены в таблице 16. Наибольшая доля неуспешных попыток приходится на ранние задания главы «Среда выполнения JavaScript» (в том числе задание «Приветствие пользователя»), что согласуется с перечисленными в свободных откликах затруднениями при первых шагах и с проблемой сравнения вещественных результатов в автоматических тестах.

Таблица 16 – Практика по главам, успешные попытки, просмотры решений, типичные ошибки

Глава	Успешных / попыток	Просмотров решения	Задание с наибольшим числом провалов	Среднее время
Среда выполнения JavaScript	50 / 102	39	Приветствие пользователя (20)	8 мин 13 с
Типы данных, переменные, операторы	12 / 31	19	Конвертация Цельсий в Фаренгейт (16)	3 мин 20 с
Условные конструкции	4 / 11	0	Стоимость доставки (4)	22 мин 20 с
Замыкания и таймеры в браузере	5 / 6	0	Перевод секунд в миллисекунды (1)	24 с
Асинхронность и цикл событий	6 / 6	0	не выявлено	26 с

За 23.04.2026 в административном журнале обращений к интеллектуальному сопровождению зафиксированы обращения к сценариям пояснения условия задачи, разбора кода обучающегося, пояснения выделенного фрагмента теории и эталонного решения (внутренние идентификаторы task_explain_task_description, task_debug_user_code, theory_explain_selection, task_explain_reference_solution). Для большинства записей в поле статуса завершения указано значение success; единичные сбои соответствуют ошибке

сетевого уровня (TypeError, неуспешный вызов fetch). Задержки генерации при отсутствии попадания в кэш достигают сотен секунд, что подтверждает необходимость информировать пользователя о длительности ожидания и наращивать кэширование повторяющихся запросов.

По совокупности откликов отмечались положительные стороны, в том числе практико-ориентированность курса (последовательное усложнение заданий, сочетание теории и практики, автоматическая проверка кода и контрольных вопросов), наглядность навигации и оформления учебного контента, возможность самостоятельной работы за счёт структурирования разделов, удовлетворённость мгновенной обратной связью по тестам, положительная оценка адаптивных сценариев, а также удобство входа через учётную запись GitHub у части респондентов. Отдельно подчёркивалась мотивационная роль представления учебного процесса в виде траектории («пути») по темам.

Замечания и риски, сформулированные в откликах и подтверждённые примерами в переписке, удобно сгруппировать по направлениям:

а) по синхронизации прогресса и доступа к этапам отмечены несоответствие индикатора прохождения фактическому состоянию, необходимость повторного прохождения тестов при повторном входе и недостаточная прозрачность блокировки практики при незавершённой теории, в том числе отсутствие явного пояснения в интерфейсе;

б) по стабильности клиентского приложения зафиксированы сбои при смене реакций на материалы, некорректное отображение аватара после повторного входа или завершения раздела, малозаметные или неинформативные уведомления об ошибках (в том числе при ограничении частоты запросов к API, код 429), а также низкая читаемость сообщений об ошибках на мобильном экране;

в) по сохранению состояния отмечена потеря введённого кода при переключении вкладок браузера или уходе с экрана задания;

г) по корректности проверки ответов выявлено нестабильное признание верного решения (в том числе после повторных действий пользователя) и

необходимость учёта погрешности представления чисел с плавающей точкой в автоматических тестах;

д) по адаптивной вёрстке и удобству интерфейса отмечены сдвиг или «уезд» блоков контента на отдельных разрешениях и неудобство одновременного просмотра теоретического материала и прохождения теста на одном экране мобильного устройства;

е) по учебному контенту и интеллектуальному сопровождению выявлены слияние абзацев и локальные дефекты оформления в отдельных теоретических фрагментах, а также заметная задержка ответа интерактивного помощника по теоретическим вопросам в отдельные моменты работы.

В совокупности положительная оценка учебной модели и структуры курса сочетается с воспроизводимыми замечаниями по интерфейсу, надёжности клиентской части и сопровождению. Отдельные пункты перечня согласуются с количественными данными подраздела 3.5, в том числе с задержкой ответа интеллектуального помощника и с разбросом результатов по темам и практическим заданиям. Обобщение выводов по апробации и отражение доработок в финальной версии платформы приведены в подразделе 3.5.

Апробация подтвердила педагогическую состоятельность заявленной модели (сочетание теории, тестов и практики с автоматизацией проверки) и одновременно выявила узкие места в надёжности клиентского состояния, согласованности прогресса с правилами доступа к этапам, информативности сообщений об ошибках и обработке граничных случаев в тестах. Количественные показатели таблиц 14–16 согласуются с этими выводами, при этом отмечаются высокая активность по практике и тестам при среднем балле около 86 %. В ходе опытной эксплуатации с обучающимися интеллектуальное сопровождение опиралось на локальную языковую модель, что проявлялось в средней задержке ответа порядка 2,5 мин. Отдельно зафиксированы количественные замеры обращения к облачной модели по журналу запросов (подраздел 3.1). Обобщения ограничены числом участников и долей давших письменный отклик; для статистически значимых выводов о эффективности

обучения потребуются расширенная выборка и отдельный дизайн исследования.

В финальной версии дипломного проекта (программной реализации и учебно-методического наполнения) результаты апробации учтены, устранены или смягчены выявленные дефекты интерфейса и логики там, где они воспроизводились по откликам; повышена прозрачность сценариев доступа к этапам обучения; скорректированы формулировки сообщений об ошибках и правила проверки заданий в случаях, выявленных при апробации; отредактировано оформление теоретических материалов; настроены параметры взаимодействия с модулем интеллектуального сопровождения с точки зрения ожидаемого времени отклика. Таким образом, раздел завершает цикл этапов разработки, верификации, пользовательской проверки и доработки перед описанием воспроизводимого развёртывания в подразделе 3.6.

3.6 Развёртывание платформы в серверной среде

Развёртывание включает доставку контейнеров, секретов и наблюдаемости; обобщённая схема приведена во второй главе (рис. 2). В репозитории представлены `docker-compose.dev.yml` (минимальный состав — PostgreSQL, Ollama, серверная и клиентская части) и `docker-compose.prod.yml` (полный стек).

В производственном контуре Docker Compose [45] развёрнуты PostgreSQL 16 с томом данных и проверкой работоспособности, Redis, Ollama, серверная часть (образ `backend/Dockerfile`), статическая клиентская часть (Nginx), обратный прокси Nginx (`deploy/nginx/`), Prometheus + Grafana и экспортёры; компоненты объединены сетями `backend-net`, `frontend-net`, `monitoring-net`. Секреты и параметры CORS, языковой модели, модуля изолированного исполнения и троттлинга задаются через `.env.production` по шаблону `.env.production.example`. На границе публикуются префиксы `/api/`, `/admin/`, статика, `/api/ai/` с тайм-аутами ~10 мин под локальную языковую

модель, лимиты тела запроса, сжатие, заголовки безопасности, ограничение частоты запросов с учётом X-Forwarded-For. Метрики собираются в Grafana, ошибки при необходимости передаются в Sentry. Миграции выполняются командой `prisma migrate deploy` в контейнере серверной части согласно комментариям в файле `Compose`.

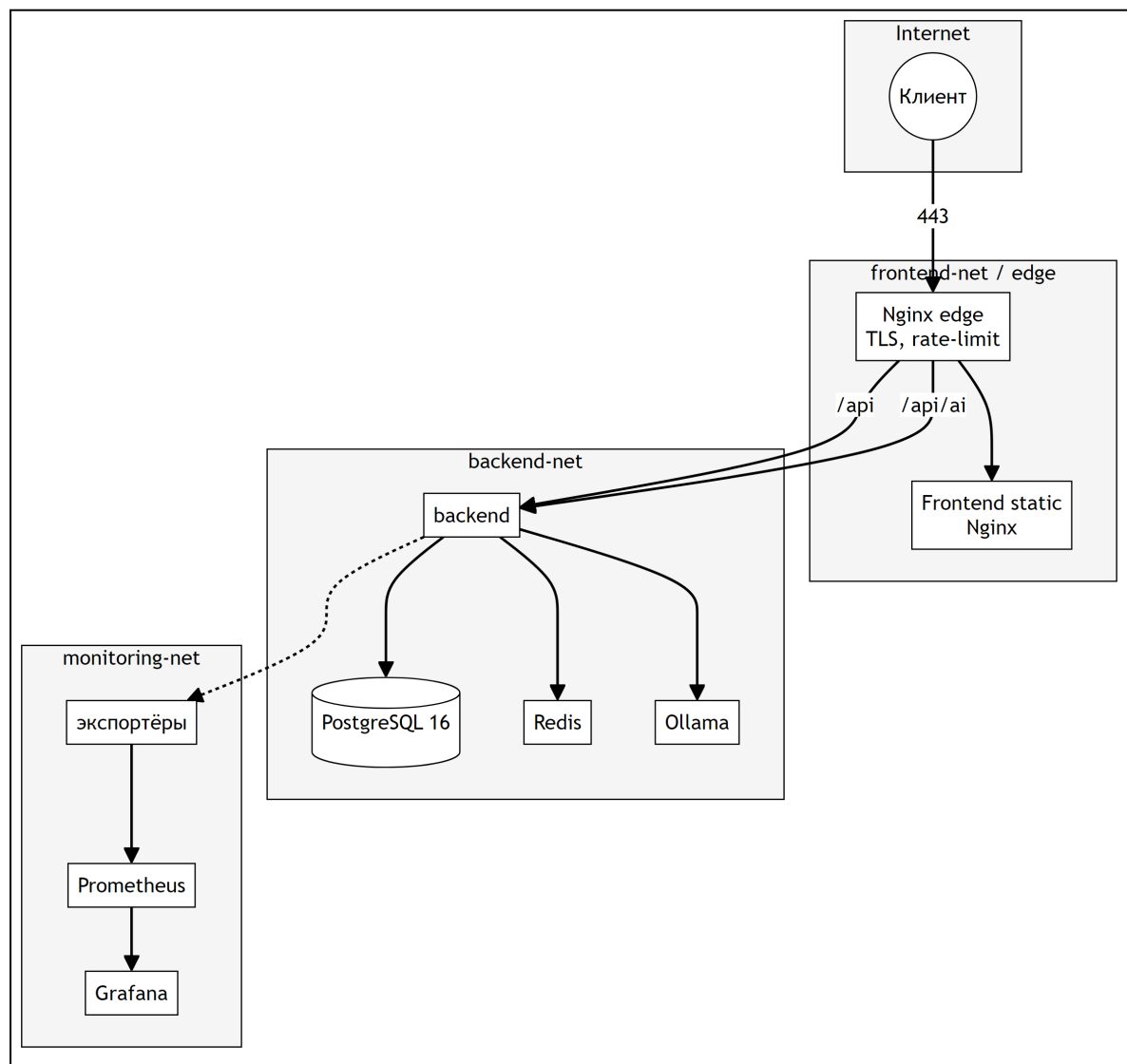


Рисунок 5 – Производственная конфигурация контейнеров платформы (обобщённая схема)

При эксплуатации TLS завершается на границе хоста. Внутри стека сервисы обмениваются данными по HTTP согласно комментариям в `Compose`. Дополнительно требуются резервное копирование тома PostgreSQL, ротация секретов и SLA-мониторинг, не описанные в автоматике репозитория.

3.7 Выводы

В третьей главе проектные решения второй главы соединены с практикой разработки и ввода платформы в ограниченную эксплуатацию. Зафиксированы организационные и технические меры при работе с генеративными моделями (политики данных, ограничения запросов, журналирование), поддерживающие требования безопасности и управляемость расходов на обращения к внешним или локальным языковым моделям, в том числе в разрезе учёта токенов (подраздел 3.1). Для исходного кода описан контур качества, включающий статический анализ, форматирование, модульные автоматизированные тесты серверной части и их место в регрессионной проверке изменений.

Приведены результаты функциональной и обзорной проверки безопасности и показано, как они дополняют формальные требования первой главы наблюдаемыми проверками с опорой на абзац после таблицы А.2 и подраздел 1.7. Учебная апробация и обобщение замечаний пользователей связывают техническую готовность с приёмлемостью интерфейса и сценариев обучения; воспроизводимая контейнерная конфигурация задаёт основу для развёртывания и переносимости окружения, а перечисленные эксплуатационные допущения отмечают границы ответственности автоматизирующего репозитория.

Таким образом, утверждения о работоспособности платформы опираются не только на архитектурное описание, но и на процедуры верификации и на материалы опытной эксплуатации. Заключение обобщает результаты всей работы, формулирует достижение цели и решение задач, зафиксированных во введении, обозначает ограничения текущей версии и направления развития без повторного поэтапного разбора глав.

ЗАКЛЮЧЕНИЕ

Цель выпускной квалификационной работы достигнута: разработана платформа онлайн-курсов программирования на стороне клиента, в которой автоматическая проверка учебного кода в изолированной серверной среде сочетается с персональными рекомендациями по правилам анализа активности и с интеллектуальным сопровождением на базе больших языковых моделей при разграничении ролей и учёте требований информационной безопасности.

По объёму реализации программный комплекс включает клиентскую и серверную части на языке TypeScript. Совокупный объём исходного кода каталогов frontend/src и backend/src (без тестовых файлов) составляет порядка 44 тыс. строк, из них около 26 тыс. на клиенте и 17,6 тыс. на сервере (более 200 файлов .ts и .tsx). Клиентская часть реализована как одностраничное приложение на React 18 и Vite с маршрутизацией react-router, встроенным редактором CodeMirror 6, около 50 объявленных маршрутов, 43 экранами-страницами в каталоге pages/, 17 переиспользуемыми компонентами интерфейса и пятью служебными модулями HTTP-слоя. Серверная часть построена на NestJS с доступом к данным через Prisma и PostgreSQL 16, включает 19 модулей каркаса, пятнадцать функциональных областей прикладной логики (таблица 3), 27 HTTP-контроллеров и 23 прикладные службы. Логическая модель данных описывает 21 сущность и 8 перечислений, для схемы выполнено 30 миграций. Учебное наполнение в каталоге materials/ насчитывает 136 файлов Markdown, спецификация MVP в specs/001-adaptive-learning-mvp/ содержит 13 согласованных артефактов и контрактов API, подсистема интеллектуального сопровождения опирается на восемь файлов промптов. Контейнерная сборка задаёт воспроизводимость среды (файлы docker-compose.dev.yml и docker-compose.prod.yml), производственный стек объединяет 11 сервисов (СУБД, кэш Redis, Ollama, серверная и клиентская части, обратный прокси Nginx, Prometheus, Grafana и экспортёры метрик).

Контроль качества и верификация охватывают статическую типизацию, ESLint и Prettier на обеих частях, сценарии анализа зависимостей Snyk, успешную сборку клиента с нулевым числом предупреждений линтера, 141 модульный тест Jest в 12 файлах серверной части (аутентификация, практические задания и модуль изолированного исполнения, контрольные проверки и адаптивный подбор вопросов, онбординг, учебные разделы и рекомендации, политики и промпты интеллектуального сопровождения), профиль сквозных e2e-проверок, функциональное тестирование по критическому пути пользователей и обзорную проверку безопасности (таблица 12). В основной части отчёта (главы 1–3) для обоснования решений оформлены 20 таблиц, 5 рисунков и 6 листингов фрагментов реализации. Платформа апробирована на занятии с участием 16 обучающихся, собрано 7 письменных откликов, дополнительно проанализированы агрегаты панели администратора и журналов обращений к языковой модели (подраздел 3.5).

Сформулированные во введении задачи по анализу предметной области, постановке требований, проектированию архитектуры и данных, реализации клиентского и серверного контуров, механизма проверки и рекомендаций, интеграции агентов сопровождения и циклу тестирования с развёртыванием решены в объёме, достаточном для получения работоспособного программного комплекса. Соответствие цели подтверждается тем, что разработанная платформа обеспечивает функции, сформулированные в требованиях: автоматическую проверку решений в изолированной серверной среде, персональные рекомендации и интеллектуальное сопровождение с учётом мер информационной безопасности. Это подкреплено перечисленными количественными показателями реализации, модульным и функциональным тестированием, проверками безопасности и апробацией в учебном процессе (глава 3).

Дальнейшее развитие может включать расширение банка учебных задач и сценариев проверки, углубление метрик эффективности обучения, интеграцию с внешними системами управления обучением и оптимизацию об-

ращений к языковой модели при сохранении единой архитектурной основы. На последующих этапах целесообразно расширить поддерживаемые языки в модуле изолированного исполнения, ужесточить политики безопасности, а также отдельно проработать геймификацию (очки, рейтинги и таблицы лидеров), систему достижений и средства общения между обучающимися и преподавателем и между пользователями.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Дятлов, С. А. Цифровизация образования: новые вызовы и возможности / С. А. Дятлов // Высшее образование в России. – 2022. – Т. 31, № 3. – С. 9–22.
2. Лапчик, М. П. Информационные технологии в образовании: учебное пособие / М. П. Лапчик. – Москва: Академия, 2019. – С. 416.
3. Максимов, Н. В. Массовые открытые онлайн-курсы по программированию: анализ образовательных моделей / Н. В. Максимов, О. Л. Голицына // Прикладная информатика. – 2023. – Т. 18, № 4. – С. 91–104.
4. Савкин, Д. А. Интерактивные среды для обучения программированию: сравнительный обзор архитектурных подходов / Д. А. Савкин // Информатика и образование. – 2023. – Т. 38, № 2. – С. 38–49.
5. Касьянова, И. С. Адаптивные системы дистанционного обучения: подходы и архитектуры / И. С. Касьянова // Информационные технологии. – 2021. – Т. 27, № 5. – С. 245–253.
6. Ананьев, И. А. Подходы к построению адаптивных систем электронного обучения / И. А. Ананьев, Т. Ю. Морозова // Открытое образование. – 2021. – Т. 25, № 6. – С. 14–24.
7. Петров, А. Н. Архитектурные решения для систем автоматической проверки программного кода в образовательных платформах / А. Н. Петров, О. С. Климова // Вестник компьютерных и информационных технологий. – 2023. – Т. 20, № 7. – С. 32–41.
8. Шилов, Н. Г. Методы построения рекомендательных систем для образовательных платформ / Н. Г. Шилов, А. М. Кашевник // Программные продукты и системы. – 2022. – Т. 35, № 3. – С. 412–421.
9. Schluntz, E. Building Effective Agents. – Электронные данные. – 2024. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://www.anthropic.com/engineering/building-effective-agents>.
10. Anthropic. Equipping agents for the real world with Agent Skills. –

Электронные данные. – 2025. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://www.anthropic.com/news/skills>.

11. ByteByteGo. Top AI Agentic Workflow Patterns. – Электронные данные. – 2025. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://blog.bytebytego.com/p/top-ai-agentic-workflow-patterns>.

12. Willison, S. OpenAI Skills. – Электронные данные. – 2025. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://simonwillison.net/2025/Dec/12/openai-skills/>.

13. Hugging Face. skills: A repository of agent skills. – Электронные данные. – 2025. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://github.com/huggingface/skills>.

14. Седжвик, Р. Алгоритмы на Java / Р. Седжвик, К. Уэйн. – 4-е изд. изд. – Москва: Вильямс, 2022. – С. 848.

15. Иванова, Г. С. Объектно-ориентированное программирование: учебник / Г. С. Иванова, Т. Н. Ничушкина, Е. К. Пугачёв. – 4-е изд. изд. – Москва: Издательство МГТУ им. Н. Э. Баумана, 2020. – С. 456.

16. Рубцова, Е. В. Алгоритмы автоматической проверки программных решений в системах онлайн-обучения / Е. В. Рубцова, А. В. Гаврилов // Современные информационные технологии и ИТ-образование. – 2022. – Т. 18, № 1. – С. 127–139.

17. Федеральное агентство по техническому регулированию и метрологии. ГОСТ Р 56939–2024. Защита информации. Разработка безопасного программного обеспечения. Общие требования. – 2024.

18. Codecademy. Codecademy. Online learning platform for technical skills. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://www.codecademy.com/>.

19. freeCodeCamp.org. freeCodeCamp. Learn to code – for free. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://www.freecodecamp.org/>.

20. Stepik. Stepik. Образовательная платформа. – Электронные данные.

– 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://stepik.org/>.

21. LeetCode. LeetCode. The world's leading online programming learning platform. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://leetcode.com/>.

22. Codewars. Codewars. Achieve mastery through coding practice and developer mentorship. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://www.codewars.com/>.

23. Attention Is All You Need / A. Vaswani, N. Shazeer, N. Parmar et al. // Advances in Neural Information Processing Systems. – Vol. 30. – New York: Curran Associates, 2017. – Pp. 5998–6008.

24. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus et al. // Advances in Neural Information Processing Systems. – Vol. 33. – New York: Curran Associates, 2020. – Pp. 9459–9474.

25. Scarfone, K. Technical Guide to Information Security Testing and Assessment. NIST Special Publication 800-115. – Электронные данные. – 2008. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://csrc.nist.gov/pubs/sp/800/115/final>.

26. OWASP Foundation. OWASP Top 10:2021. The Ten Most Critical Web Application Security Risks. – Электронные данные. – 2021. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://owasp.org/Top10/>.

27. Google. web.dev. Security and identity. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://web.dev/explore/secure>.

28. National Vulnerability Database. CVE-2025-29927. Next.js Middleware Authorization Bypass. – Электронные данные. – 2025. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://nvd.nist.gov/vuln/detail/CVE-2025-29927>.

29. Laverdet, M. isolated-vm: secure and performant V8 isolation for Node.js. – Электронные данные. – 2026. – [Электронный ресурс]. Дата

обращения: 20.04.2026. <https://github.com/laverdet/isolated-vm>.

30. Государственный комитет СССР по стандартам. ГОСТ 19.301–79. Единая система программной документации. Программа и методика испытаний. Требования к содержанию и оформлению. – 1979.

31. Institute of Electrical and Electronics Engineers. IEEE Std 829–2008. IEEE Standard for Software and System Test Documentation / Institute of Electrical and Electronics Engineers. – New York: IEEE, 2008. – P. 150.

32. Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. – 10-е изд. изд. – Москва: Вильямс, 2019. – С. 1136.

33. Орлов, С. А. Технологии разработки программного обеспечения / С. А. Орлов. – 5-е изд. изд. – Санкт-Петербург: Питер, 2019. – С. 608.

34. Meta Platforms. React. The library for web and native user interfaces. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://react.dev/>.

35. Microsoft. TypeScript Documentation. Handbook. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://www.typescriptlang.org/docs/handbook/intro.html>.

36. Vite. Vite. Next Generation Frontend Tooling. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://vite.dev/guide/>.

37. CodeMirror. CodeMirror 6 Reference Manual. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://codemirror.net/docs/>.

38. NestJS. NestJS. A progressive Node.js framework for building efficient and scalable server-side applications. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://docs.nestjs.com/>.

39. PostgreSQL Global Development Group. PostgreSQL 16 Documentation. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://www.postgresql.org/docs/16/>.

40. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – 8-е изд. изд. – Москва: Вильямс, 2019. – С. 1328.
41. Карвин, Б. Программирование баз данных SQL. Типичные ошибки и их устранение / Б. Карвин. – Москва: Рид Групп, 2020. – С. 336.
42. Prisma Data Inc. Prisma ORM Documentation. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://www.prisma.io/docs/>.
43. Jones, M. JSON Web Token (JWT). RFC 7519. – Электронные данные. – 2015. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://datatracker.ietf.org/doc/html/rfc7519>.
44. Hardt, D. The OAuth 2.0 Authorization Framework. RFC 6749. – Электронные данные. – 2012. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://datatracker.ietf.org/doc/html/rfc6749>.
45. Docker Inc. Compose Specification. Docker Documentation. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://docs.docker.com/compose/compose-file/>.
46. OpenJS Foundation. ESLint. Find and fix problems in your JavaScript code. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://eslint.org/docs/latest/>.
47. Stylelint. Stylelint. A mighty CSS linter that helps you avoid errors and enforce conventions. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://stylelint.io/>.
48. Prettier. Prettier. An opinionated code formatter. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://prettier.io/docs/en/>.
49. jscpd. jscpd. Copy/paste detector for programming source code. – Электронные данные. – 2026. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://jscpd.dev/>.
50. Knip. Knip. Find unused files, dependencies and exports in JavaScript and TypeScript projects. – Электронные данные. – 2026. – [Электронный

ресурс]. Дата обращения: 20.04.2026. <https://knip.dev/>.

51. Федеральное агентство по техническому регулированию и метрологии. ГОСТ Р ИСО/МЭК 25010–2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов. – 2015.

52. Российская Федерация. Законы. О персональных данных : Федеральный закон от 27.07.2006 № 152-ФЗ (ред. от 08.08.2024). – 2006. – № 31 (ч. I). Ст. 3451.

53. Российская Федерация. Законы. Об образовании в Российской Федерации : Федеральный закон от 29.12.2012 № 273-ФЗ (ред. от 24.07.2024). – 2012. – № 53 (ч. I). Ст. 7598.

54. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург: Питер, 2019. – С. 352.

55. Приёмы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. – Санкт-Петербург: Питер, 2020. – С. 368.

56. Влиссидес, Дж. Применение шаблонов проектирования. Дополнительные штрихи / Дж. Влиссидес. – Москва: Вильямс, 2017. – С. 208.

57. Норман, Д. Дизайн привычных вещей / Д. Норман. – Москва: Манн, Иванов и Фербер, 2018. – С. 384.

58. Хант, Э. Программист-прагматик. Путь от подмастерья к мастеру / Э. Хант, Д. Томас. – 2-е изд. изд. – Москва: Лори, 2020. – С. 286.

59. Толоконников, А. В. Применение больших языковых моделей в задачах персонализации обучения программированию / А. В. Толоконников, Е. И. Соколова // Программная инженерия. – 2024. – Т. 15, № 2. – С. 78–89.

60. Anthropic. Model Context Protocol Specification. – Электронные данные. – 2025. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://modelcontextprotocol.io/specification>.

61. Bjelogrić, N. The Agentic Handbook. – Электронные данные. – 2025. – [Электронный ресурс]. Дата обращения: 20.04.2026. <https://www.nibzard.com/agentic-handbook>.
62. OpenAPI Initiative. OpenAPI Specification 3.1.0. – Электронные данные. – 2021. – [Электронный ресурс]. Дата обращения: 26.04.2026. <https://spec.openapis.org/oas/v3.1.0>.
63. Corbett, A. T. Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge / A. T. Corbett, J. R. Anderson // User Modeling and User-Adapted Interaction. – 1994. – Vol. 4, no. 4. – Pp. 253–278.
64. Deep Knowledge Tracing / C. Piech, J. Bassen, J. Huang et al. // Advances in Neural Information Processing Systems. – Vol. 28. – New York: Curran Associates, 2015. – Pp. 505–513.
65. OWASP Foundation. OWASP Top 10 for Large Language Model Applications (GenAI Security Project). – Электронные данные. – 2025. – [Электронный ресурс]. Дата обращения: 26.04.2026. <https://genai.owasp.org/llm-top-10/>.
66. Куликов, С. С. Тестирование программного обеспечения. Базовый курс / С. С. Куликов. – 4-е изд. изд. – Минск: Четыре четверти, 2020. – С. 312.

ПРИЛОЖЕНИЕ А

Требования к проекту

Таблица А.1 – Трассировка функциональных требований к прецедентам UML

Код	П.	Формулировка требования	Прецедент
FR-01	а)	регистрация на платформе и аутентификация с использованием подписанного идентификатора сессии	UC-01
FR-02	б)	просмотр каталога учебных курсов, состава тем и учебных заданий, входящих в выбранный курс	UC-01
FR-03	в)	изучение теоретического материала, представленного в форме структурированных учебных блоков	UC-01
FR-04	г)	выполнение практических заданий с использованием встроенного редактора программного кода	UC-01
FR-05	д)	автоматическая проверка предложенного решения по совокупности тестовых сценариев с предоставлением сведений о результате каждого сценария и обобщённой оценки	UC-02
FR-06	е)	получение персональных рекомендаций по дальнейшему изучению учебного материала, формируемых на основании анализа учебной активности	UC-03
FR-07	ж)	обращение к интеллектуальному сопровождению за разъяснением учебного материала, формированием подсказки по выполнению задания и анализом допущенных ошибок	UC-04

Продолжение таблицы А.1

Код	П.	Формулировка требования	Прецедент
FR-08	з)	просмотр сведений о собственном прогрессе по курсу, включая совокупный показатель завершённых заданий и индивидуальные показатели по темам	UC-03
FR-09	а)	создание учебных курсов, формирование структуры тем и учебных заданий	UC-05
FR-10	б)	подготовка учебных заданий, включающая описание условия, формирование тестовых сценариев и задание ограничений по времени исполнения и объёму памяти	UC-05
FR-11	в)	опубликование подготовленного учебного материала и его последующее редактирование с сохранением версионной истории	UC-05
FR-12	а)	просмотр сведений об учебной активности группы обучающихся, включая агрегированные показатели завершённости тем и распределение допущенных ошибок	UC-05
FR-13	б)	формирование адресных рекомендаций отдельным обучающимся на основании сведений об их учебной активности	UC-05
FR-14	а)	управление учётными записями пользователей и назначение ролей в соответствии с ролевой моделью	UC-06

Продолжение таблицы А.1

Код	П.	Формулировка требования	Прецедент
FR-15	б)	конфигурирование параметров платформы, в том числе ограничений на исполнение пользовательского кода, параметров аутентификации и параметров взаимодействия с внешними сервисами	UC-06
FR-16	в)	просмотр журналов обращений к платформе и сведений о событиях информационной безопасности	UC-06

Таблица А.2 – Нефункциональные требования к разрабатываемой платформе

Код	Характеристика	Содержание требования
NFR-01	Производительность	Среднее время отклика серверной части на типовой запрос пользовательского интерфейса не должно превышать 300 мс при одновременной работе до 50 пользователей.
NFR-02	Производительность проверки	Время выполнения проверки одного решения учебного задания не должно превышать 5 с при ограничении объёма памяти изолированного контекста значением 64 МБ.

Продолжение таблицы А.2

Код	Характеристика	Содержание требования
NFR-03	Надёжность	Доступность серверной части системы должна составлять не менее 99 % в течение учебного семестра при заданных условиях эксплуатации.
NFR-04	Сопровождаемость	Отношение объёма автоматизированных тест-сценариев модульного уровня к объёму исходного кода серверной прикладной логики должно составлять не менее 0,3.
NFR-05	Качество исходного кода	Отсутствие ошибок и предупреждений уровня «error» по результатам проверки статическими анализаторами ESLint и Stylelint в составе процесса непрерывной интеграции.

Продолжение таблицы А.2

Код	Характеристика	Содержание требования
NFR-06	Удобство использования	Адаптивная вёрстка для экранов шириной от 320 пикселей (медиазапросы, гибкие отступы). Учёт рекомендаций WCAG 2.1 (уровень AA) по контрасту, клавиатурному управлению и разметке интерактивных элементов; подтверждается ручными проверками по сценариям ролей в подразделе 3.5.
NFR-07	Локализация	Полная поддержка пользовательского интерфейса на русском языке; формат отображения дат и чисел согласно правилам русской локали.
NFR-08	Переносимость	Корректное функционирование клиентской части в актуальных версиях браузеров на ядрах Chromium, Gecko и WebKit; развёртывание серверной части на операционных системах семейства Linux в среде контейнеризации.

Продолжение таблицы А.2

Код	Характеристика	Содержание требования
NFR-09	Совместимость	Поддержка стандарта обмена данными между клиентской и серверной частями в формате JSON; описание прикладного программного интерфейса в соответствии со спецификацией OpenAPI.
NFR-10	Сопровождаемость зависимостей	Отсутствие в составе зависимостей пакетов с известными уязвимостями уровня «высокий» и «критический» по результатам автоматизированной проверки.

Таблица А.3 – Перечень основных сущностей модели данных

Сущность	Группа	Назначение
Учётная запись пользователя	Учётные записи	Сведения об учётной записи, роль, состояние активности
Токен сброса пароля	Учётные записи	Хранение хэшированных одноразовых токенов восстановления
Журнал действий администратора	Учётные записи	Аудит действий с защищаемыми сущностями
Учебное направление	Каталог материалов	Корневой элемент иерархии (выбор языка программирования)

Продолжение таблицы А.3

Сущность	Группа	Назначение
Раздел учебного направления	Каталог материалов	Группировка тем в составе направления
Тема	Каталог материалов	Логическая единица обучения. Объединяет теорию, задания, проверки
Теоретический материал	Учебные материалы	Структурированный текст в формате Markdown с черновой и опубликованной версией
Фрагмент теоретического материала	Учебные материалы	Подготовленные фрагменты для извлечения по запросу языковой модели
Практическое задание	Учебные материалы	Учебное задание с описанием, начальным кодом и тест-спецификацией
Контрольная проверка	Учебные материалы	Проверочное задание с множественным выбором
Вопрос контрольной проверки	Учебные материалы	Отдельный вопрос проверки с вариантами и пояснениями

Продолжение таблицы А.3

Сущность	Группа	Назначение
Попытка выполнения практического задания	Учебная активность	Сведения о попытке (код, статус, результаты тест-сценариев)
Попытка прохождения контрольной проверки	Учебная активность	Снимок ответов, итоговый процент, рекомендуемая тема
Подсказка	Учебная активность	Контекстная подсказка, выданная пользователю при выполнении задания
Событие телеметрии	Учебная активность	Регистрация событий учебной активности (просмотры, отправки, попытки)
Состояние учебного прогресса	Адаптация	Состояние пользователя по теме (не начато, в процессе, завершено)
Состояние освоенности учебного навыка	Адаптация	Численная оценка освоенности по тегам учебных навыков
Параметры адаптации	Адаптация	Настраиваемые пороговые значения и правила адаптивного алгоритма
Параметры интеллектуального сопровождения	Адаптация	Настраиваемые параметры обращений к языковой модели

Продолжение таблицы А.3

Сущность	Группа	Назначение
Запрос к языковой модели	Интеллект. Сопровождение	Журнал обращений к языковой модели в разрезе сценариев и провайдеров
Ответ языковой модели	Интеллект. Сопровождение	Текстовый результат с признаками безопасности содержимого

ПРИЛОЖЕНИЕ Б

Макеты пользовательского интерфейса

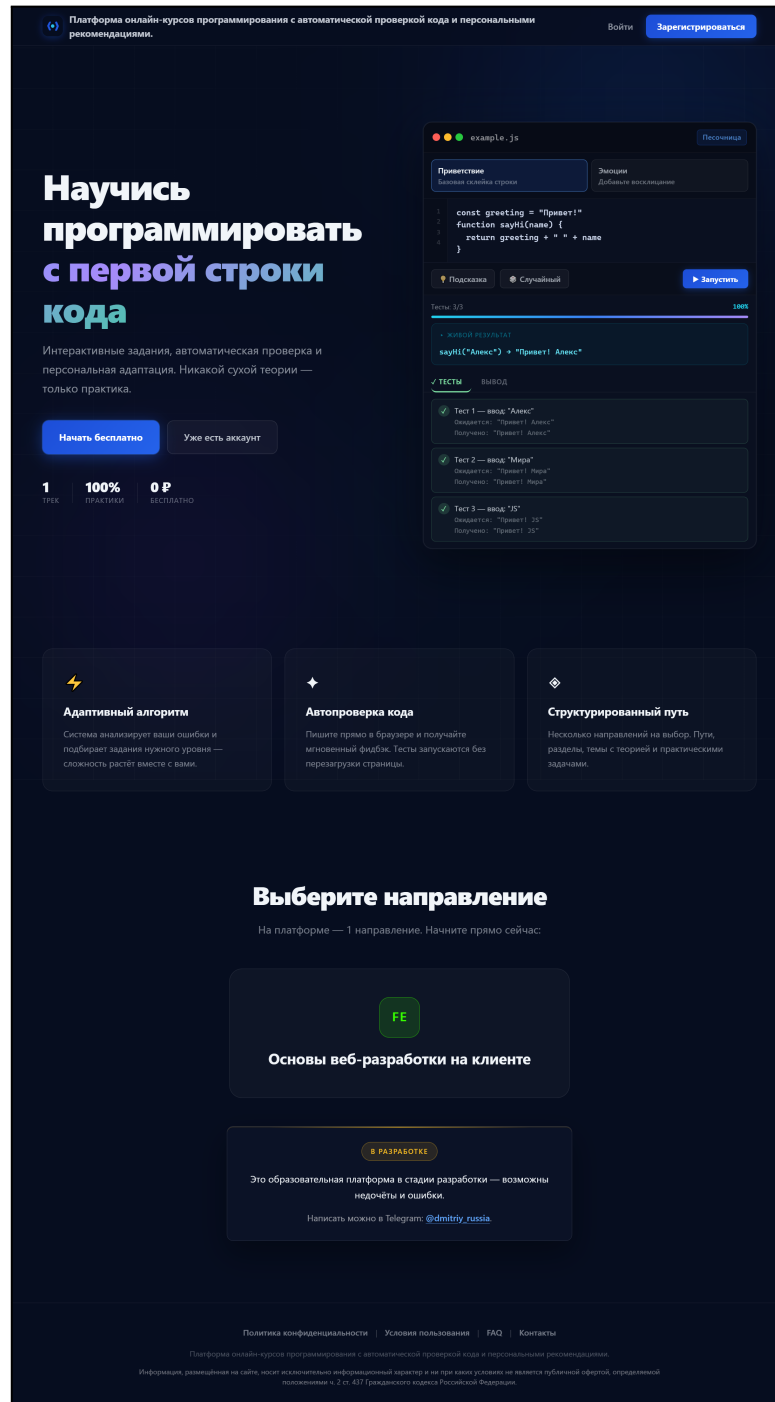


Рисунок Б.1 — Главная страница для неавторизованного пользователя

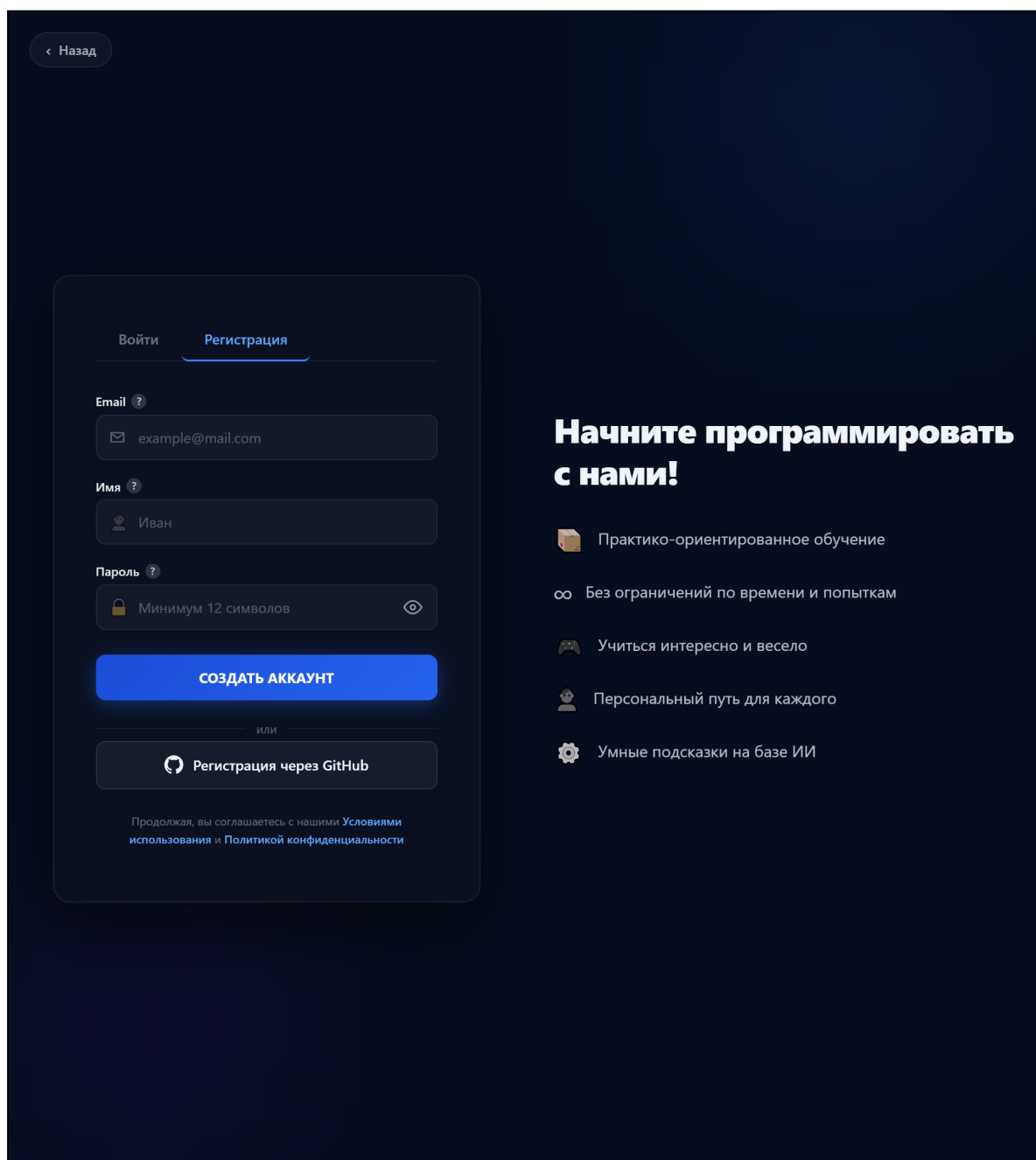


Рисунок Б.2 – Процесс регистрации

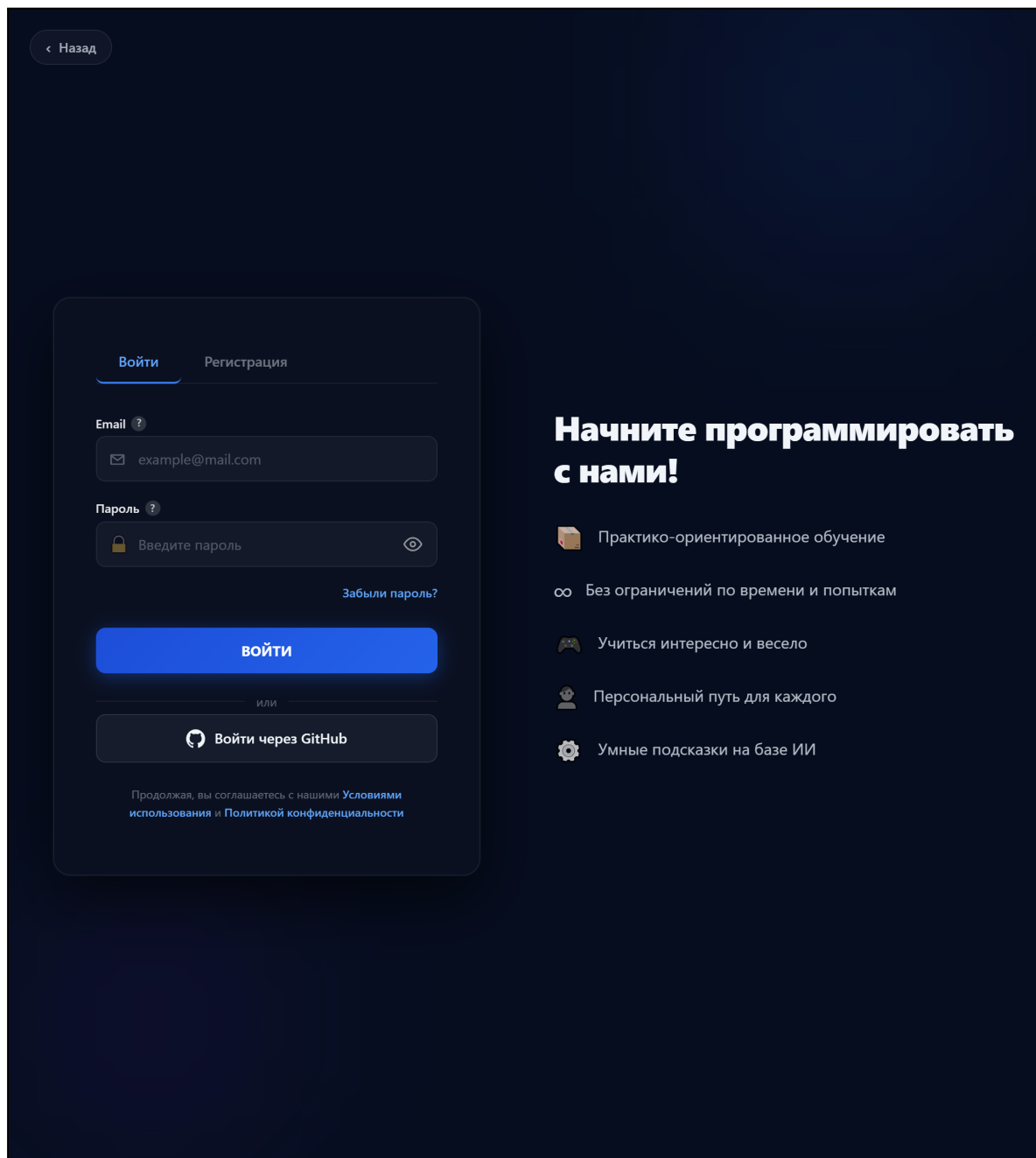


Рисунок Б.3 – Процесс авторизации

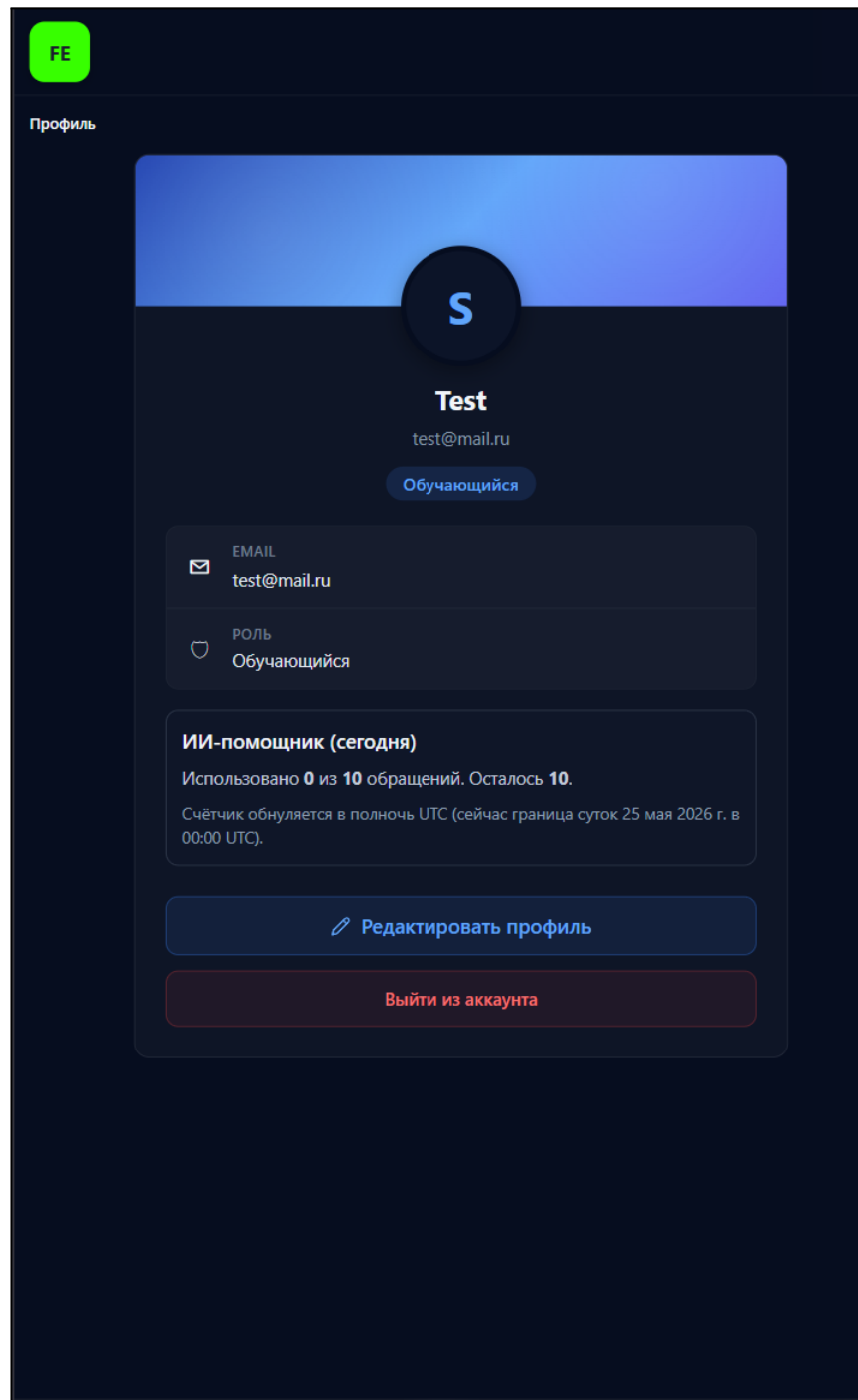


Рисунок Б.4 – Страница профиля пользователя

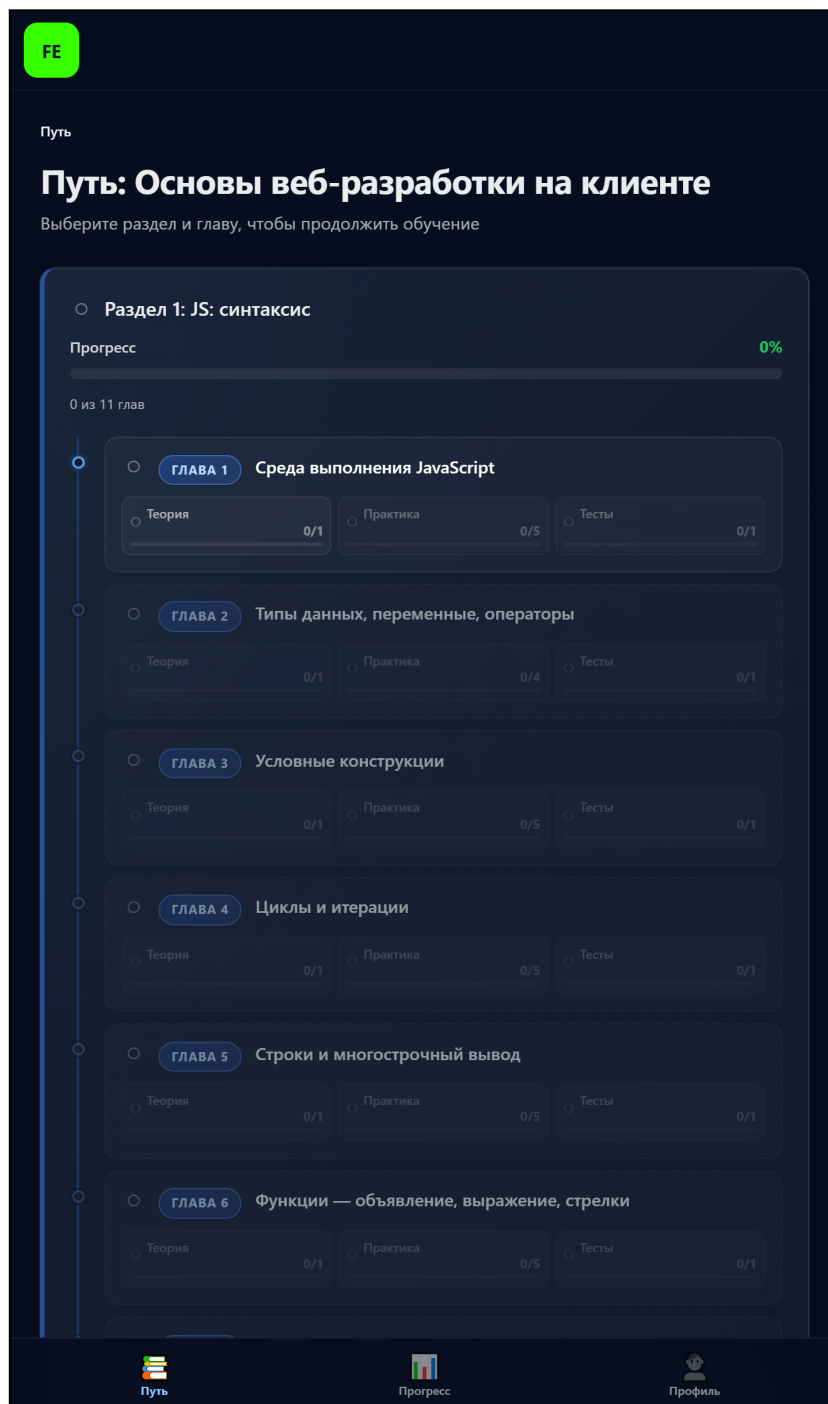


Рисунок Б.5 – Страница учебного пути

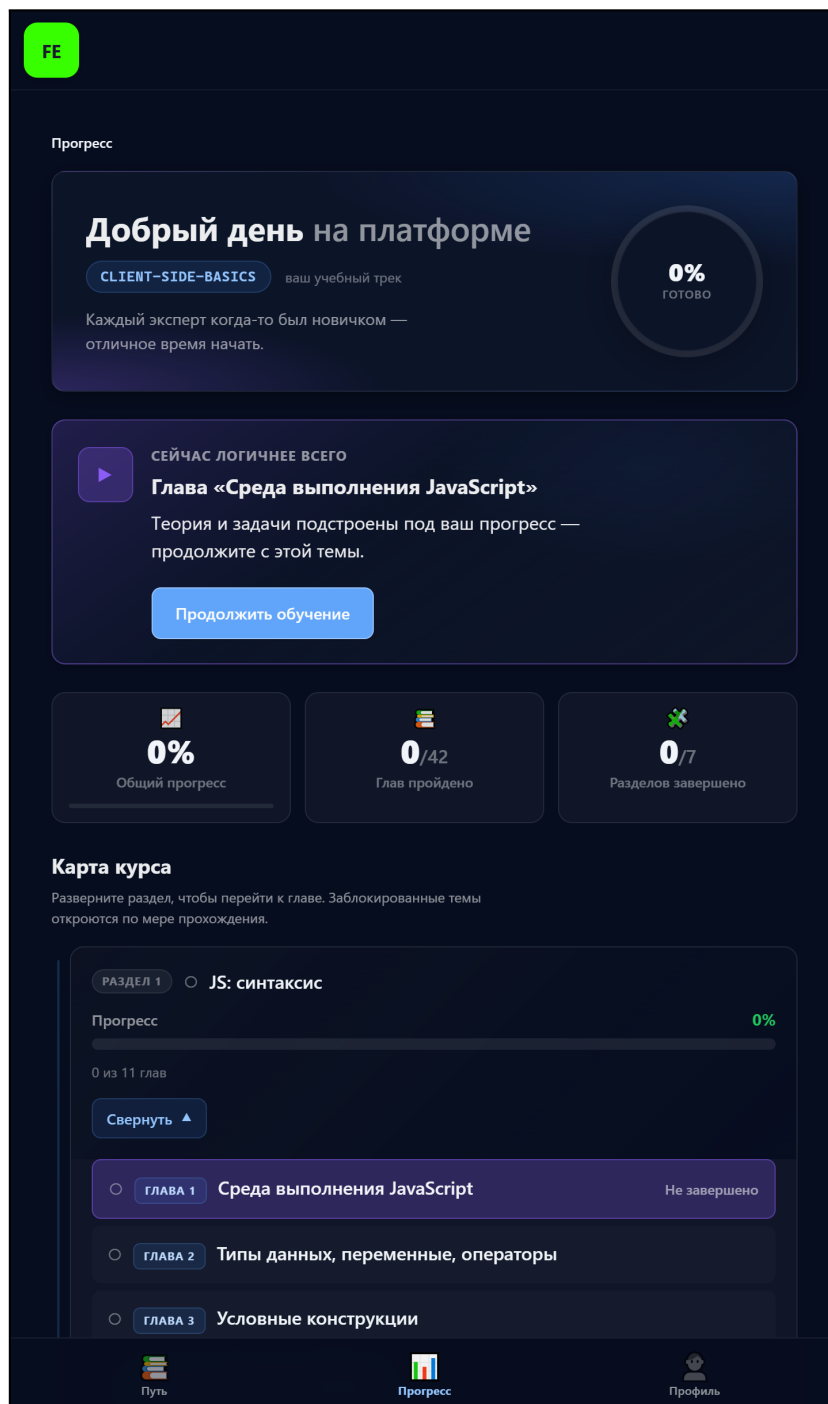


Рисунок Б.6 – Страница прогресса пользователя

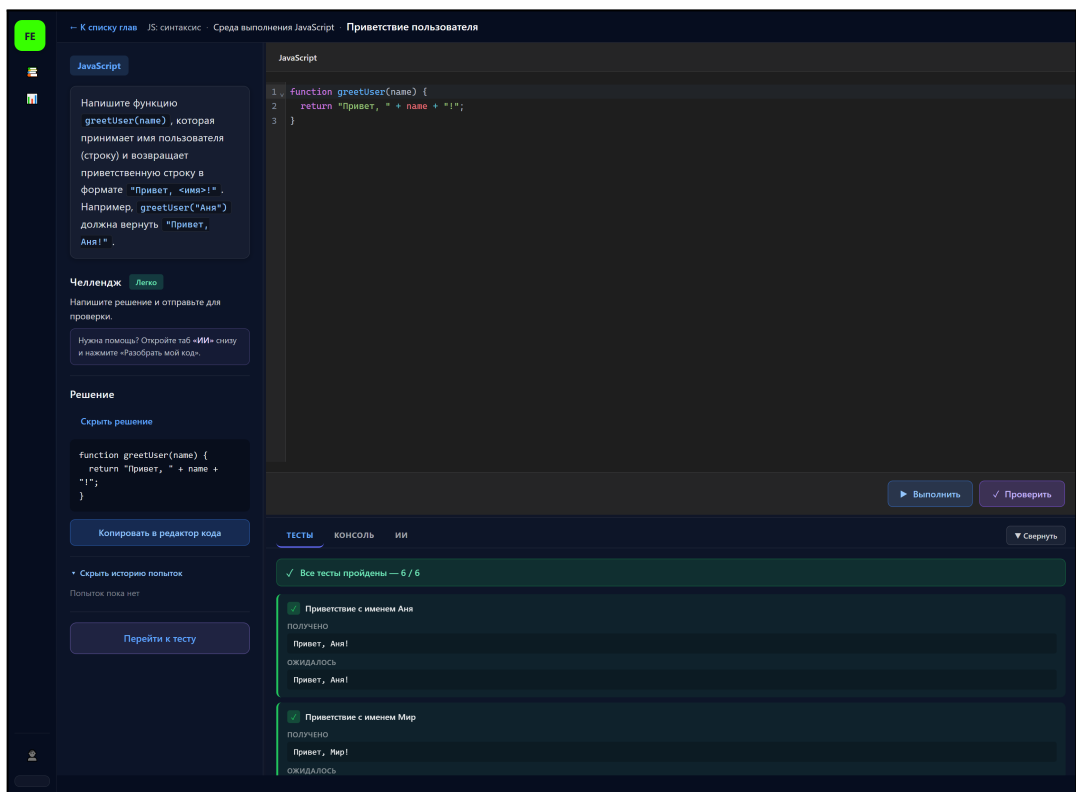


Рисунок Б.7 – Страница решения практического задания

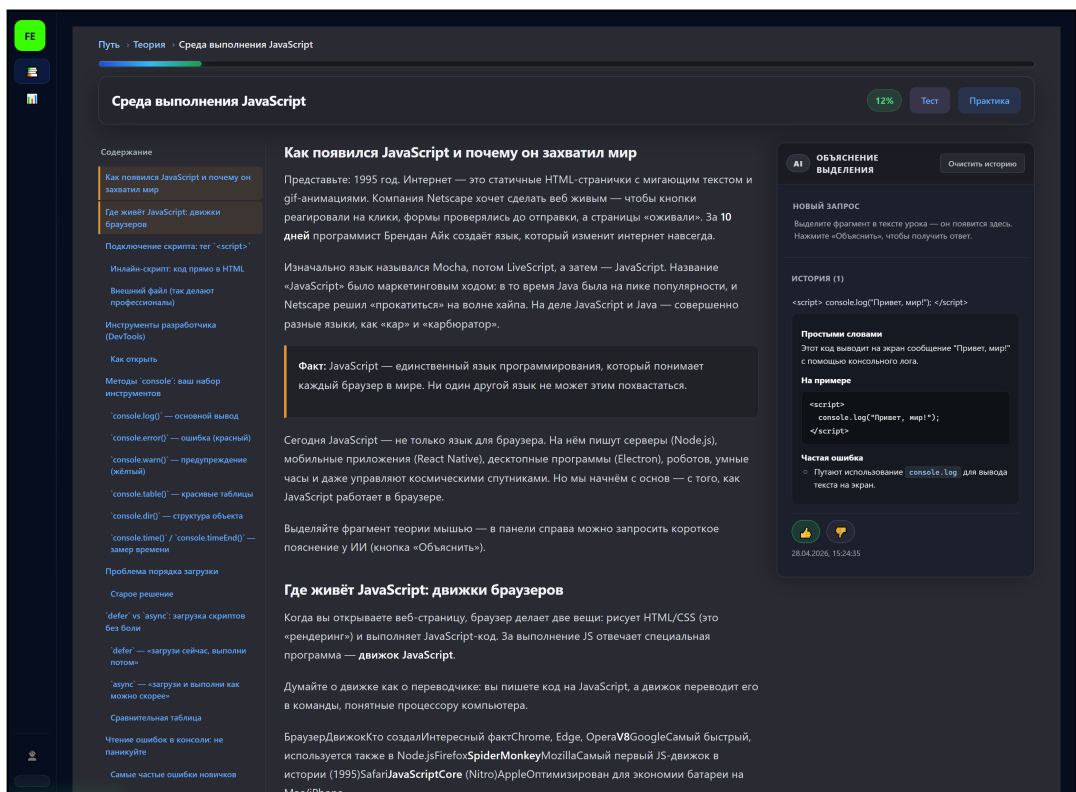


Рисунок Б.8 – Страница просмотра теоретического материала

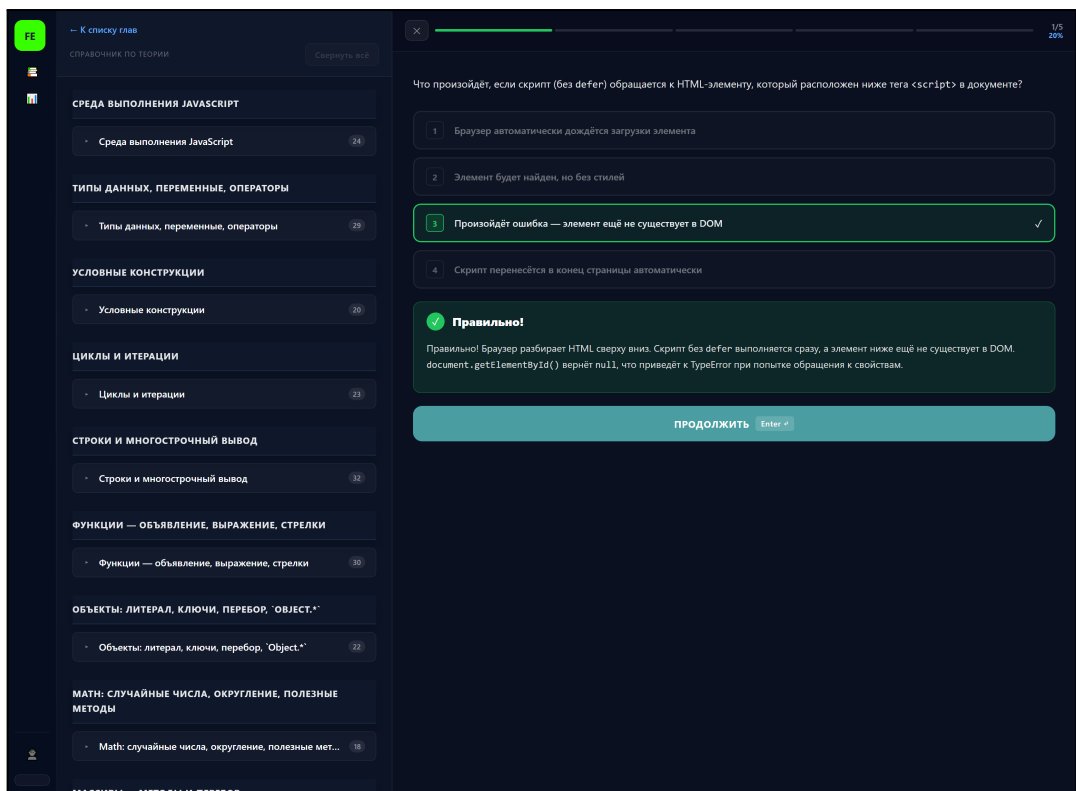


Рисунок Б.9 – Страница начала тестирования

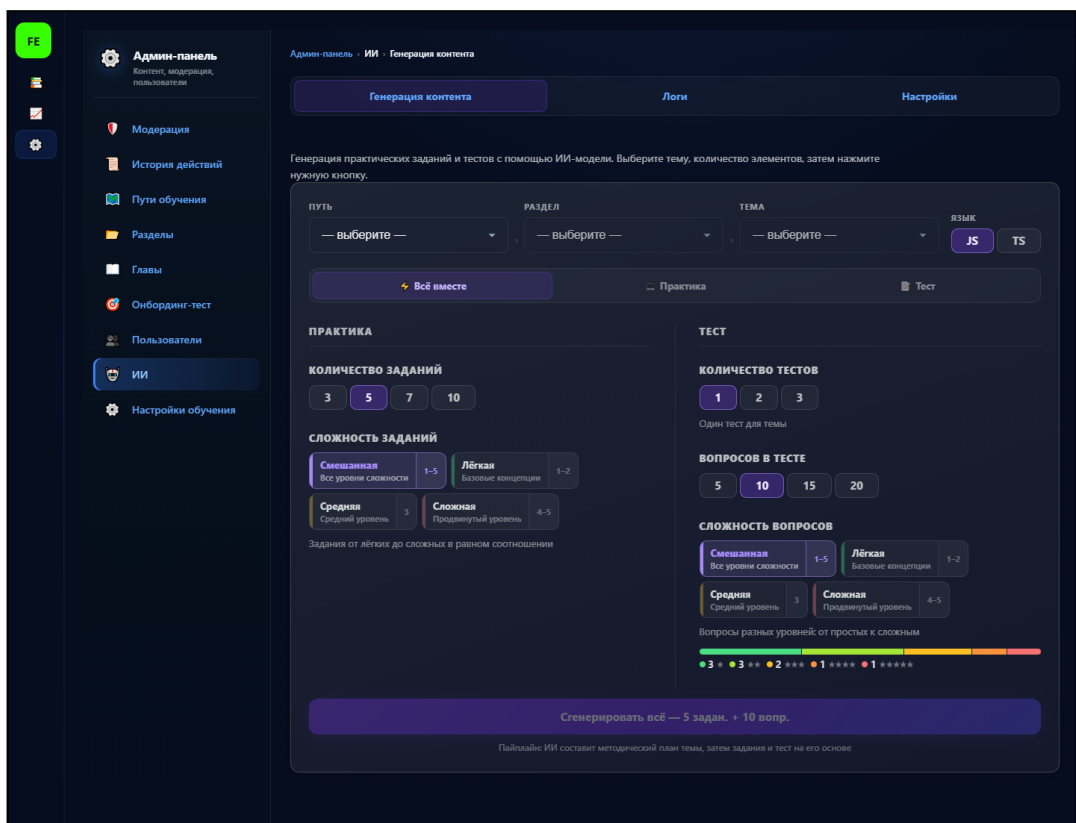


Рисунок Б.10 – Панель администратора

ПРИЛОЖЕНИЕ В

Модели данных, схемы процессов и UML-диаграммы

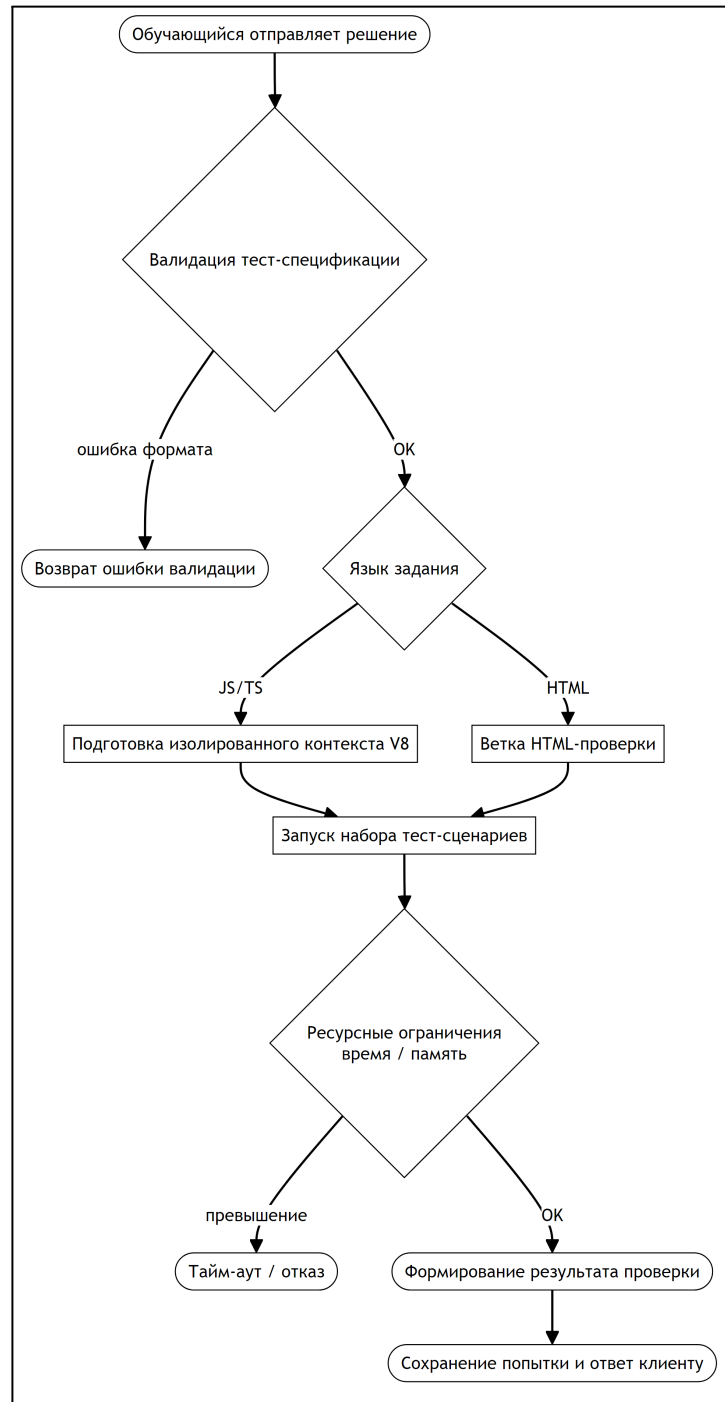


Рисунок В.1 – Упрощённая схема процесса автоматической проверки решения учебного задания

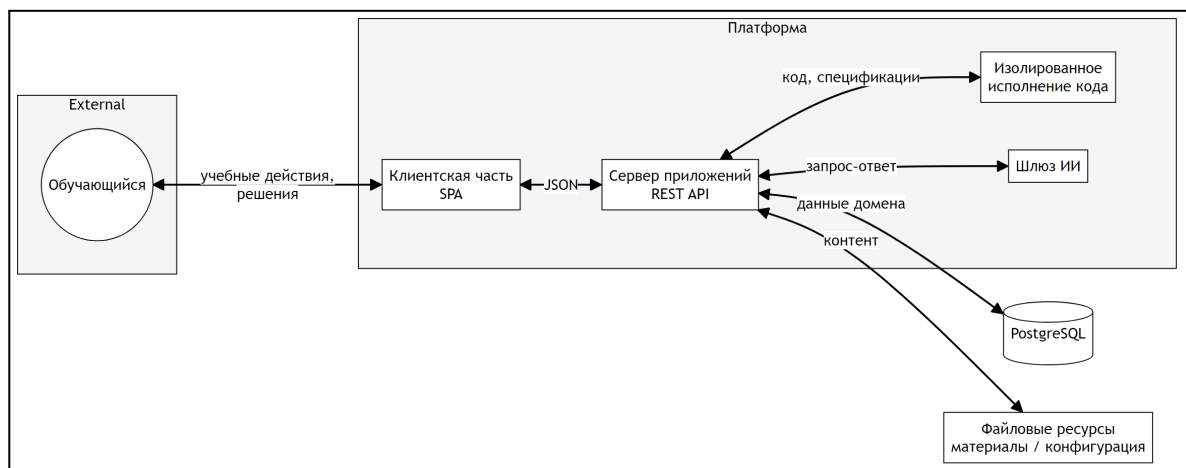


Рисунок В.2 – Укрупнённая диаграмма потоков данных между компонентами платформы

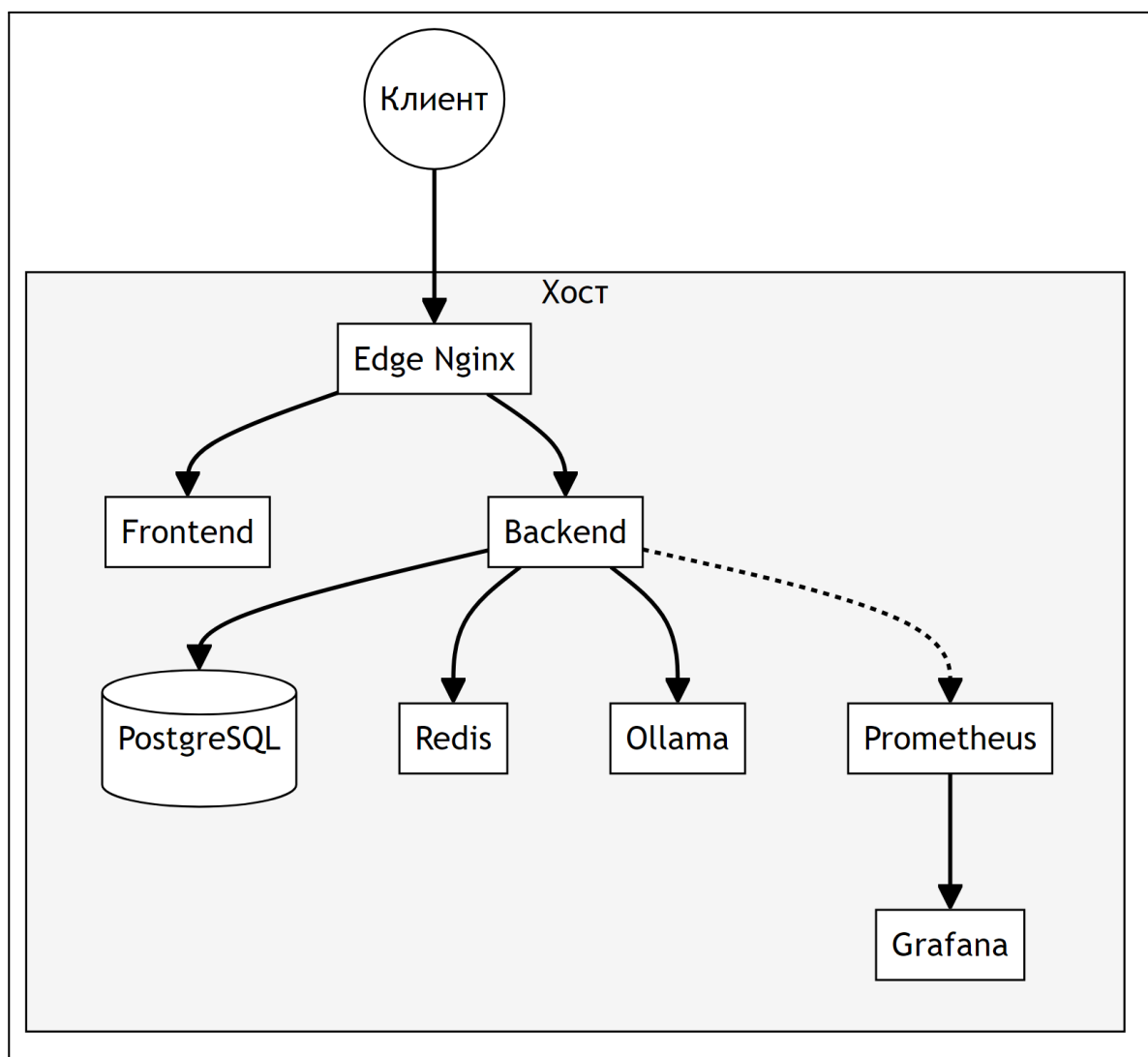


Рисунок В.7 – Схема развёртывания production-стека (по docker-compose.prod.yml)

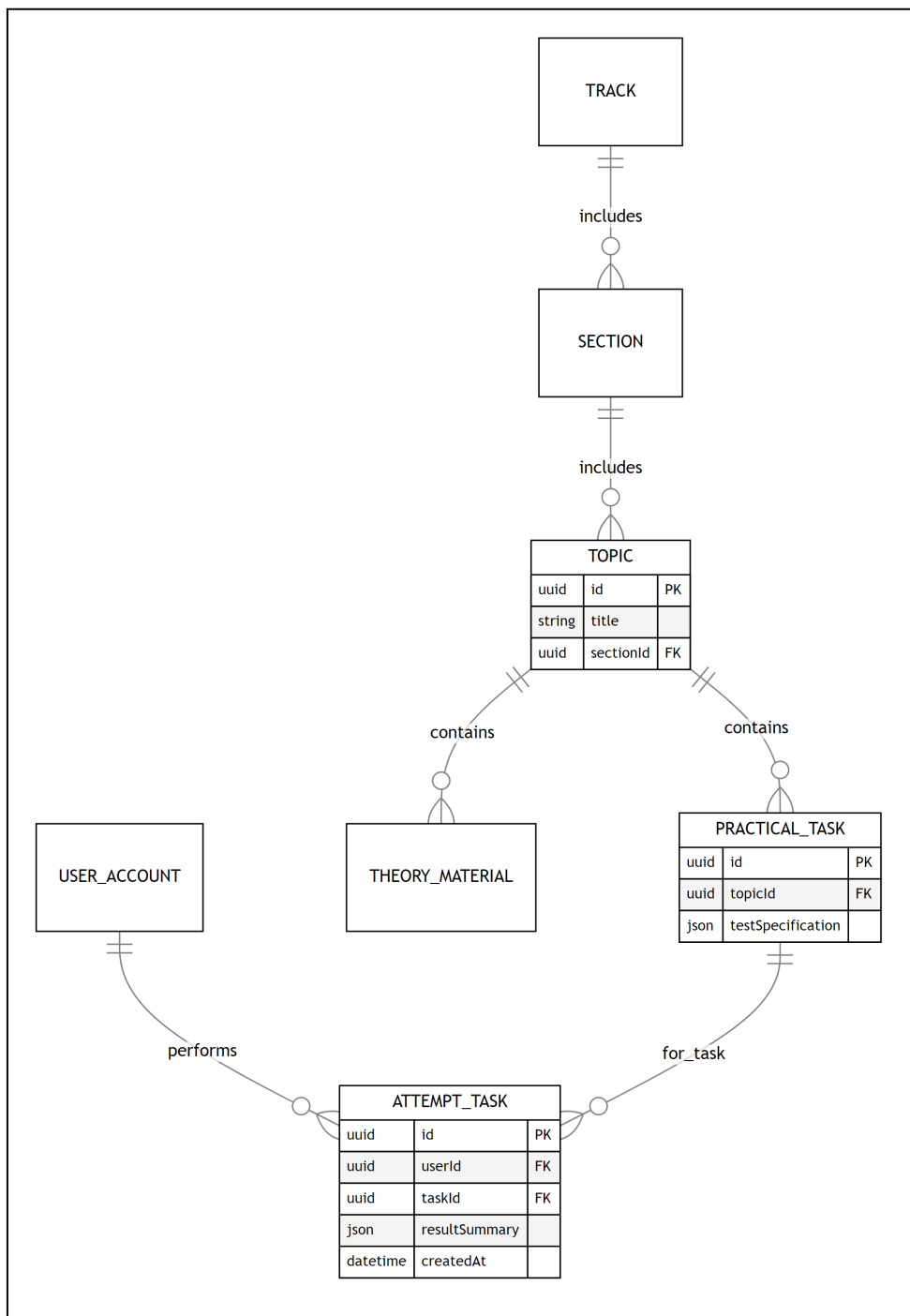


Рисунок В.3 – Концептуальная ER-модель (фрагмент предметной области «учебный контент и попытки»)

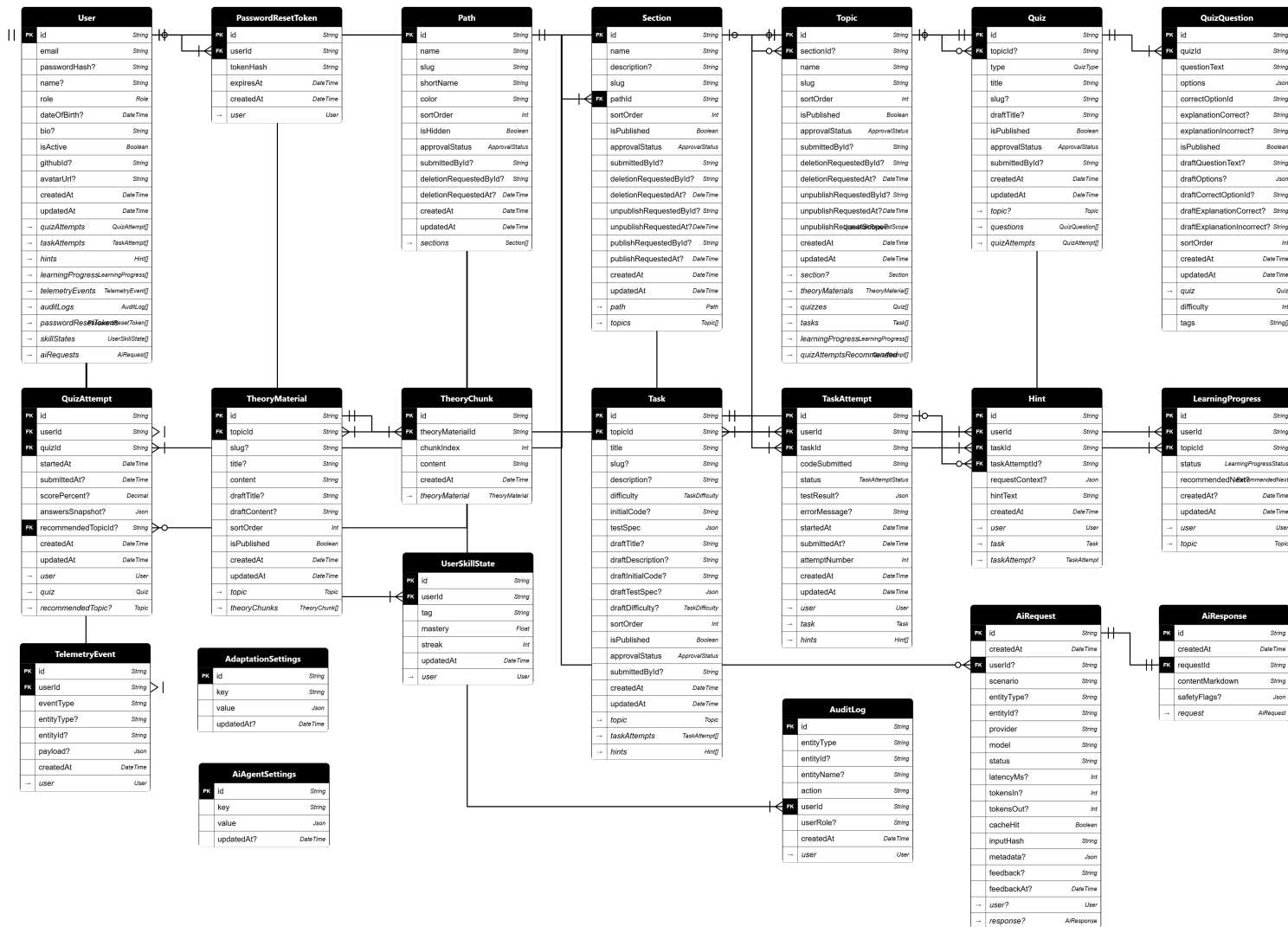


Рисунок В.4 – Дatalogическая модель: основные отношения и атрибуты (по схеме Prisma)

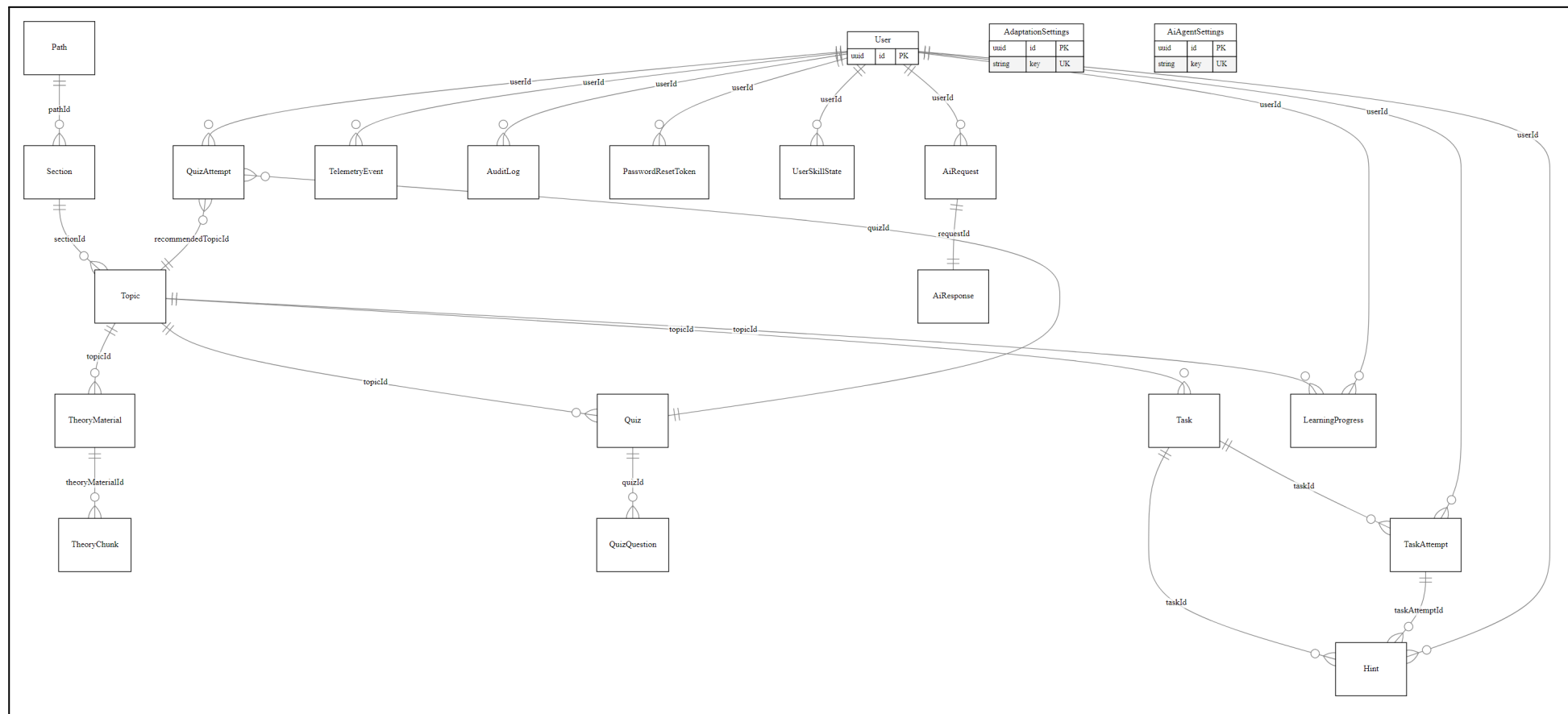


Рисунок В.5 – Физическая модель данных (таблицы и внешние ключи)

Имена таблиц соответствуют объектам в схеме public СУБД PostgreSQL 16 и файлам миграций ORM Prisma (backend/prisma/migrations). На рёбрах диаграммы указаны имена столбцов, по которым заданы внешние ключи (аналогично полям FK в schema.prisma); системные имена ограничений ..._fkey приведены в SQL-миграциях. Связи Topic.sectionId, Quiz.topicId, QuizAttempt.recommendedTopicId, Hint.taskAttemptId и AiRequest.userId допускают NULL там, где это задано в схеме. Узел User показан без полного списка атрибутов (упрощение). Типы полей, ограничения UNIQUE, индексы и значения по умолчанию приведены в SQL-миграциях. Логическая структура отношений приведена на рисунке В.4. Таблицы AdaptationSettings и AiAgentSettings не содержат FK на другие таблицы (хранилище «ключ–значение» для параметров адаптации и интеллектуального сопровождения). Связь запросов к языковой модели с учебными сущностями не задаётся FK: поля scenario, entityType, entityId в AiRequest используются на уровне приложения.

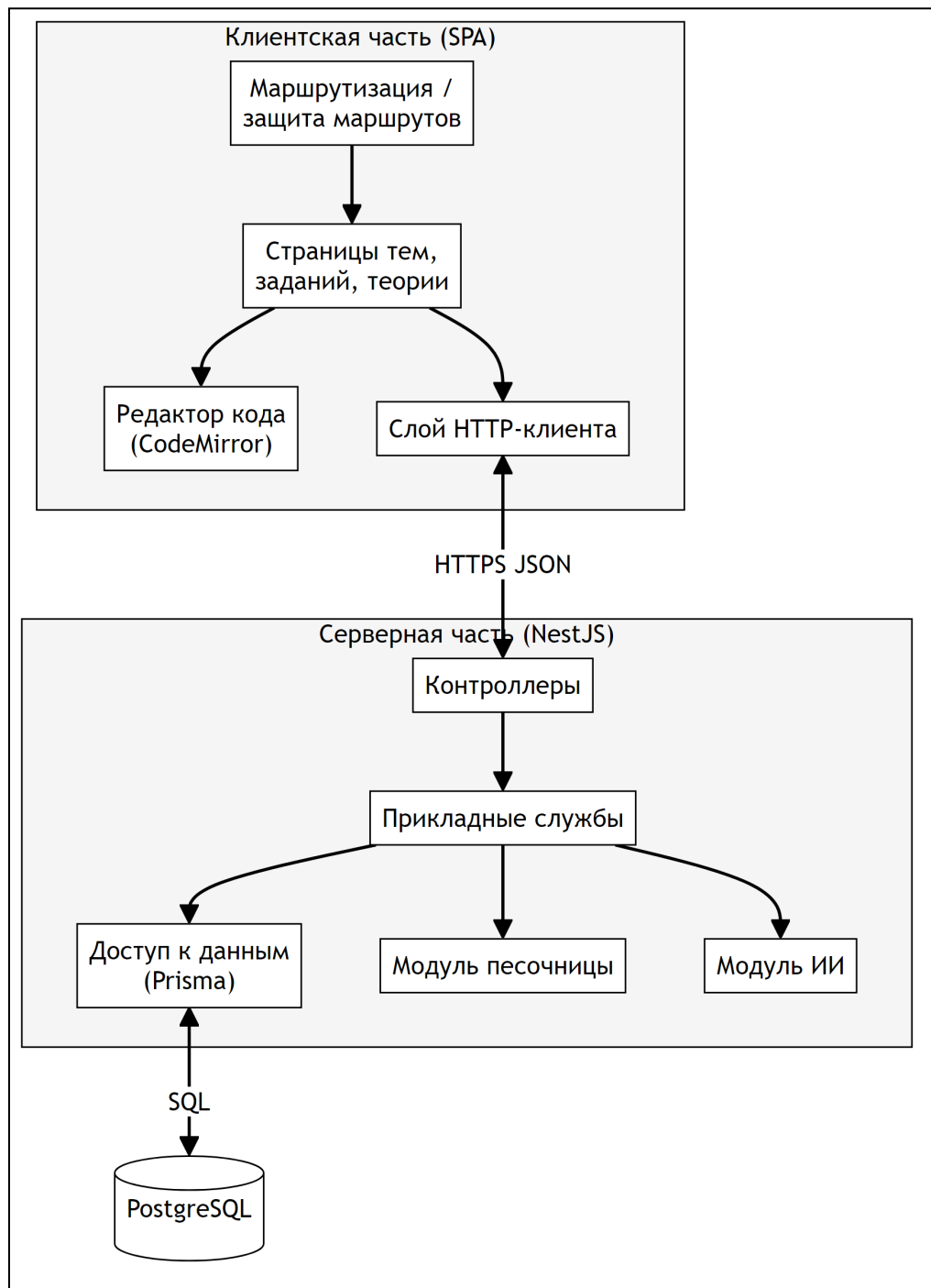


Рисунок В.6 – Укрупнённая диаграмма компонентов клиентской и серверной частей

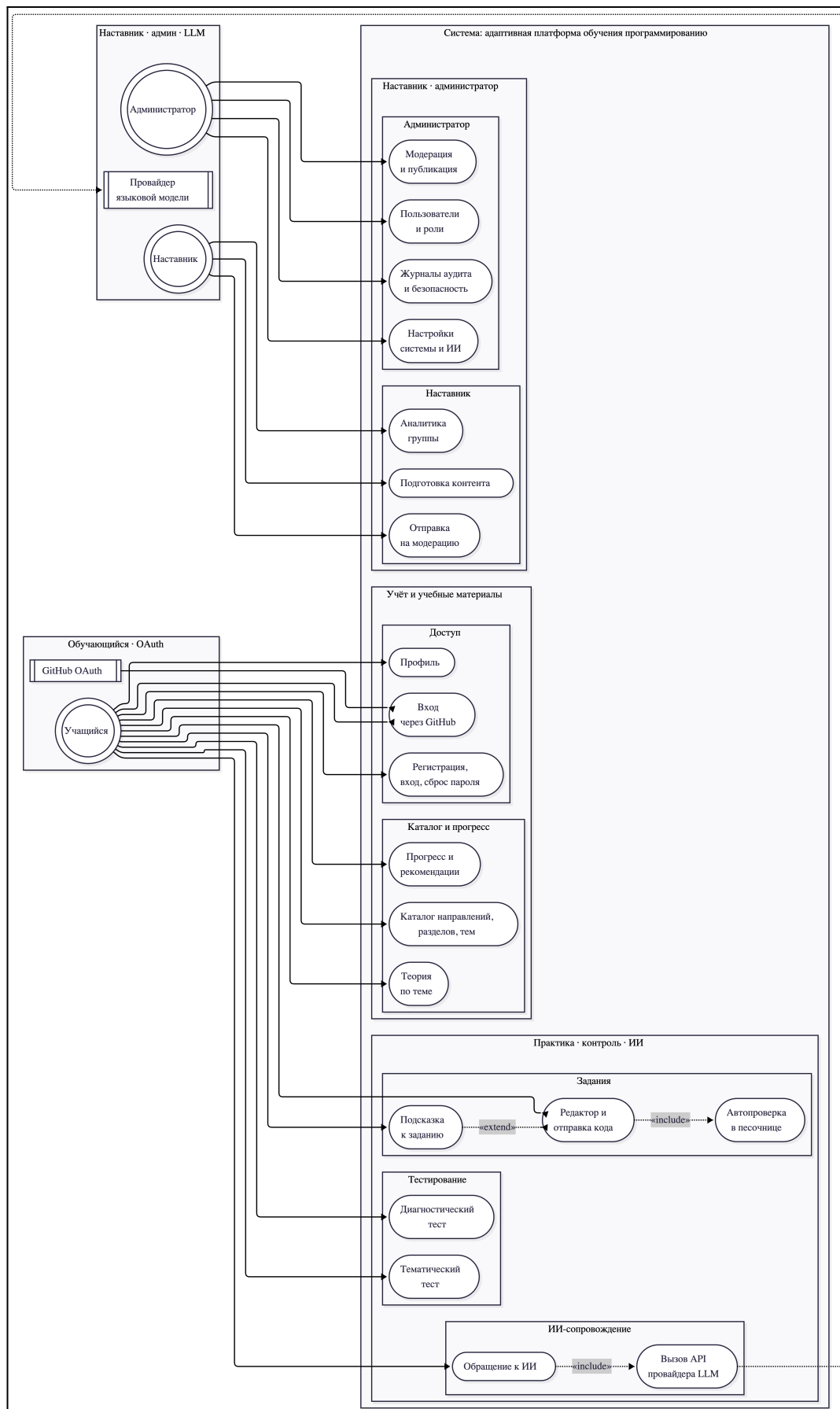


Рисунок В.7 – Диаграмма прецедентов UML разрабатываемой платформы

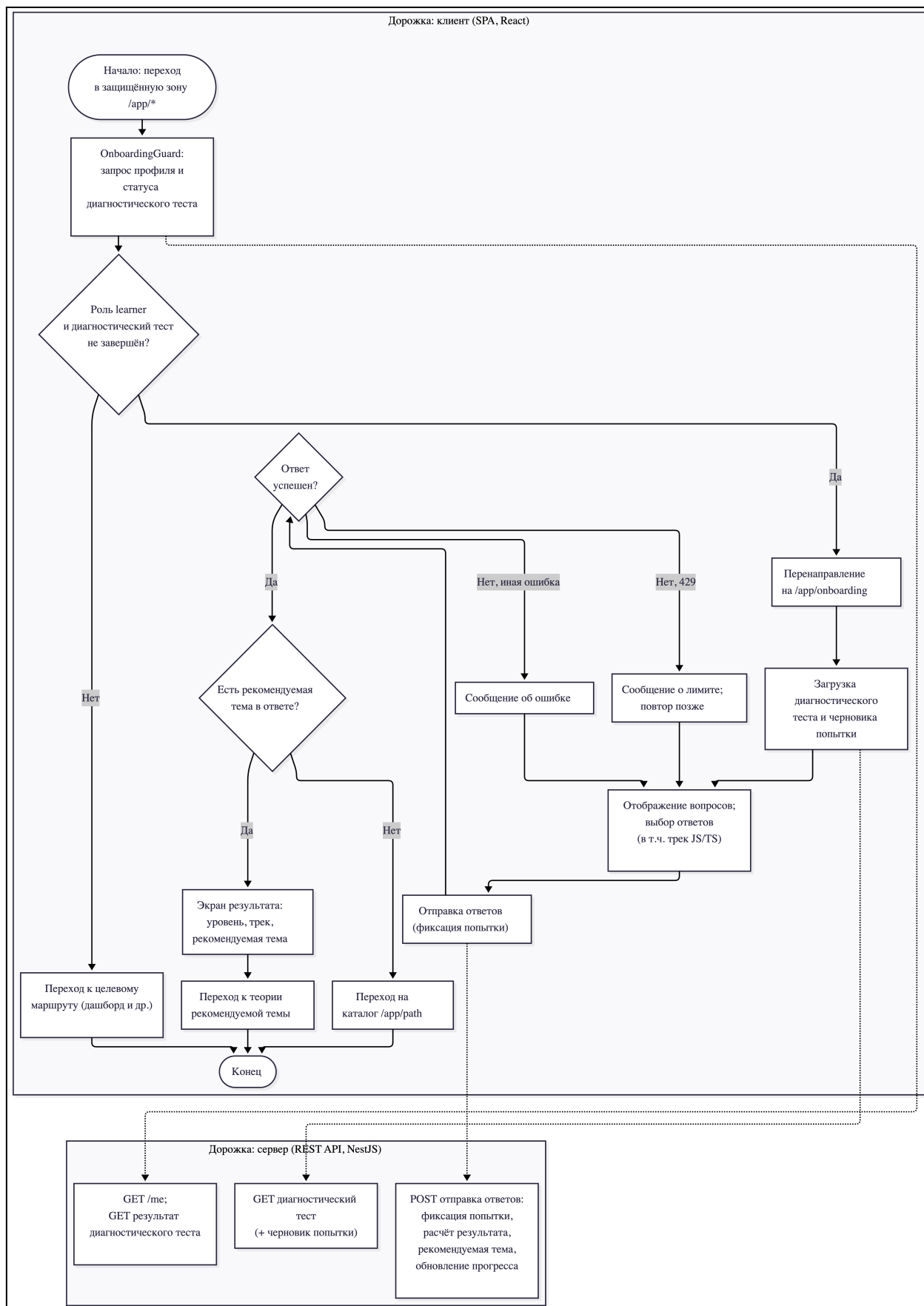


Рисунок В.8 – Диаграмма деятельности UML сценария диагностического теста (дорожки клиента и сервера)

ПРИЛОЖЕНИЕ Г

Листинги ключевых программных модулей

Листинг Г.1 – Точка входа модуля изолированного исполнения пользовательского кода (sandbox.service.ts)

```
1 @Injectable()
2 export class SandboxService {
3   run(userCode: string, testSpec: TestSpec): SandboxResult {
4     const validationError = this.validateTestSpec(testSpec);
5     if (validationError) return { status: 'failed', /* ... */ };
6     if (testSpec.language === 'html') return this.runHtml(userCode, testSpec);
7     return this.runJs(userCode, testSpec);
8   }
9 }
```

Листинг Г.2 – Пример тест-спецификации для автоматической проверки (JSON)

```
1 {
2   "language": "javascript", "functionName": "add",
3   "tests": [
4     { "input": [1, 2], "expected": 3 },
5     { "input": [0, 0], "expected": 0 }
6   ]
7 }
```

Листинг Г.3 – Фрагмент алгоритма рекомендации следующего шага обучения (adaptation.service.ts)

```
1 if (quizScore !== null && quizScore >= settings.quizMinScoreToAdvance) {
2   status = 'completed'; recommendedNext = 'next_topic';
3 } else if (quizScore !== null && quizScore >= settings.quizMinScoreTheory) {
4   recommendedNext = 'more_theory';
5 } else if (failedTasks > 0 && manyHints) {
6   recommendedNext = 'easier_practice';
7 } else if (hasPassedTask && failedTasks === 0) {
8   recommendedNext = 'next_topic';
9 } else { recommendedNext = 'repeat_topic'; }
10 await this.prisma.learningProgress.upsert({ /* ... */ });
```

Листинг Г.4 – Фабрика клиентов языковой модели: разрешение тайм-аута (llm-client.factory.ts)

```
1 const dbTimeout = await this.getSettingNumber('ai_timeout_ms');
2 const envTimeout = this.config.get<number>('AI_TIMEOUT_MS');
3 const defaultTimeout = provider === 'cloud' ? 15000 : 180000;
4 const timeoutMs = Math.max(1000, dbTimeout ?? envTimeout ?? defaultTimeout);
```

Листинг Г.5 – Фрагмент конфигурации развёртывания (docker-compose.prod.yml)

```
1 services:
2   postgres:
3     image: postgres:16-alpine
4     environment: { POSTGRES_DB: ${POSTGRES_DB:-learning_platform} }
5     healthcheck: { test: ["CMD-SHELL", "pg_isready -U postgres -d learning_platform"] }
6   redis:
7     image: redis:7-alpine
8     command: redis-server --requirepass ${REDIS_PASSWORD:-redis_secret_password}
9   ollama: { image: ollama/ollama:latest }
```

Листинг Г.6 – Фрагмент схемы данных ORM Prisma (schema.prisma)

```
1 model User {
2   id String @id @default(uuid())
3   email String @unique
4   passwordHash String?
5   role Role @default(learner)
6   taskAttempts TaskAttempt[]
7   learningProgress LearningProgress[]
8   aiRequests AiRequest[]
9 }
```